

kayfabe v3 — the architecture, the machine as it runs, and every constant we touch
A working document, assembled 2026-09-20 (w821). Written to be argued with.

branch w749-fable-legb

Contents

Read this first	4
What changed since the v2 bundle (w818)	4
The markers, used consistently throughout	4
Part 1 — The v3 architecture (the proposal)	5
0. □□□□ THE SEVEN AXES — every constant in this design varies along at least one	5
0.0.1 ∅∅∅ The Turing floor is an ARCHITECTURAL BOUNDARY, not a scoping decision	6
0.1 The axis band for (Dg, Dh) is a DIAGONAL, not a product	6
0.2 □□□ The axes are not evenly distributed across the four planes	7
0.3 ∅ What this forbids, concretely	8
1. The shape	8
2. The vCPU trap path	9
2.1 □□□□ The privilege boundary INSIDE the guest is the organising idea	9
2.2 The doorbell plane — a bit table, and the hint ring is DELETED	12
2.3 The privileged plane — one ordered ring, and per-vCPU rings were WRONG	14
∅∅∅ The mechanism, settled at w821 after an adversarial review	15
2.3.1 The design	15
2.3.2 □□□ The ordering hole across the queue boundary — found at w821, by neither of us	16
2.4 The wakeup protocol	16
2.5 The doorbell page is READ-ONLY, not emulated	17
3. Memory	18
4. Addresses	18
4.1 Mirror, never adopt — and the review’s reasons are stronger than mine	18
4.2 The diff	18
4.3 A VA space is the OBJECT. The PDB is an attribute of it, per GPU.	18
4.4 Where the root actually comes from — it is ours to write	19
5. Completions and interrupts	19
6. The scrub	20
7. Portability — derive, do not hardcode	20
8. What is deleted	20
9. Rewrite, not refactor	21
10. Still open — and what the owner ruled at w821	21
10.1 Ruled at w821 — closed, with where each ruling landed	21
10.2 Genuinely open	23
Part 2 — The surface, at constant level	24
0. The four planes, and why they are different shapes	24
1. GSP RPC — the plane we answer as the firmware	24
1.1 What this plane is, for a reader who does not know NVIDIA’s constants	24
1.2 The structures on the wire	24
1.3 The functions	25
1.4 □ The reply is always built from local state	28
2. RM objects and control commands — the plane that rides inside the RPC	28
2.1 What this plane is	28
2.2 The object classes we model	28
2.3 Control commands	30
2.4 □□□□ The synchronous verbs — the list the trap path is judged against	31
□□□ The two status-only verdicts hold — and they are the result worth keeping	32
∅∅∅ The “highest-value gap” was not a gap — retracted	32
□ The UVM gap closes — in the negative	32

3. BAR0 MMIO — reads from DRAM, writes to the vCPU	33
3.1 The asymmetry, and why it exists	33
3.2 The shadow — two structures, and they are not the same thing	33
3.3 The trapped-write register table	33
3.4 <code>WPR2</code> is the worked example of two axes crossing in one register	36
3.5 Whole apertures, not registers	36
4. Channels and pushbuffers — the plane unprivileged guest code drives	37
4.1 What a channel is, for a reader who does not know NVIDIA's constants	37
4.2 The three kinds — and an honest statement of what is built	37
4.3 The GPFIFO entry — 8 bytes	38
4.4 The method header — one dword	39
4.5 Every pushbuffer method we recognise	39
4.6 What happens when each kind's doorbell rings	41
5. <code>Where this inventory is knowingly incomplete</code>	41
5.1 Gaps in what we implement	41
5.2 Questions a survey could not answer	42
5.3 <code>Axis</code> coverage — the honest statement	43
5.4 What would close these	43
5.5 <code>What the review of THIS document found, kept as the record</code>	43
Part 3 — The Windows axis	44
0. <code>Why this is answerable at all — and exactly where the oracle stops</code>	44
1. <code>THE HEADLINE WAS WRONG — GSP is default-ON for Turing+, on every OS</code>	44
<code>And the registry key does not mean what the community advice implies</code>	45
<code>AND THE RETRACTION ABOVE OVER-CORRECTED — the struct IS defined, and it changes the answer</code>	45
<code>How I got the retraction wrong — I read a build with the branch compiled out</code>	45
<code>So what IS established, stated at the right strength</code>	45
1.1 <code>What the public record actually contains — searched w821</code>	46
<code>And the precise reason the claim is untested: it has no known-positive</code>	46
<code>How the survey got it wrong, and it is this tree's own documented failure</code>	47
<code>What replaces it, and it matters on every OS</code>	47
2. Under WDDM the OS owns the page tables — and we never see the writes	47
3. TDR — a hard, OS-enforced 2-second deadline	48
4. Four more that matter, briefly	48
5. <code>The escape hatch — TCC, and it collapses most of the table</code>	48
6. <code>What is the SAME — this bounds the work, and it is a lot</code>	49
7. <code>A free gift — detecting a Windows guest from the FIRST RPC</code>	49
8. <code>A correction to this project's own framing</code>	49
9. What could NOT be determined, and the first measurements to take	49
Part 4 — The machine, as it is	51
0. How to read this	51
2. Processes and threads	52
2.1 The processes	52
2.2 The sandbox — two stages, forced order	53
2.3 The threads	53
3. The vCPU trap path, step by step	54
3.1 How a guest access reaches Rust	54
3.2 A DOORBELL write (BAR0 0x00BB0090) — the hot path	54
3.3 An MMU_INVALIDATE write (BAR0 0x00B830B0) — the one the guest spin-polls	54
3.4 Back-pressure — the rule that makes deferral safe	55
4. Locks, and the blocking rules	55
4.1 The ranked order	55
4.2 The blocking rules — what is actually enforced	55
4.3 <code>The gate that is RED right now</code>	56
4.4 Where we block on purpose	56
5. Data structures	56
5.1 The object graph — identity vs label	56
5.2 The runtime model	57
5.3 The address table — what v2 deletes	57
5.4 Crate layering	57
5.5 The isolate boundary — two layers, not one	57
6. Installing a mapping, and observing a completion	58

6.1 The mapping chain, end to end	58
6.2 The single store	58
6.3 Completions — two mechanisms with opposite writers	58
6.4 Interrupts — what gets announced	59
7. The proposal, and the arguments against it	59
P1. Mirror, not adopt	59
P2. No address table; the host page tables are the model	59
P3. Lazy everywhere; strict only at recycle — and recycle is the guest’s job	59
P4. No RM verb on a vCPU, structurally	59
P5. Deletion order	59
P6. What must NOT be deleted	59
8. Open problems — named, not designed away	60
Part 5 — The surface we present (v2 plan-level)	61
1. GSP RPCs	61
1.1 The object-model RPCs — KEEP, unchanged	61
1.2 UPDATE_BAR_PDE (0x46) — KEEP, but it is the BAR2 root	61
1.3 SET_GUEST_SYSTEM_INFO (0x01) / _EXT (0x40) — KEEP	61
1.4 GET_GSP_STATIC_INFO (0x41) — KEEP, forged by construction	61
1.5 The inert set — KEEP as inert, they are load-bearing	61
1.6 Events we post — KEEP	61
1.7 Everything else — refused by name, and that is the design	62
2. RM control commands	62
2.1 Page-directory controls — [DELETE the concept, keep the decode]	62
2.2 GPU_PROMOTE_CTX (0x2080012b) — KEEP	62
2.3 Channel controls served by ObjectPolicy — KEEP	62
2.4 The 47 forged tables — KEEP forged, but they are a per-die liability	62
2.5 Input-only controls — KEEP	62
2.6 Denied by capability — KEEP DENIED	63
3. MMIO registers	63
3.1 Boot constants — KEEP	63
3.2 The GSP boot FSM — KEEP	63
3.3 Queue doorbells and the interrupt tree — KEEP	63
3.4 The MMU invalidate trio — [DELETE most of what it triggers]	63
3.5 The framebuffer windows — [DELETE]	63
3.6 The doorbell — KEEP, it is the hot path	63
4. Channel / pushbuffer operations	64
4.1 EMULATED — we implement the effect	64
4.2 TRANSLATED — operands rewritten, the GPU moves the bytes	64
4.3 REFUSED — the fault tripwire	64
4.4 PASSED THROUGH	64
5. □ Where this inventory is knowingly incomplete	64
Part 6 — The scrub question (open owner ruling)	66
What is measured	66
Why this is a decision and not a bug	66
The argument each way	66
If the ruling is “\$46 wins”, the work and its gate	66
What must not be read into this page	67

Read this first

This bundle is five documents, in the order they should be read:

1. **The v3 architecture** — the design I propose, rewritten after the owner’s review and after an adversarial review of the worker protocol. Short, and the part to attack first.
2. **The surface, at constant level** — *new in this edition*. Every GSP RPC, RM object class, RM control command, BAR0 register and pushbuffer method we touch, **by its NVIDIA constant name and hex value**, each with a plain-language sentence saying what it *is* for a reader who does not know NVIDIA’s constants, and what we do with it.
3. *new in this edition*. **The Windows axis** — *new in this edition*. The first survey of the os axis, which carried **zero coverage** until now. Read it before costing any Windows work: the headline is a **premise** problem, not a compatibility one.
4. **The machine, as it is** — what the code actually does today, from a `file:line` survey. Every place the code contradicts the proposal is marked.
5. **The surface we present (v2)** — the earlier plan-level inventory: KEEP / DELETE per item. Kept because the *plans* in it are still the plans; document 2 supersedes its *data*.
6. **The scrub question** — one open owner ruling, isolated because it is a live security defect and it is the smallest change on the list.

What changed since the v2 bundle (w818)

- **A second privilege boundary.** [owner] The doorbell is adversarial **to the guest’s own root**, because unprivileged guest userspace writes it. This is not hardening bolted on — it restructures the entire vCPU trap path. v3 §2.
- **∅ The doorbell hint queue and every FULL_REFRESH-shaped fallback are DELETED**, replaced by a 64 KiB bit table. Deleted from the *proposal*, before any code was written.
- **∅ Per-vCPU register rings are refuted** — they preserve per-vCPU order where the device needs the guest’s cross-vCPU order.
- **□ The worker wakeup protocol is fully specified**, with its bit widths, its layout, its memory orderings and the interleavings that break it.
- **□ This document, part 2** — the constant-level reference the v2 bundle did not have.

The markers, used consistently throughout

marker	meaning
[MEASURED]	a number from a real boot, with the run that produced it
[CODE]	read out of the tree, with <code>file:line</code>
[ogkm]	read out of NVIDIA’s open kernel modules, with <code>file:line</code>
[PROPOSE]	a design opinion. These are the arguable ones.
[UNVERIFIED]	the survey could not establish it. Deliberately not filled in with a guess.
∅	a refusal, a deletion, or a thing that is <i>not</i> true
□	a load-bearing fact
□	a trap, or a claim whose evidence is weaker than it looks

□ **Nothing in this bundle is filled in from memory of NVIDIA’s headers.** Every constant name, number and structure layout is cited to a file and line in either this tree or the open kernel modules under `research_clones/ogkm`. Where a survey could not establish something, it says **[UNVERIFIED]** rather than guessing — that convention is load-bearing, because this project’s most expensive recurring failure is a plausible value that was never measured.

Part 1 — The v3 architecture (the proposal)

STATUS: LIVE, 2026-09-20 (w820). **Supersedes** `THE_ARCHITECTURE_v2.md`, which this document keeps only as the record of what was proposed before the review corrected it.

∅ **w820 rewrote §2 entirely** around a second privilege boundary the owner named: the one **inside the guest**, between unprivileged guest userspace and guest root. The doorbell hint queue and its “inspect all doorbells” fallback are **deleted**, not fixed; per-vCPU register rings are **refuted**; the worker wakeup protocol is specified with its orderings. §§1, 8 and 10 were updated to match. Everything else is unchanged from w819.

∅ Written from the owner’s point-by-point review. Where the review corrected me, the correction is marked **[CORRECTED]** with what I had wrong — those are the load-bearing edits, not footnotes.

0. □□□□ THE SEVEN AXES — every constant in this design varies along at least one

[NEW, w821] [owner, 2026-09-20] listed the axes this product must span. They are the reason almost nothing in Part 2 may be a literal in our source, and they are **prior to** every other section: a design that is correct on one point of this lattice and cannot be extended to the rest is a retrofit waiting to happen.

tag	axis	posture	how variation is absorbed
K	guest kernel version	∅ all major versions	∅ [CORRECTED w821] <i>not</i> via struct layouts — RM and UVM structs are NVIDIA-defined and kernel-independent. What K actually moves that we see is DMA addressing : a guest booted with a vIOMMU hands us IOVAs , not GPAs , in every system address we are given. = $K \times V$, and unhandled
Dg	guest driver version	∅ all major versions	generated from that version’s headers; a per-version profile entry , never if version ==
Dh	host driver version	∅ all major versions, decoupled from Dg	we author every host call; our host-verb signatures do not accept a guest flag word
A	GPU architecture	□ Turing and newer — ∅ FORCED, not chosen (below)	a format family descriptor (four families span Turing→Blackwell)
die	GPU die within an arch	∅ any die	derived per die, maintained per family ; a new die is a descriptor, not a code path
V	VMM	Ⓞ the one axis where a version floor is legitimate — QEMU, Cloud Hypervisor	the core is VMM-agnostic; the VMM shim is the only place that knows

tag	axis	posture	how variation is absorbed
OS	guest OS	Linux and Windows	⊙ [SURVEYED w821 — see Part 3.] No longer zero-coverage. ⊙ GSP is default-on for Turing+ on every OS (the shared core has no OS branch) — an early claim that Windows defaults it off was retracted. What does bite: under WDDM the OS owns the page tables and the PDE-update path is never RPC'd to us; TDR is a hard 2 s vs Linux's 4/30 s; UVM does not exist. □ TCC collapses most of it, and bGspNocatEnabled is a one-comparison Windows detector

⊙ **Dg and Dh are TWO axes, not one.** The operator chooses the guest driver; we do not. A design that assumes they match is a defect. □ This is the structural reason for `author_host_flags_never_forward_them`: forwarding a guest-supplied flag word into a host RM call is not merely risky, it is **an implicit assertion that the two versions agree**.

⊙ **Host OS is Linux only** — the one scope relaxation, taken deliberately.

0.0.1 ⊙⊙⊙ The Turing floor is an ARCHITECTURAL BOUNDARY, not a scoping decision

[w821] [owner] “*pre-Turing it can't be used.*” □ Correct, and it reclassifies the A-axis floor. `_gpumgrIsRmFirmwareCapableChip` is exactly:

```
return (decodePmcBoot42Architecture(pmcBoot42) >= NV_PMC_BOOT_42_ARCHITECTURE_TU100);
```

⇒ **Below Turing there is no GSP at all.** A pre-Turing guest driver does not speak this protocol *at any level*: it runs **monolithic RM** and drives the hardware directly, with no command queue, no RPC envelope, and no firmware to impersonate. □□□ So “*we are the GSP*” is not merely harder there — **it is meaningless**, and no amount of engineering moves the floor.

⇒ The floor should be read as “*the architecture does not exist below Turing*”, not as “*we chose to start at Turing*”. □ Everywhere else in this document a posture is a decision that could be revisited under a product argument (`support_matrix_asymmetry`); **this one cannot be**.

□ **And it separates two gates this document had been blurring:**

gate	what it tests	where
capability	⊙ hardware — is there a GSP on this die at all? <code>arch >= TU100</code>	<code>_gpumgrIsRmFirmwareCapableChip</code>
default-on policy	⊙ within capable hardware, should firmware be used by default? On Windows this is per-SKU (<code>devId, ssId, WS/server, TCC/MCDM</code>)	<code>gpumgrIsDeviceRmFirmwareCapable + WindowsFirmwarePolicyArg</code>

⇒ The Windows uncertainty in Part 3 §1 lives **entirely inside the second gate**. It has no bearing on the first, and therefore none on our floor.

0.1 The axis band for (Dg, Dh) is a DIAGONAL, not a product

[owner ruling, 2026-08-09] We owe NVIDIA vGPU's own interoperability policy and no more: exact match; same major branch different minor; and **n-1** (a newer host supports the previous major or LTS branch). ⇒ A bounded window, not every pair.

□ What that **obliges us to build**: we must know *both* versions and be able to say “*this pair is outside the band*” — **refused by name**. ∅ A compatibility policy that cannot state an out-of-band pair is not a policy, it is a hope, and it decays into the 0x56 is the forgiven status trap where a wrong configuration simply runs on.

∅ **[CORRECTED w821] The band is not 2-D**. An $n-1$ guest driver must also **support the die**, or it never binds at all — so the refusal key is (Dg, Dh, die), not (Dg, Dh). □ And a third interaction the section missed: Dg × A — the guest supplies its *own* firmware image and protected-region metadata, whose layout our fake GSP must accept. That is a guest-driver artifact whose shape is architecture-dependent, and it reaches us as data we must parse.

0.2 □□□ The axes are not evenly distributed across the four planes

This is the useful part, and the surveys in Part 2 measured it rather than assuming it:

plane	dominant axis	evidence
GSP RPC function numbers	□ stable across Dg in the measured window	the NV_VGPU_MSG_FUNCTION_* enum is byte-identical between 580.159.04 and 610.43.02 for every id we use; 610 only <i>appends</i> (rpc_global_enums.h)
∅ GSP RPC <i>payload structs</i>	∅ Dg, and they break	[CORRECTED w821 — the row above was true of the wrong population, which is this document’s own named failure mode.] The <i>id</i> is stable; the <i>body</i> behind it is not. rpc_alloc_object_v2B_04 → v2C_01, rpc_map_memory_dma_v03_00 → v2C_05, 1 895 changed lines in generated/g_rpc-structures.h. ⇒ Same posture as RM controls, not a stable plane
GSP RPC message framing	∅ Dg, and it BREAKS	the per-element header is 48 bytes at 580 (authTag/aad/checkSum@32/seqNum@36/elemCount@40) and 16 bytes at 610 (mctpHeader/nvdmHeader/checkSum@8/seqNum@12) — message_queue_priv.h:43-51 VS :52-67
RM control command numbers	Dg (additive), struct layouts break	FINN-generated; the <i>number</i> is stable, the <i>parameter struct</i> is versioned
BAR0 register offsets	A and die	NV_PGSP_QUEUE_HEAD = 0x110c00 from ampere/ga102/dev_gsp.h; the doorbell is 0x30090 on Turing, Ampere and Blackwell
∅∅∅ Which BAR carries the doorbell	A — and it changes which trap plane exists	[MISSING until w821, and it is the worst kind of omission: an axis that moves the mechanism, not a constant.] Ampere/Ada: BAR0. Hopper+: unprivileged clients map it over BAR1 (bBar1Mapping, not privilege-gated), which v3 §2 does not trap. See §2.1
Doorbell token encoding	∅ die, within one arch	NV_CTRL_VF_DOORBELL field widths are per-arch swref, and GB202 sets bit 30 where Ampere does not
Channel / engine class ids	A, ∅ and die	TURING_CHANNEL_GPFIFO_A 0xC46F (the floor arch) → AMPERE_ 0xC56F → HOPPER_ 0xC86F → BLACKWELL_ 0xC96F. ∅ [CORRECTED w821] not purely per-arch: BLACKWELL_CHANNEL_GPFIFO_B 0xCA6F exists on GB202/203/205/206 while GB100 has only _A — die-level inside one architecture

plane	dominant axis	evidence
Pushbuffer method encoding	□ remarkably stable	NVC56F_DMA_SEC_OP/METHOD_ADDRESS/_-SUBCHANNEL/_COUNT is bit-identical to NVC36F_*; the GPFIFO entry differs only in PRIV (dropped at C56F) and INVAL_SCOPE (added)
Page-table format	○ A , four families	Turing binds kgmmuFmtInitLevels_GP10X; our one built format is _GA10X

⇒ □□□ **The two planes that break hardest are the two the guest drives most:** RPC framing (Dg) and page-table format (A). Both are already modelled as **descriptors** rather than code paths — ElementLayout { hdr_size, checksum_off, seqnum_off, elem_count_off, TransportHdr } and GmmuFmt — and that is the shape every other axis-carrying fact should take.

□ **The 580/610 element header is the worked example to argue from.** It is a structure whose *size and every field offset* changed between two supported guest driver versions, and absorbing it as data rather than as a branch is the right shape.

○○ [CORRECTED w821 — but the descriptor as it stands does NOT absorb the pair it cites, so the example proves less than I claimed.] Three things it cannot express: elemCount **no longer exists** at 610 (the count is derived from length), so an elem_count_off: Option<usize> models its absence but not its *replacement*; hdr_size is **runtime-conditional**, not per-version — Confidential Compute adds an encryption tag to every element header; and 610 **validates** the mctpHeader version and nvdmHeader vendor on every element it receives, which is behaviour, not layout. ⇒ The honest claim is that the descriptor absorbs *offsets*, and that a Dg break can also move **derivation** and **validation** — which a layout descriptor cannot hold at all.

□ **And a Dg break that is neither layout nor number:** 610 reads a GSP-RM **heartbeat** from a mailbox register when an RPC times out, to classify the failure; 580 has no such path. ⇒ A slow worker gets a *different and fatal* diagnosis at 610. **A version axis can add a new way to be judged, not just a new way to be parsed.**

0.3 ○ What this forbids, concretely

1. ○ **No literal in a comparison.** derive_per_die_maintain_per_family. Nothing may test a raw offset, class id or command number against a hardcoded constant.
2. ○ **No C parsed with regex or sed.** Generate from the headers with a real parser front end; [owner] “*That will age better.*”
3. ○ **No capture-derived table treated as truth.** A table captured from one driver on one die **expires as a vendor regression** — and this tree measured that 11 of 56 captured rows were empty and *every empty one checked against hardware was contradicted*.
4. ○ **No guest flag word forwarded to a host call.** It is a Dg↔Dh coupling in disguise.
5. ○ **No “Linux-shaped” assumption left unmarked.** Until a Windows guest runs, every guest-side inference in this document is **os-limited**, and Part 2 marks the ones that are.

□ **Where this document is weakest on the axes**, stated plainly so it is not discovered later: the constant-level tables in Part 2 were surveyed against **Ampere GA10x with a Linux guest**. The axis column in each table says what *should* vary; it does **not** certify that each entry has been exercised on a second point of that axis.

1. The shape

One process. The VMM (QEMU, or later any hypervisor) with the kayfabe core linked in. No isolate children, no scratchpad process, no IPC, no sandbox plane of our own.

Two kinds of thread besides the vCPUs:

thread	count	job
worker	N, capped at 255	the doorbell plane: scan the token table, claim a token, run the managed channel’s work

thread	count	job
register drainer	1	□ [NEW w821] drains the privileged MPSC ring in reservation order — GSP RPC kicks, interrupt-tree writes, TLB-invalidate triggers. ∅ It must be <i>one</i> thread: “an ordered ring drained by N workers” is not ordered
VA manager	1	□ <i>new</i> . One synchronous thread doing all mmap/munmap — page-table diffs, BAR windows, channel mappings. It replaces the scratchpad isolate.

□ **[w820]** The worker cap is **structural, asserted at startup** — not a configuration convention. `workers_polling` is 8 bits of `worker_vcpu_poll` (§2.4) and a wrap would read **0 while workers are polling**, which is the lost wakeup the protocol exists to prevent.

□ The VA manager exists because mapping is the one activity that must be serialized and must never happen on a vCPU. Giving it a thread rather than a lock makes “who may map” a structural answer instead of a rule.

∅ **Sandboxing is not ours.** Sandbox the whole hypervisor process — a container, or the hypervisor’s own jail. [owner, 2026-09-20] “*most security bugs didn’t even come from the isolates... the entire isolate plane is a new plane that needs sharing fds and handles and stuff back and forth... It by itself is a source of many bugs.*” □ This session supplied the proof: `barmirror.rs:1988` does a **cross-process IPC round trip on a vCPU while holding a lock** — a defect that cannot exist without the isolate plane. Add the 1.62 s spawn stall, *Wedge* (a failure mode that exists only because there is a socket), the fd passing, cancel-by-signal and the foreign-handle gate, and the plane’s bug surface exceeds what it protects.

∅ **Proc stops existing. [CORRECTED]** A process is not a GPU concept. v3 speaks only in **channels, VA spaces** and **RM objects**. With isolates gone the whole per-proc container, its arena, its pending-release queue and the dup-connected-component analysis go with it.

2. The vCPU trap path

Only **BAR0**, only **writes**. BAR1/BAR2 are never trapped — ensuring that is a requirement, not an optimisation. The entry point is one function, `nvkvm_trap_bar0_write`.

2.1 □□□□ The privilege boundary INSIDE the guest is the organising idea

[NEW, w820] [owner, 2026-09-20] “*the doorbell is adversarial also to the guest itself, since unprivileged guest userspace writes to it (that’s below guest kernel) ... It’s not about the guest corrupting itself from the kernel/root in the guest, it’s about a sandboxed/container process in the guest corrupting the root in the guest.*”

This is not hardening bolted onto the design. It **splits the trap path in two**, and every property each half needs is derived from which side of the line it sits on:

plane	who can write it	what it may therefore be
doorbell (usermode window)	unprivileged guest userspace	fixed-size, idempotent, non-degradable, needs no identity ⇒ a bit table, no queue
privileged registers (everything else in BAR0)	guest root only (RM)	an ordered ring that must not overflow — a guest root that overflows it harms only itself

⇒ The test for any future trapped page is one question: **can unprivileged guest userspace write it?** If yes it gets no queue, no degradation mode and no shared structure. If that is not achievable for some page, we do not map it.

Guest-root-corrupts-guest-root is inside the guest’s own trust model and not ours to defend. Guest-container-corrupts-guest-root is the value proposition.

Why no check at the trap can fix this — three facts from ogkm

1. **The doorbell page is ONE page per GPU, aliased to every client.** `usrmodeConstruct_IMPL` takes `pMemDesc = pKernelFifo->pRegVF` (`usrmode_api.c:47`) — a single per-GPU `ADDR_REGMEM memdesc` (`kernel_fifo_gv100.c:370`), 64 KiB (`NVC361_NV_USERMODE_SIZE`), doorbell at `+0x090`. `usrmodeCanCopy` is `NV_TRUE`, so it is **DUP'd, not copied**. There is **no per-client and no per-channel doorbell page on Ampere**. Hopper adds `bPriv`, gated on `RS_PRIV_LEVEL_KERNEL`, but that selects `BAR1-vs-BAR0 mapping`, not which channels you may ring. □ RM waives its own privilege check on this window **by name**: `memdescSetFlag(pRegVF, MEMDESC_FLAGS_SKIP_REGMEM_PRIV_CHECK, NV_TRUE)` (`:374`). Handing a `BAR0` register window to unprivileged userspace is deliberate, documented NVIDIA behaviour.
2. **The token carries no capability.** `kfifoGenerateWorkSubmitTokenHal_GA100` (`kernel_fifo_ga100.c:224`) is a plain concatenation of `runlistId` and `chId`. RM's comment "*Caller cannot make assumption about this handle*" is a request, not an enforcement: the space is small enough to enumerate by counting.
3. **The trap carries no attributable identity**, and this is the load-bearing claim, so it is cited rather than asserted. We get a vCPU, a GPA and 32 bits the attacker chose — nothing says *who* rang. □ **The two mappings provably coincide**: the usermode window's address is a **BAR0-relative register offset** (`kfifoGetUsermodeMapInfo_GV100` returns `gpuGetRegBaseOffset_HAL(NV_REG_BASE_USERMODE)`, `kernel_fifo_gv100.c:165`), and the driver's own mmap gate admits it precisely *because* it lies inside the register `BAR` — `IS_REG_OFFSET` tests `offset >= nv->regs->cpu_address` and within `nv->regs->size` (`kernel-open/common/inc/nv.h:854`), where `nv->regs->cpu_address` is `BAR0`'s physical base, which under KVM is a **guest-physical** address. The guest kernel reaches the same registers by `ioremap` of that same base. ⇒ Both mappings land in **one KVM memslot at one GPA**. ☐ **[CORRECTED w821 — my reasoning here was wrong, though the conclusion survives.]** I wrote that the trap "*cannot distinguish kernel from userspace. Not with more checks. Ever.*" **False**: an MMIO exit hands us the vCPU, and its **CPL is readable** (`KVM_GET_SREGS`, $\sim 1\ \mu\text{s}$). Kernel-vs-user is distinguishable. ⇒ The load-bearing unavailability is not privilege, it is **process identity**: nothing tells us *which* of the guest's unprivileged processes rang, and that is what an authorization decision would need. The conclusion — the ring cannot be authorized at the trap — holds, for that reason and not for the one I gave. □ And CPL is not free to consult on a hot path; it is an argument about what is *knowable*, not a proposal to read it per doorbell.

☐☐☐ **What a hostile ring actually buys — and it is LESS than w820 claimed [CORRECTED w821 by the owner, and the correction matters because it moves the threat model off the wrong mechanism onto the right one.]**

w820 argued that a hostile ring "*promotes the attacker's timing hint into a control input over victim-state processing.*" ☐ **That overstates it.** [owner]:

"it doesn't have access to userd or ring or GP_GET/GP_PUT, so it's a harmless check to see if there is work and then there isn't. That's just cycles, just like the real GPU costs cycles to inspect a doorbell."

□ **Correct, and the mechanism is worth stating exactly.** A worker that takes a spuriously-rung token reads the victim's `GP_PUT` and compares it to our cursor. If the victim has not submitted, `GP_PUT == cursor`, **there is no work, and we do nothing**. The attacker supplies a 32-bit token and nothing else; every byte we then act on comes from memory only the victim can write. ⇒ Our cost profile is hardware's: **ring anything, pay a redundant look**.

⇒ **The double-take is therefore a CORRECTNESS boundary, not a security one**, and w820 was wrong to promote it. Two *legitimate* rings from the victim itself race exactly the same way — the attacker only makes the window easy to hit. □ The four-state token word in §2.4 is still **mandatory**; what changes is *why*. It is owed to the victim's own concurrency, not to an adversary, and a design that justified it by the adversary would have deleted it the moment the adversary was ruled out.

□ **The one residual, named rather than claimed absent**: the passthrough branch and the managed branch differ in cost (an inline MMIO write versus two `fetch_orS`), so ring timing is a weak oracle for "*does a channel exist at this token, and is it passthrough.*" ☐ Both branches are $O(1)$ with no data-dependent work on victim state, so there is no timing channel into anything the victim owns. [owner] "*barely any meaningful timing of the MMIO trap can be observed.*" Agreed; recorded so the claim is bounded rather than absolute.

□□□ **So the threat is not what we parse — it is what an unprivileged write can REACH** [owner, and this is the sharp version of the whole section]: the hazard is not that an unprivileged process makes us look at a victim's channel. It is that an unprivileged write, landing on a page guest userspace can map, **must not be able to reach the privileged plane or exhaust anything**. Two rules follow, and they are the real content of §47:

- ☐ **A degradation path is an amplifier.** Any "*queue full = scan everything*" fallback lets an unprivileged process force work on the attacker's schedule. Deleted (§2.2).
- ☐☐☐ **Any offset in a guest-userspace-mappable page that is NOT the doorbell must return immediately, doing nothing.** See §2.2 — this was a live hole in the w820 design and the owner found it.

000 THE HOPPER+ BREAK — on Hopper and Blackwell the doorbell is written through BAR1 [NEW w821, from an adversarial review. This is the most serious defect found in v3, and it is an architectural break, not a wording problem.]

v3 §2 opens “BAR1/BAR2 are never trapped — ensuring that is a requirement.” 0 On Hopper and newer that requirement silently discards work submission.

[logkm] usermode_api.c:63-98: for hClass >= HOPPER_USERMODE_A the allocation takes bBar1Mapping from the caller, and when set, pMemDesc = pKernelFifo->pBar1VF with memClassId = NV01_MEMORY_SYSTEM — a **BAR1** window onto the VF register block (kernel_fifo_gh100.c, MEMDESC_FLAGS_MAP_SYSCOH_OVER_BAR1), placed wherever the **guest’s** RM writes a BAR1 PTE for it.

000 bBar1Mapping is **NOT** privilege-gated. Only bPriv is (!bPrivMapping || privLevel >= RS_PRIV_LEVEL_KERNEL). Any unprivileged client may ask for the BAR1 form.

And both of NVIDIA’s own consumers do, with bPriv = NV_FALSE:

- **UVM** — nv_gpu_ops.c:5644: .bBar1Mapping = NV_TRUE, .bPriv = NV_FALSE when isDeviceHopperPlus(device).
- **nvidia-push** — nvidia-push-init.c:968, carrying NVIDIA’s own explanation: > “The BAR1 mapping is used for (faster and more efficient) writes to perform work submission, > but can’t be used for reads. If we ever want to read from the USERMODE region (e.g., to read > PTIMER) then we need a second mapping.”

⇒ Four consequences, and the first is a product failure:

1. 000 **Every UVM doorbell — and, on this evidence, every CUDA doorbell — is lost on Hopper+**, silently, because it lands on a BAR1 page we deliberately do not trap.
2. 0 §2.1 **fact 3 is Ampere/Ada-scoped**. On Hopper+ the guest **kernel** still rings BAR0 (RM’s own CeUtils channel allocates VOLTA_USERMODE_A and rings via GPU_VREG_WR32) while **userspace rings BAR1** — different pages, different GPAs.
3. 0 §2.1 **fact 1 mis-describes bPriv**. It selects the NV_VIRTUAL_FUNCTION_PRIV region (pBar1PrivVF, its own doorbell at NV_VIRTUAL_FUNCTION_PRIV_DOORBELL 0x2200), not “BAR1-vs-BAR0”. bBar1Mapping does that. ⇒ There are **three** usermode memdescs on Hopper+ (pRegVF, pBar1VF, pBar1PrivVF), not one.
4. 0 §0.2 **has no row for which BAR carries the doorbell**. It is an A-axis variable that changes **which trap plane exists at all** — the most consequential kind of axis variation, and the one §0 was written to catch. It did not.

00000 **And the inversion, which is the genuinely good news and changes what is possible**. On Hopper+ the privilege separation the owner asked for **exists in hardware, for free**: the unprivileged doorbell is a *distinct page at a distinct GPA* from the kernel’s. ⇒ The literal form of §47 — “do not register write traps in the unprivileged page” — which §2.1 records as unreachable on Ampere **becomes reachable on Hopper+**, because there the two planes are already separate. 0 The architecture should be built so that separation is *exploited where it exists* rather than assumed absent everywhere.

0 RULED, w821 — take (a), and the constraint is lightly lifted [owner] “nested vGPU is unsupported, so there is only 1 doorbell page possible, and we map the right one from host userspace to the correct BAR1 MMIO. For that still KVM memslot read-only. Would then allow BAR1 trapping, but only on that doorbell page, nothing else. Constraint lightly lifted.”

000 [CORRECTED w821 — I wrote a HYPOTHESIS as an established motive and then leaned on it.] The owner offered, and explicitly flagged as “just a hypothesis, not proven”, that the BAR1 form exists so each **vGPU / MIG partition** can own its own doorbell page. I recorded it as “the owner’s reading of NVIDIA’s motive is the load-bearing simplification” and derived “there is exactly one doorbell page” from it. 0 **Both moves were wrong**: the motive is unproven, and a design decision must not rest on why a vendor did something.

⇒ **The question re-asked without the motive: how many doorbell-bearing apertures can exist?** That is answerable directly, and the answer is **not one**.

kfifoConstructUserModeMemdescs_GH100 (kernel_fifo_gh100.c:113-132) builds **two** memdescs in a loop, then calls the Volta constructor which builds a **third**:

memdesc	aperture	VF-relative range (GH100)	size	privilege to obtain
pBar1PrivVF	BAR1 (ADDR_SYSTEM, MAP_SYSCOH_OVER_BAR1)	NV_VIRTUAL_FUNCTION_PRIV 0x00000-0x2FFFF	192 KiB	0 kernel (bPriv requires RS_PRIV_LEVEL_KERNEL)
pBar1VF	BAR1 (same flags)	NV_VIRTUAL_FUNCTION 0x30000-0x3FFFF	64 KiB	0 none — bBar1Mapping is ungated
pRegVF	BAR0 (ADDR_REGMEM)	the VF register block	64 KiB	none

⇒ □ **Up to three doorbell-bearing apertures per GPU, not one** — and the doorbell lives at +0x90 inside NV_VIRTUAL_FUNCTION, i.e. in the **first 4 KiB page** of the 64 KiB unprivileged BAR1 mapping.

□□□ **What DOES survive, and it is the fact the design actually needs:** each of those is **constructed once per GPU, at init** — `pKernelFifo->pBar1VF` is a single memdesc that every client which asks for the BAR1 form maps. ⇒ The count of *doorbell objects* is bounded and known without appealing to anyone’s motive.

○ **Still open, and it is the one that decides “one page”:** whether multiple clients mapping that **same** memdesc land on the **same guest-physical BAR1 page** or get separate BAR1 addresses. `memdescAddRef` suggests sharing, but that is inference, not a read of the mapping path. **Not checked.** §10.

□□□□ **And the reason this no longer matters much — the mechanism was never really resting on it.** We identify the page **by RM object handle** (below), not by “*there is only one.*” ⇒ If it turns out there are N, we trap N; the classifier keys on (`bar`, `off`) from a generated descriptor set either way. **The hypothesis was load-bearing only for the rhetoric, not for the design** — which is precisely why writing it in as settled fact was worth catching.

The revised rule, replacing “BAR1/BAR2 are never trapped”:

BAR1 is never trapped, with exactly one exception: the single page the guest maps the usermode object at. That page is a read-only KVM memslot over the real host doorbell page — reads pass through to hardware, writes trap into §2.2. Nothing else in BAR1 is trapped, ever.

○ This is a **lightening**, not a widening: the trapped set grows by **the pages that carry a doorbell and no others** — one on Ampere/Ada, and on Hopper+ the first page of the 64 KiB unprivileged BAR1 window (plus the privileged one **only if** we ever serve `bPriv`, which requires guest kernel privilege and which we have no reason to grant). It stays inside the shape §2.5 already uses for BAR0’s doorbell page.

□ **The other 60 KiB of that BAR1 window is guest-userspace-mappable and carries no doorbell.** ⇒ It goes through R2’s **middle arm** — trap, return immediately, do nothing — for exactly the reason §2.2 gives. ○ Passing it through writable would hand unprivileged guest userspace a direct write path to real VF registers.

How we find that page — and there is a better answer than pattern-matching PTEs. `[owner]` observed that the VA manager’s PTX-derived diff for BAR1 already carries the mapping, so the page is visible to us. □ **[PROPOSE]** but the diff should be the **confirmation**, not the identification:

1. □ **Primary — the RM side, which is unambiguous.** We serve `GSP_RM_ALLOC` for `HOPPER_USERMODE_A` (§2.2 of Part 2), so we already hold the object handle **and** the `bBar1Mapping` flag the guest asked for. When that object is subsequently mapped, we know which mapping it is **by handle**, with no inference.
2. ○ **Confirmation — the BAR1 diff.** The PTX walk reports the page; we assert it matches the handle-derived answer. ○ A disagreement is a **refusal**, not a tiebreak.

□ Why not the diff alone: identifying a page by “it looks like the VF register block” is a **pattern match on guest-chosen data**, and a hostile guest chooses that data. The handle is ours.

What this adds to the descriptor set: `doorbell_bar` and `doorbell_off` become generated per-arch fields, so the trap classifier in §2.2 keys on (`bar`, `off`) rather than on `off` — which is why its pseudocode now takes `bar` as a parameter.

□ **Still open (§10):** what happens on Hopper+ when the guest allocates the usermode object **without** `bBar1Mapping` (the BAR0 form is still legal there), and whether both forms can be live at once in one guest. ○ Not yet checked.

□□□ **And the compensation, restated because it changes what is achievable:** on Hopper+ the guest kernel rings **BAR0** while userspace rings **BAR1**. The two privilege domains are at **different pages, different GPAs** — so §47’s literal form, “*do not register write traps in the unprivileged page*”, unreachable on Ampere, **is reachable there**. ⇒ The classifier should exploit that separation where the hardware provides it rather than assume it absent everywhere.

□ **The tension with the owner’s literal instruction, stated rather than resolved silently** `[owner]` “*I would not register write traps in the unprivileged bar0 page.*” The strongest form of that — **map the page writable straight through to hardware, never trap it** — is not reachable today, for the reason in fact 3: guest RM rings the *same physical register* from the kernel, so not trapping it also gives up intercepting **kernel** channel doorbells. The scrub is one of those (§6), and it is the live cross-client leak. So v3 takes the implementable reading: **trap, but let nothing behind the trap be shared, growable or degradable.**

□ The literal form becomes reachable the day **every channel rung through that page is passthrough**. That is a real future state, not a fiction, and it is the reason §2.2 is built as a table rather than a queue: when that day comes, the doorbell trap is deleted rather than rewritten.

2.2 The doorbell plane — a bit table, and the hint ring is DELETED

[CORRECTED, w820] The w819 text here said “*push the token on the queue, set this channel’s rung bit; queue full = set ‘inspect all doorbells’ flag.*” That fallback is exactly the amplifier above, and the queue is exactly the

exhaustible shared structure. **Both are deleted.** The table alone carries the plane.

```
nvkvm_trap_write(bar, off, val):
```

```

if (bar, off) == chip.doorbell:          # 0 generated per die/arch, never a literal (§0.3 r1)
    tok = val & chip.token_mask          # per-die decoded fields (§2.2)
    if PASSTHROUGH(tok):
        write the host doorbell INLINE. No queue, no wake, no lock. Return.
    else:                                # MANAGED (translated or emulated), or UNKNOWN
        # 2 bits per token ⇒ 32 tokens per u64 word ⇒ index by tok>>5
        fetch_or(table[tok >> 5], RUNG << (2 * (tok & 31)))    # AcqRel
        fetch_or(summary[tok >> 11], 1 << ((tok >> 5) & 63))    # Release
        bump work_seq; maybe write(eventfd)                    # §2.4
        Return.

elif chip.userspace_mappable(bar, off):  # *** THE GUARD — see below
    Return.                               # do NOTHING. Not a queue push, not a counter.

else:                                    # PRIVILEGED: reachable only by guest root
    if is_read_register(off): shadow_write(off, val)    # §41 item 4, synchronous
    append to the privileged queue (§2.3)
    bump work_seq; maybe write(eventfd)
    Return.
```

000 **THE GUARD IS NEW AT w821, AND WITHOUT IT THE DESIGN HAD A REMOTE DoS.** [owner]:

“if the write is in the userspace doorbell page ... but not at the doorbell offset, return immediately, ignore, do nothing. Otherwise a DoS attack exists by triggering from unprivileged userspace the `else` branch and bumping garbage on the privileged ring to try to fill it and crash the guest.”

☐☐☐ **Exactly right, and it is the defect §2.1 was written to prevent, sitting in §2.1’s own pseudocode.** The doorbell page is **64 KiB**; the doorbell is **four bytes of it**. Everything else on that page is guest-userspace-writable and, under the w820 `if/else`, fell through to the privileged branch — so any unprivileged guest process could push unbounded garbage onto the plane reserved for guest root and wedge it.

⇒ ☐ **The right shape is a three-way classification, not a two-way one**, and the middle arm is the one the threat model actually needs: *doorbell / guest-userspace-reachable but not the doorbell / privileged*. ☐ And the middle arm must be derived from **which pages RM maps to non-kernel clients** (§10.7), not from a hand-written offset range — otherwise a future userspace-mappable page falls into the privileged arm by default, which is the same bug again.

00 **This is also why the earlier framing was harmful and not merely imprecise.** w820 spent its threat model on *“the attacker makes us parse a victim’s pushbuffer”* — which the owner has now shown is a no-op — while the actual reachable attack was one `else` away in the same function. ☐ **A threat model aimed at the wrong mechanism does not merely fail to help; it draws attention away from the arm that was exploitable.**

000 **[CORRECTED w821, from an adversarial review — three defects, all mine.] (a)** The earlier pseudocode indexed `table[tok >> 6]` and `summary[tok >> 15]`. Those are **1-bit-per-token** shifts, while §2.4 specifies **two** bits per token. The corrected shifts are above, and the earlier `w = table[tok >> 6]` was a **dead load** besides. **(b)** It wrote the doorbell offset as the literal `0x00BB0090` — violating this document’s own §0.3 rule 1 in the first place it had the chance to. **(c)** It deferred **every** non-doorbell write with no synchronous arm, which breaks `THE_CONSTRAINTS.md` §41 item 4. See §2.3.

⇒ **A vCPU now takes NO lock at all on the doorbell path.** The w819 text said “one lock, the queue’s”; with the queue gone there is none.

Sizing — 000 and my first answer was an axis error I made myself [CORRECTED w821, before the owner saw it.] I first wrote “ 2^{19} tokens = a 64 KiB table” from `ampere/ga100/dev_ctrl.h:26` and presented it as the design constant. 0 **That is true of Ampere and false as a general claim** — precisely the mistake §0 exists to prevent. The doorbell token layout is **not one encoding**; it is at least three, and they do not even live in the same header or constant family:

chip	VECTOR	RUNLIST_ID	extra decoded bits	header
Turing TU102	11:0	22:16	—	turing/tu102/dev_ctrl.h:36
Ampere GA100	11:0	22:16	—	ampere/ga100/dev_ctrl.h:26

chip	VECTOR	RUNLIST_ID	extra decoded bits	header
Blackwell GB202	11:0	22:16	Ø RUNLIST_DOORBELL 30:30, set to ENABLE by the token generator (kernel_fifo_gb202.c)	blackwell/gb202/dev_-vm.h:28
Blackwell GB100	11:0	22:16	Ø Ø RUNLIST_DOORBELL 22:22 — overlapping the top bit of RUNLIST_ID — plus GSP_DOORBELL 31:31, RSVD 15:12, RSVD2 31:23	blackwell/gb100/dev_-vm.h:622

Three separate hazards fall out, and only the first is about size:

1. Ø **The token space is not 2^{19} everywhere.** Worst case across the table is bits {11:0, 22:16, 30, 31} = **21 bits** ⇒ **2 Mi tokens** ⇒ **a 256 KiB table.** ⇒ [PROPOSE] allocate from the **generated field widths of the running die**, and let 256 KiB be the *bound* the allocator is checked against — not a constant anywhere in source. A scan of 256 KiB is still ~4096 cachelines behind a 4 KiB summary, which is still cheaper than a ring.
2. Ø **The constant family renames across the arch boundary.** Ampere and Turing call it NV_CTRL_VF_DOORBELL in dev_ctrl.h; Blackwell calls it NV_VIRTUAL_FUNCTION_DOORBELL_* in dev_vm.h. □ **A generator keyed on the Ampere name finds nothing on Blackwell and silently produces no fields** — an empty result that reads exactly like “no variation here”. This is the a_capture_derived_table_expires_as_a_vendor_regression shape, arriving through a *generator* rather than a capture.
3. □ **GB100’s GSP_DOORBELL bit at 31 — RESOLVED w821, and it is safe by construction.** w821 first flagged this as “*the sharpest open question in this section*”, on the theory that an unprivileged write addressing the **firmware** rather than a runlist would understate §2.1’s threat model. [owner] “*well, kernel channels also have doorbells. Same thing — they do not queue on the privileged queue, treated like a normal doorbell, also with the fill, and reading when there is no work is a no-op.*” □ **Right, and the reason is structural rather than a judgement call: we implement no behaviour for that bit at all.** Because the table is sized to what the register can *express*, a token with bit 31 set indexes a real slot; because no channel is registered there, the worker finds no work and releases. ⇒ It degenerates to the same no-op as any other unowned token. Ø It is **not** a path to NV_PGSP_QUEUE_HEAD — that is a different register at 0x110c00, on a page guest userspace cannot map, and it is the *only* thing that kicks the RPC plane. ⇒ **Masking to the full expressible space is what makes this safe**, which is the second time that decision has paid for itself.

⇒ TOKEN_MASK is therefore **per-die, generated**, and masking remains the right operation rather than validating: it is what the hardware does with undecoded bits, **and** it removes an error path — no refusal, no counter, nothing to misread. Sizing to what the register can *express* rather than to what is *legal* is this tree’s own lesson, [a_bound_on_reads_is_not_a_bound_on_emits]: the attacker writes the register, not the legal space.

□ Scan cost is **arithmetic, not yet measured**: a summary of one bit per u64 is 1 KiB for Ampere’s table and 4 KiB for the 21-bit worst case; the loaded case streams u64 compares and skips zero words. Sub-microsecond warm, low single-digit µs cold. **That is cheaper than maintaining a ring** — which is why the ring is deleted rather than repaired.

⇒ What a hostile guest process can buy is: one fetch_or on two bits in a fixed table that was going to be scanned anyway. **Exactly hardware’s cost profile: ring anything, pay a redundant look.**

Ø [CORRECTED w821] I wrote “*on a per-token cacheline so there is not even a contention point.*” **False on both halves.** At 2 bits per token a cacheline holds **256 tokens**, and one summary word covers **2048**; and every doorbell — passthrough excepted — also RMWs the single worker_vcpu_poll line that §2.4 itself says ping-pongs at ~10⁷/s. ⇒ There **is** a contention point, it is worker_vcpu_poll, and the honest claim is that contention is bounded by that one line rather than growing with the number of channels.

2.3 The privileged plane — one ordered ring, and per-vCPU rings were WRONG

[CORRECTED, w820 — this defect was in my proposal, found by an adversarial review.] I had proposed per-vCPU register rings with a single-bit owner claim. Per-vCPU rings preserve only **per-vCPU** order, and the order the device needs is the one the **guest** established, which spans vCPUs:

```
T1 on vCPU0, holding an RM lock: write PDB_LO, PDB_HI → ring0
T1 releases the lock
T2 on vCPU1, takes the lock:      write TRIGGER → ring1
Worker B drains ring1 first ⇒ TRIGGER fires against the OLD PDB
```

On hardware this cannot happen: vCPU0's trap returns before T1 releases the lock, and PCIe posted writes to one function are ordered. This is not exotic — it is **every cross-CPU register sequence RM orders with its own lock**.

☐☐☐ The mechanism, settled at w821 after an adversarial review

[owner] proposed “a queue that has a lock multiple threads may read from” instead of a dedicated drainer, on the grounds that “an additional context switch is more expensive than one small lock for the vCPU”, and asked for this to be investigated before deciding. It was.

☐ **What the owner got right, and it is most of it:** the two-lock instinct (a tiny push/pop lock kept separate from a drain-ownership lock) is the correct shape if you use locks at all, and the within-queue ordering argument is sound — one global FIFO whose enqueue point is totally ordered preserves the guest's cross-vCPU order, because vCPU0's push completes before its trap returns and the guest's own lock release/acquire supplies the happens-before.

☐ What decides it against locks, and it is not a performance argument.

1. The cost comparison weighs the wrong two things. The wake is paid in **both designs** — the vCPU writes the eventfd either way. What the shared-lock design saves is a context switch **on a worker core**, never on the vCPU. ⇒ It trades a cost the vCPU never pays against a risk only the vCPU bears.

2. Lock-holder preemption puts a host scheduler slice inside an MMIO trap. `q.lock` is a userspace lock; userspace cannot disable preemption. A worker holding it for ~50 ns can be preempted with it held, and a vCPU that then traps waits for that worker to be rescheduled — **a full CFS slice, milliseconds, on an oversubscribed box**. At a few thousand pops/second the tail fires every few minutes. ☐ “Still microseconds” is the **mean**; constraint 4 is about the **tail**, and `worst_trap` is exactly the instrument that has caught this class before.

3. Under SCHED_FIFO vCPUs the tail becomes a deadlock. A FIFO vCPU spinning on a lock held by a preempted SCHED_OTHER worker on the same core never yields; the worker cannot run; the guest core is frozen until the RT throttle intervenes. ☐ Latency-tuned KVM hosts do run vCPUs SCHED_FIFO. A futex instead of a spinlock only converts the deadlock into a polite blocking wait — which is still a blocking wait on a vCPU.

4. “Any worker may drain” also means “possibly no worker will drain.” Workers are the threads that make blocking host calls. If all of them are inside one, the queue is not drained until one next polls — which makes any “wait for a slot” policy **unbounded**.

⇒ **Decision: lock-free on the vCPU, one dedicated drainer.** [owner] had already sanctioned the fallback — “holding a register drainer is not that bad; we just need the decision that is race-free and correct and secure, perf is a second issue” — and this is that decision. ☐ It also takes the vCPU's lock count from **one to zero**, which is strictly better than §41 item 1 requires rather than merely compliant with it.

2.3.1 The design

- 1. One bounded lock-free MPSC ring per device, fixed capacity, preallocated and prefaulted.** ☐ Never growable: growth means calling the allocator on a vCPU, possibly faulting — blocking work, under a lock, on a vCPU, three invariants in one line. ☐ It has its **own** cursor. `work_seq` is a wake sequence and nothing else (§2.4). ☐☐ **[CORRECTED w821 — the claim must be CONDITIONAL, and fetch_add is not.]** I first wrote “the producer's `fetch_add` on the enqueue cursor is the global order.” ☐ In the standard bounded-ring shape the producer then **waits for the slot's sequence stamp** if the consumer has not caught up — *the vCPU waiting on a worker, in lock-free costume* — and a producer that claims and then bails leaves the consumer **stalled at that slot forever with no diagnostic**. ⇒ **Read the cursor, test fullness, take the slot with a CAS.** On full: poison and return, having claimed nothing. On CAS failure: retry, looping only against peer producers that each complete in one instruction. ☐ `THE_CONSTRAINTS.md` §48.2. ☐ This trades wait-freedom for lock-freedom on the claim, and that is the right trade: wait-freedom is worthless if the claim can then be stuck waiting on the consumer. ☐ **The total order still holds** — a successful CAS on a single cursor is as totally ordered as a `fetch_add`, which is all §2.3's cross-vCPU argument needs.
- 2. One dedicated drainer thread, spin-then-park:** spin briefly after the last item, then block on an `EFD_NONBLOCK` eventfd. ⇒ Zero wakes under load, one at idle.
- 3. Peek → apply → commit.** The consumer cursor advances only *after* a successful apply, so a failed apply retries at the head instead of being re-queued at the tail, and `applied_seq` becomes a clean monotonic number other paths can fence against.
- 4. Queue everything; shadow every readable register; ☐ the drainer NEVER stores to the shadow of a guest-writable register.** Queueing pure-state registers looks wasteful and is what keeps `PDB=X`, `TRIGGER`, `PDB=Y`, `TRIGGER` correct — a drainer that read `PDB` from the shadow when applying the first trigger would see `Y`. ☐ And the clobber it prevents: vCPU writes `A` (`shadow=A`, queued), drainer starts applying `A`, vCPU writes `B` (`shadow=B`, queued), drainer finishes and writes back `f(A)` — the guest now reads a value older than one it wrote. ⇒ Worker-owned bits live in registers the guest does not write, or are updated by atomic RMW; a guest write-1-to-clear is `fetch_and`, **never** load-store, or the worker's concurrent set is lost against the ISR's clear.

5. **Full ⇒ poison, never wait.** Enter a sticky error state, drop further writes, surface a guest-visible fault by the normal mechanism — the emulated equivalent of the card falling off the bus — and **do not** write the shadow for a poisoned write. **○** Silently dropping a single write is the wrong shape: it leaves device state that a later, differently-privileged guest process inherits. **□** The privilege line is what licenses poisoning rather than blocking: only guest root reaches this queue (\$2.2's middle arm), so a flood harms only itself. **□** Size from measurement, and treat sustained occupancy above ~25 % as a defect to investigate, not a number to grow.

2.3.2 **□□□ The ordering hole across the queue boundary — found at w821, by neither of us**

Order is preserved *within* the queue. Doorbells go around it. A passthrough doorbell hits hardware inline on the vCPU; a managed one sets a table bit a *different* worker consumes. Neither is ordered against a queued register write:

1. Guest writes register R that a channel's servicing depends on. → shadow + queue. Trap returns.
2. Guest rings the doorbell. → hardware NOW, or a worker NOW.
3. The drainer applies R, 30 μs later.

On hardware, posted writes to one function are ordered and the device saw R first. Here the doorbell is serviced against pre-R state. **□ Whether any current register has that dependency is a per-die question that will be answered by a hang, not by review.**

⇒ **[PROPOSE] The fence is nearly free and uses space we already allocate.** \$2.2's token table is two bits inside a u64 slot; the doorbell trap also stamps the current `enqueue_pos` into that slot, and the doorbell worker services it only once `applied_seq >= stamp`. The vCPU pays one extra atomic load. **○** The worker waits — **the vCPU never does**, which keeps the owner's doorbell invariant intact: *queued now = eventually scheduled, never blocks*.

○ Passthrough has no worker to wait, so it needs a proof, not a fence. The obligation is: *no queued register write can affect how hardware services a passthrough channel*. Plausible — RM waits for its RPC replies before ringing, and USERD is memory — **but it is an obligation, and \$10 carries it as one.**

2.4 The wakeup protocol

One word, `worker_vcpu_poll`, and the `futex prepare_to_wait-then-schedule` idiom. [owner] supplied the field split; an adversarial review supplied the orderings and the layout.

```
worker_vcpu_poll : u64 = { work_seq : 56 (HIGH) , workers_polling : 8 (LOW) }
```

vCPU — publish the work, then `fetch_add(1 << 8) (AcqRel)`; if the pre-value's `workers_polling > 0`, `write(eventfd, 1)`. **Worker** — `seen = work_seq (Acquire)` **before** scanning; scan every source; then CAS to register as polling **only if** `work_seq == seen (AcqRel)`; on failure, rescan.

□ Registering only when the sequence is unchanged is what deletes the “who clears the flag, and when” question that the flag-based version could not answer. Any work published after `seen` fails the CAS, so a worker cannot sleep on work it did not see.

Why 56/8 is right, with the number. ABA needs the counter to wrap *inside one worker's scan*, not globally — and every vCPU CASes the **same cacheline**, so aggregate bump rate is capped by ping-pong at roughly 107/s regardless of vCPU count. 32 bits wraps in ~430 s (a descheduled worker could straddle it); 40 bits in ~30 h; **56 bits in ~228 years**. No spin-loop fallback is needed.

○ work_seq must be the HIGH half. With it low, the carry at 2^{56} lands in `workers_polling` — a phantom poller, permanent, making every subsequent vCPU trap pay a `write()` syscall, silently. High, the carry falls off the top of the u64. The benign direction (a worker's `fetch_add(1)` carrying *into* `work_seq`) only causes a rescan.

□ Cap the worker count at 255 structurally, asserted at startup. `workers_polling` wrapping would read **0 while workers are polling** — precisely the lost wakeup the protocol exists to prevent. Saturating is worse than asserting: it makes the count wrong in a way that still reads plausible.

○ Orderings are load-bearing and x86 TSO hides their absence. With `Relaxed` on the vCPU's bump, the ring/table store may become visible *after* it on ARM: the worker scans empty, registers, sleeps, and the vCPU already read `workers_polling == 0` so sends no wake. Passes every bench in this tree; fails on the first Grace host. Rule: **every write to a work source happens-before the sequence bump, and every seen load happens-before the scan.**

○ The eventfd is a sum, not a queue of wakeups. M writes landing before the first `read()` collapse to one wakeup under `EPOLLEXCLUSIVE`, leaving `N-1` workers asleep with work queued. ⇒ `EFD_SEMAPHORE | EFD_NONBLOCK`, **one shared epoll instance**, `read()` consumes exactly one. **□** And a hard invariant: **no handler may wait on another queued item**. If one ever does, that collapse stops being latency and becomes a permanent hang.

○ A bit that says work exists is not a bit that says you may touch the hardware. Per token, two bits, four states:

```

vCPU:    fetch_or(RUNG)
worker:  CAS RUNG → BUSY      (fail ⇒ not ours, move on)
        act
        CAS BUSY → IDLE      (fail ⇒ state is BUSY_RUNG ⇒ CAS BUSY_RUNG → BUSY, act again)

```

Exactly one worker per channel at a time, no re-ring lost, no lock. □ Per §2.1 this is a **security** boundary: without it an unprivileged guest process triggers the concurrent walk of root’s channel deliberately.

⊗ **Never clear RUNG on “cannot act yet.”** A doorbell can be acted on before a register write that preceded it on the same vCPU (different planes, §2.2 vs §2.3). On hardware the pending bit persists until the channel is schedulable; ours must too. Clearing it on a refusal is a lost doorbell — the same class as this tree’s `forwarded=0 refused=8` engine-object rows.

⊗⊗ **[CORRECTED w821 — the four states as written cannot express that.]** CAS RUNG → BUSY **clears** RUNG. A worker that then finds it cannot act has no transition back. ⇒ Add BUSY → RUNG as the “*put it back*” edge, distinct from BUSY → IDLE. □ And that creates a livelock the doc must close: an unactionable token is then **found on every scan**, so a worker treating “*found work*” as “*do not sleep*” spins forever — while the thing that would make the channel schedulable is a **register write or RPC that also needs a worker**. ⇒ “**Found but unactionable**” must count as “**no work**” for the purpose of deciding to sleep. The `work_seq` bump from the register drainer is what wakes it again, and that is exactly the wakeup the sequence protocol already provides.

⊗⊗⊗ **[NEW w821 — a lost doorbell IS reachable with the summary bitmap, and I never specified its clearing discipline.]** The interleaving:

```

Worker W: seen = work_seq
W: load table word w → 0
    vCPU:  fetch_or(table[w], RUNG)          ← work appears
    vCPU:  fetch_or(summary, bit w)
    vCPU:  bump work_seq → S+1
W: CLEAR summary bit w      (it saw the word as 0)
W: rescan, trusting the summary → misses the token
W: CAS register-as-polling with work_seq == S+1 → SUCCEEDS (the bump preceded the rescan)
W: sleeps.    RUNG set, summary clear, no further bump.    ⊗ LOST until an unrelated ring.

```

□ The fix is the standard one and must be **in the API shape**, not a comment: **clear the summary bit first, then re-read the word** (Release on the clear, Acquire on the re-read); if the word is non-zero, set the bit back. ⊗ A summary that is *never* cleared is not an index, and a scan that always walks the full table makes the summary pointless — so “just don’t clear it” is not an escape.

□ **Deregistration is unspecified above and needs to be:** who decrements `workers_polling`, and when relative to the `read()`. §2.4’s Q3-style safety argument assumes deregister-then-scan; that ordering is the contract, not an implementation detail.

⊗ **[CORRECTED w821] The eventfd paragraph contradicts itself.** It mandates `EFD_SEMAPHORE | EFD_NONBLOCK` with one shared epoll, and then the instrumentation note describes workers *parked in a blocking* `read()`. With `EFD_NONBLOCK` no worker ever parks there. ⇒ Keep the non-blocking form; the instrumentation note applies only to the blocking variant and is retained as a warning about **which** design it would bite. □ Also: with a **single shared** epoll instance `EPOLLEXCLUSIVE` is irrelevant — it arbitrates between epoll *instances*. The wakeup collapse it was cited against is a property of the per-worker-epoll configuration we are **not** using.

⊗ **The claim is acquired INSIDE the scan, after seen, and released before the scan returns.** Caching “this ring was empty” across a `seen` read loses an item **forever**. This must be enforced by the API shape, not by a comment, because it is exactly the shape someone optimises.

□ **Instrumentation note.** A worker parked in a blocking `read()` counts in `workers_polling` but is **not** in `epoll_wait`. A census built on the word and one built on epoll state disagree only under load — the one condition in which anyone consults them.

2.5 The doorbell page is READ-ONLY, not emulated

□□□ The doorbell page is mapped into the guest as a **KVM read-only memslot** over the real host doorbell page. Consequences, all of them deletions:

- Every **read** is hardware, with **no exit and no code** — including the microsecond counter *on this page*. ⊗⊗ **[CORRECTED w821 — “there is no PTIMER implementation in v3” was overdrawn.]** The read-only memslot removes the **usermode window’s** timer (`NVC361_TIME_0/1` at page `+0x080`). It does **not** remove the **kernel’s** clock: RM reads `NV_PTIMER_TIME_0/1` at `0x9400/0x9410` (kepler/gk104/dev_timer.h:35) — a *different BAR0 page*, reached with ordinary register reads. ⇒ That page needs its own answer, and each option costs something: read-trap it (contradicts “*only writes trap*”), shadow it (is emulation, which §8 deletes), or memslot it over the host’s page

(□ a page of **side-effect registers**, which the constraints forbid mapping wholesale). `THE_CONSTRAINTS.md` already anticipates this — “possibly with that one page still trapped.” **Unresolved; §10.**

- Every **write** traps, and runs §2.2.
- When we need to ring a doorbell (passthrough inline, or translated from a worker) we write the host page **directly**, bypassing the read-only mapping.

□ Emulated channels need no doorbell written at all — there is no hardware counterpart.

□ **[CORRECTED at w819] Passthrough and managed are different costs, and I had them merged.** A passthrough doorbell is a table lookup and one dword, synchronously, on the vCPU — **microseconds and a return**. Only translated/emulated doorbells set a bit and wake. Translated and emulated behave identically from the vCPU’s side; only passthrough is inline.

⇒ **This excludes `ioeventfd` for passthrough:** an `ioeventfd` would buy a cheap exit and then pay for it with a thread context switch to deliver a dword the vCPU could already have written.

3. Memory

Guest vidmem is **one host allocation**, and its size is a **command-line parameter of the hypervisor invocation**, exactly like guest RAM. **[CORRECTED]** v2 sized it by a bisecting probe and then rebound the advertised framebuffer to whatever the probe won; that is a measurement standing in for a decision. An operator asking for 12 GiB either gets it or gets a refusal at startup.

Guest FB offset **is** the offset into that allocation. Guest systemem is the hypervisor’s memfd reached through a static layout.

4. Addresses

4.1 Mirror, never adopt — and the review’s reasons are stronger than mine

[CORRECTED] I argued adopt was merely unsound because guest PTEs hold guest-physical addresses. The review gives three independent defeaters, any one of which is fatal:

1. You **race the host kernel module’s own tables and invalidates**.
2. The host driver **exposes no userspace interface** to participate in its locking, so adopt needs a **host kernel module** — a deployment bottleneck for the whole product.
3. You need a translation **anyway**, because guest physical ≠ host physical. The only escape is a contiguous host range for guest physical, which then fails on **fragmentation** even when there is free memory.

⇒ Mirror is also **completely rootless**, which is the security posture the product needs.

4.2 The diff

The PTX walk kernel produces a **per-VA-space delta**: a list of VA spaces that changed, and the entries to apply to each. ○ **The previous state lives in vidmem, held by the PTX itself** — a snapshot of the full PD/PT tree per VA space, keyed by root address, with allocations freed when no longer referenced. **There is no host-side table of old pages.**

The delta is executed by the VA manager thread as `mmap/munmap`.

□ **Batching, and it is the answer to the syscall-cost question.** `[owner]` set the **TLB defer flag** — which `ogkm` demonstrably has — on every mapping call but the last. One invalidate for the whole batch instead of one per page.

4.3 A VA space is the OBJECT. The PDB is an attribute of it, per GPU.

○○○ **[CORRECTED TWICE — and `ogkm` refutes the correction I accepted.]** I first wrote that a root is *only* a walk seed and never an identity. The review corrected me to (PDB base, aperture), and I agreed. **`ogkm` says neither of us was right, and the original `option<Pdb>` design was.** Four independent reasons, all cited:

1. **RM’s identity is the object plus `vasUniqueId`**, a monotonic atomic assigned at creation (`virt_mem_mgr.c:143`). Every runtime lookup is by client handle, never by PDB.
2. **One VA space has N PDBs on N GPUs** — the PDB lives in per-GPU state (`gvaspaceGetPageDirBase(pGVAS, pGpu, gpu_vaspace.c:1874)`).

3. **A VA space can CHANGE its PDB, and can have NONE.** `SET_PAGE_DIRECTORY` is explicitly repeatable for re-size/migrate (`nvos.h:3084`); `gvaspaceResize` migrates the root (`gpu_vaspace.c:3235`); and the same function reads if (`NULL == gvaspaceGetPageDirBase(...)`) goto `doneGpu`; Hardware even encodes the absent case: `NV_RAMIN_SC_PAGE_DIR_BASE_TAR-GET_INVALID`.
4. **Nothing stops two VA spaces naming the same PDB** — there is no global PDB→VAS registry; RM's only check is local to the target VAS (`gpu_vaspace.c:2855`).

⇒ (PDB, aperture) **is a point-in-time HARDWARE FINGERPRINT, not a durable identity.** Use the VA-space object (its resource key) as identity; carry `Option<(Pdb, Aperture)>` as a mutable, per-GPU, possibly-absent attribute used to seed a walk and to match hardware state.

□ This retroactively vindicates the existing `Vas::pdb` doc — “None until a declaration arrives, and a space is nameable, routable and populatable before then” — which is exactly what `ogkm` describes. □ And it means my `Pdb(0)` fix treated a symptom: the durable fix is to stop using the fingerprint as the key at all.

4.4 Where the root actually comes from — it is ours to write

⊗ **Root discovery is not a problem, and the reason is stronger than “we can find it”.** `_gmmuWalkCbuUpdatePdb` is a **no-op inside a guest or CPU RM** on GSP parts (`gmmu_walk.c:599`): *channel instance blocks are written by GSP-RM — which is us*. The guest RPCs the instance block's physical address to GSP (`kernel_channel.c:2626`). We are the party that programs `NV_RAMIN_PAGE_DIR_BASE_{TARGET,VOL,LO,HI}`, and `TARGET` is the aperture, written as one unit with the address (`kern_gmmu_gp100.c:123`).

□□□ And `COPY_SERVER_RESERVED_PDES` is **the guest handing us real PDE physical addresses**. In the GSP split one logical VA space exists as *two* objects: client RM owns all page-table memory; **server RM — us — gets an exclusive 512 MiB window** (`0x1_0000_0000`, size `0x2000_0000`) plus the physical addresses, apertures and page shifts of the client's PDE pages for that window, so the server can map into the same hardware tables without allocating any (`ctrl90f1.h:261`, `gpu_vaspace.c:4216`).

⊗ **DMA_SET_PAGE_DIRECTORY is UVM's call, not a mapping call.** “DMA” is the *device's DMA/VA-management sub-interface* of `NV01_DEVICE_0` — legacy taxonomy, and every sibling (`GET_PTE_INFO`, `INVALIDATE_TLB`, `SET_VA_SPACE_SIZE`) is VA-space-wide. What it does is relocate a VA space's top-level page directory onto **client-supplied** memory; the caller is UVM at GPU-VA-space registration (`nv_gpu_ops.c:8986`). `UNSET` is its exact inverse.

⊗ **DUP_OBJECT aliases, it does not copy.** `vaspaceapiCopyConstruct` is literally `vaspaceIncRefCnt(pVAS)`; `pVaspaceApi->pVASpace = pVAS`; (`vaspace_api.c:440`). Two handles, one object — the normal UVM flow, not an anomaly.

5. Completions and interrupts

[CORRECTED] Most completion code deletes.

kind	what we do
Passthrough	nothing. We do not inspect the channel and do not care when it finishes.
Translated	nothing. The semaphore address was already forwarded; the GPU writes it.
Emulated	forge the completion at the end of our own call — there was no GPU work.

⇒ The completion-watch list, the 250 ms observer sweep and the CPU-executor completion writer all go.

What remains is the armed interrupt — and `ogkm` settles its shape.

□□□ **Registration is PER ENGINE, not per channel and not per semaphore.** `NV01_EVENT_NONSTALL_INTR` *requires* its notifier to be a **Subdevice**; the notify index maps to an engine (`NV2080_NOTIFIERS_CE0..n`, GR, HOST) and lands in `pGpu->engineNonstallIntrEventNotifications[rmEngineId]` (`event_notification.c:688`). The channel class `NVA06F` has **no completion event at all** — only error/RC notification. So the interrupt is a **broadcast wake to every waiter on that engine**, carrying no channel and no semaphore identity.

□□□ **The race is the WAITER'S burden, and it is handled three independent ways.** RM's own semaphore-surface registration re-reads the value under a spinlock and returns `NV_ERR_ALREADY_SIGNALLED` — with the race written out in a comment (`sem_surf.c:1405`); callers loop on it (`nvidia-drm-fence.c:991`); and the OS-event wait path **never trusts the event**, looping on the notifier word in memory and using the event only to decide *when to re-check* (`nvidia-push.c:751`).

⇒ [owner's hypothesis, confirmed] we only need to fire on **genuinely new completed work** — an advance of the engine. We do **not** have to fire retroactively for work already complete, because the guest re-checks after arming. □ But the converse is binding: once armed, a later release **must** produce an interrupt. There is no level-triggered delivery to fall back on.

○ **And the interrupt is requested in the PUSHBUFFER**, not by a control: NV906F_NON_STALL_INTERRUPT pushed after the release (nvidia-push.c:967), or CE_LAUNCH_DMA ..._INTERRUPT_TYPE_NON_BLOCKING, which RM sets **only when a completion callback was requested** (channel_utils.c:657; default NONE). □ There is **no AWAKEN_ENABLE field** in the headers shipped here — an earlier note in this tree citing one should be re-read.

□ **The channels we most care about POLL anyway.** UVM never uses interrupts for channel completion — it spins (uvm_channel.c:2158). The CeUtils scrubber registers a non-stall event only for its *asynchronous callback* API; every blocking wait is while (READ_CHANNEL_PAYLOAD_SEMA(..) < target) { ... } (channel_utils.c:343), and when it holds the GPU lock it services the ISR **inline** rather than sleeping.

6. The scrub

□ **There IS a dedicated scrub channel, and a second one beside it.** OBJMEMSCRUB owns a CeUtils channel per heap, kind CE_SCRUBBER_CHANNEL OR FAST_SCRUBBER_CHANNEL (SEC2's SWL_SCRUBBER_CHANNEL under Confidential Compute) — mem_scrub.c:171, ce_utils.c:246. A **separate** CeUtils instance serves MemoryManager's general internal copies (mem_mgr.c:4114), so scrubbing and other RM-internal CE work do not share a channel.

⇒ That makes the scrub a clean, named, **translated** channel. The guest's own RM scrubs before reuse and orders it correctly; our only job is to let the work reach the GPU instead of forging its completion. [measured w813] today the kernel tokens are forwarded=0 — 140 doorbells rung, none sent — which is the cross-client leak.

7. Portability — derive, do not hardcode

[owner] in priority order:

1. Derive from **host userspace calls** wherever possible.
2. **Computed** values are allowed.
3. Generate Rust from the **C headers** of ogkm and friends — dies, families, driver versions.
4. Read **nvkvm-pv** for how it already solved this.

○ **Do not parse C with regex or sed.** Use a real parser/compiler front end. [owner] *"That will age better."* □ This tree's own history agrees: a capture-derived table expired as a vendor regression, and 11 of 56 captured rows were empty and every empty one that was checked against hardware was **contradicted**.

The surviving hand-written surface should be a small set of per-architecture-series constants.

8. What is deleted

Isolates and the whole IPC plane · the sandbox plane · Proc · the address table · joins (all four generations) · VA→phys translation on the submission path · the operand gate · publication epochs, the dirty gate, sweep-skip · the CPU CE executor · the completion watch · the *usermode-page* PTIMER emulation (○ **not** the kernel NV_PTIMER page at 0x9400 — see §2.5) · the framebuffer probe/rebind · **every on/off flag for things that no longer have two sides** [owner].

□ [w820] **Three more, and they were in the v3 proposal itself — not in the old tree:**

- **The doorbell hint queue.** Superseded by the 64 KiB bit table (§2.2). Deleting it deletes ring overflow, the multi-consumer problem for doorbells, and an unprivileged guest process's ability to exhaust anything.
- **"Inspect all doorbells" / any FULL_REFRESH-shaped fallback.** There is no refresh because there is no queue to fall back *from* — there is only the scan, which is the normal mode. [owner, 2026-09-20] *"guest userspace can exist without something like FULL_REFRESH."*
- **Per-vCPU register rings and their owner-claim bit.** Refuted in §2.3: they preserve per-vCPU order where the device needs the guest's cross-vCPU order.

○ Deleting a mechanism that only ever existed in a design document costs nothing and is the cheapest deletion available. □ Both doorbell deletions were **licensed by a threat model**, not by a benchmark — which is the only reason they were found before the code existed.

○ **[CORRECTED] What to keep is narrower than I said.** I proposed keeping "the instruments". The review is right that some instrument *prose* is actively misleading — three stale "The default" comments in one day, one of

which made a full survey report that the single store does not run. = Keep the **measurements** and the **gates**; audit the **prose**, and delete what can no longer be checked. A comment asserting a default will rot; a comment asserting a dated measurement will not.

9. Rewrite, not refactor

[owner asked for an honest view] **Rewrite, in a fresh crate set, with the old tree as a read-only reference.**

The deletions above are not edits inside the existing files — they are the removal of the organising ideas those files are built around (per-proc isolation, an address table, a publication pass). Deleting them incrementally means running two architectures in one file set for the duration, which is how the stale defaults and contradictory tests got there in the first place.

□ The one non-negotiable: **the rewrite must be graded continuously against the existing suite** — the 30-arm guest suite, the bare-metal baseline, and the hardware gate (forwarded= per token). A rewrite that cannot be graded is a rewrite that cannot be landed.

⊗ [measured 2026-09-20] crates/*/src is **144,175 lines of code** and 125,503 lines of comment. A 50k target is a ~3x reduction of code, and the four crates carrying the deletions are `kayfabe-isolate-host` (46,895), `kayfabe-abi` (43,588), `kayfabe-qemu-raw` (37,057) and `kayfabe-device` (27,496). `kayfabe-abi` should shrink by being **generated** rather than deleted.

10. Still open — and what the owner ruled at w821

⊗ **Numbering was out of order and one item appeared on both the resolved and open lists.** Rebuilt.

10.1 Ruled at w821 — closed, with where each ruling landed

#	question	ruling	lands in
R1	Does a hostile doorbell give an attacker a control input?	⊗ No . It has no access to USERD, the ring or GP_PUT; it costs cycles, as on hardware. The double-take is a correctness boundary owed to the victim’s own concurrency, not a security one	\$2.1
R2	Non-doorbell offsets in a guest-userspace-mappable page	⊗⊗⊗ Return immediately, do nothing. Without this, unprivileged userspace floods the privileged plane — a live DoS in the w820 design	\$2.2
R3	The Hopper+ BAR1 doorbell	□ Trap the doorbell-bearing page(s) and nothing else, read-only memslot, <code>doorbell_bar/doorbell_off</code> generated per arch. □ [AMENDED w821] the ruling was first recorded as “ <i>exactly one doorbell page exists</i> ”, derived from an unproven hypothesis about NVIDIA’s motive. There are up to three doorbell-bearing apertures per GPU; the ruling stands because identification is by RM handle , not by count	\$2.1

#	question	ruling	lands in
R4	GB100's GSP_DOORBELL bit 31	□ Safe by construction — we implement no behaviour for it, so it degenerates to an unowned token and a no-op. Kernel channels have doorbells too and are treated identically	\$2.2
R5	Per-arch constant-family renames	□ Code it per giant GPU family — the rename boundary is the family boundary, so it is a bounded, enumerable set	\$0.3, \$2.2
R6	Fault delivery	□ We do fault DELIVERY; we guarantee no fault RECOVERY. Recovery is unreachable from guest userspace, so we never need to reverse-engineer it — ogkm's recovery cases are nameable and each gets a named workaround. UVM <i>managed</i> memory is later and is simulatable by forcing DMA memory , which never changes aperture	\$2.1 of Part 2, \$10.3
R7	BQL	□ Never on a vCPU. Permitted only in slow paths that are not our MMIO traps — memslot moves, installation time	\$1
R8	Locks on a vCPU	□ At most one, microseconds, for MMIO only. Slow paths (driver setup) may take more, including BQL	\$2.3, \$41
R9	Batching the VA diff	□ Confirmed as intended: set the TLB-defer flag on every mapping call but the last, minimising host-side invalidate barriers within one refresh. □ A refresh on the host must still happen last — the defer batches the barrier, it does not remove it	\$4.2
R10	Turing	□ Must be supported. It is the declared floor and it is <i>not</i> the easy end — it needs the GP10X format family, which our one built format (GA10X) is a superset of by one match arm ⇒ additive	\$0
R11	Multi-GPU / non-GA10x chip tables	□ Rewrite the chip table to be derivable rather than special-casing. Judged fixable	\$7

#	question	ruling	lands in
R12	The register queue, after investigation	□ Keep the dedicated drainer thread. [owner] <i>“fable argued hard to keep it and the arguments are correct. The MMIO trap must be extremely fast, the rest has much more room in the millisecond.”</i> = Generalised as THE_CONSTRAINTS.md §48 — the latency budget is asymmetric, and a thread spent to keep the trap lock-free is a good trade, not an extravagance	\$2.3, §48

10.2 Genuinely open

1. **The interrupt race, narrowed.** §5 settles *whose burden* (the **waiter’s**, three independent ways). What is open: how to honour *“once armed, a later release must produce an interrupt”* when registration is **per engine** and carries no channel or semaphore identity — and when `NVC36F_NON_STALL_INTERRUPT`, the method that *requests* it, is **not recognised anywhere in the tree**.
2. □ **The privileged queue — SETTLED at w821 (§2.3), investigated as the owner asked.** Lock-free bounded MPSC + one dedicated drainer; spin-then-park; overflow **poisons** rather than waits. = The vCPU’s lock count goes from one to **zero**. □ The deciding argument is not performance: a shared lock puts a **host scheduler slice inside an MMIO trap** via lock-holder preemption (milliseconds in the tail, a deadlock under `SCHED_FIFO` vCPUs), and *“any worker may drain”* also means *“possibly no worker will drain”*, which makes any wait-for-a-slot policy unbounded. □ **Still open under it:** the ring’s capacity must come from a **measurement**, not a guess.
3. □ **Where the walk’s root comes from.** [owner] this should be analysable from ogkm — the call must be used somewhere, and if it is only UVM-managed memory then it is the wrong thing to be looking at. = **And the owner supplies the design answer that makes it actionable:** the walk already tells us whether a leaf is **system or vidmem physical**, and *that* decides the mechanism — an **RM allocation at a VA** in the VA-manager thread, or a **DMA-map ioctl**.
4. **The crate model** — to be redrawn against §8.
5. **The scan-cost figure in §2.2 is arithmetic, not a measurement.**
6. **Enumerate every BAR0/BAR1 range RM maps to a non-kernel client**, generated rather than asserted. □ R2 makes this load-bearing: the *middle* arm of the trap classifier is defined by this set, and a page missing from it falls into the **privileged** arm by default — which is R2’s bug again, arriving by omission.
7. **The kernel `NV_TIMER` page at 0x9400** — §2.5 covers the usermode window only.
8. **A vIOMMU in the guest hands us IOVAs, not GPAs.** Unhandled; the real content of axis κ .
9. **Windows guest** — [owner] read ogkm’s Windows-specific code and lawfully published material on NVIDIA’s Windows behaviour. ∅ In progress; nothing claimed yet.
10. ∅∅ **[NEW w821] The passthrough-doorbell ordering obligation.** §2.3.2 fences *managed* doorbells against queued register writes with an `applied_seq` stamp. A **passthrough** doorbell has no worker to wait, so it needs a **proof** that no queued register write can affect how hardware services a passthrough channel. Plausible (RM awaits its RPC replies before ringing; USERD is memory) — ∅ but unproven, and the failure mode is a hang, not a diagnostic.
11. □ **[NEW w821] The privileged ring’s capacity**, from measured peak occupancy under the constraint suite. Sustained occupancy above ~25 % is a defect to investigate, not a number to raise.
12. **Hopper+ with the BAR0 form of the usermode object** — still legal there. Whether both forms can be live at once in one guest is unchecked (R3).
13. ∅ **Do multiple clients mapping the SAME pBar1VF memdesc share one guest-physical BAR1 page, or get separate BAR1 addresses?** `memdescAddRef` suggests sharing, but that is inference and the mapping path was not read. □ This is what actually decides whether *“one page”* is true on Hopper+; the handle-based identification in R3 is correct either way, but the **size of the trapped set** depends on it.
14. □ **A standing hygiene item, from w821:** an owner **hypothesis** was written into this document as an established vendor motive and then used to justify a design simplification. = **A claim’s epistemic status is part of its citation**, exactly as its date is. Hypotheses get marked, and a design decision may not rest on one.

Part 2 — The surface, at constant level

STATUS: LIVE, 2026-09-20 (w821). Companion to `THE_ARCHITECTURE_v3.md`. Supersedes the *data* in `THE_SURFACE_v2.md`; that document's KEEP/DELETE *plans* still stand and are kept.

◊ **Written for a reader who does not know NVIDIA's constants.** Every entry gives the constant name, its numeric value, one plain sentence saying what the thing *is*, and what we do with it. □ Every name and number is cited to `research_clones/ogkm` or to this tree. Anything a survey could not establish is marked **[UNVERIFIED]** rather than guessed.

0. The four planes, and why they are different shapes

A guest driver reaches a real GPU through exactly four kinds of traffic. *kayfabe* intercepts all four, and each has a different shape because each has a different **trust** and **ordering** requirement — not because of taste.

plane	transport	who can drive it	our posture
GSP RPC	a command/status ring in guest memory, kicked by a BAR0 register write	guest root (RM)	we are the GSP: we answer, forge or forward
RM control	carried <i>inside</i> GSP RPC GSP_RM_CONTROL, and via <code>ioctl</code> on the host side	guest root (RM)	serve, forward, refuse — by command number
BAR0 MMIO	reads untrapped from a DRAM shadow; writes trap to the vCPU	guest root , except the usermode window	allowlist (§41), shadow write-through
channel / pushbuffer	a GPFIFO ring in guest memory, kicked by a doorbell write	□ unprivileged guest userspace	passthrough / translated / emulated

□□□ **The last row is the one that restructured the design.** The doorbell is the only surface an unprivileged guest process touches directly, which makes it adversarial to the guest's own root — see `THE_ARCHITECTURE_v3.md` §2.1 and `THE_CONSTRAINTS.md` §47.

⇒ Read the four sections below in that order: they go from most-privileged transport to least-privileged, and the machinery gets **weaker and more paranoid** as it goes down.

1. GSP RPC — the plane we answer as the firmware

1.1 What this plane is, for a reader who does not know NVIDIA's constants

Modern NVIDIA GPUs run a firmware processor called the **GSP** (GPU System Processor). On such a part the host driver does not talk to the hardware directly for most things — it sends **messages** to the GSP, and the GSP's own copy of the resource manager (**GSP-RM**) does the privileged work. *kayfabe*'s central trick is that **we are the GSP**: the guest's stock, unpatched driver sends us these messages believing we are firmware.

The transport is a pair of rings in **guest systemem**, not a register interface:

- a **command queue** the guest writes and we read,
- a **status queue** we write and the guest reads,
- each ring made of fixed-size **elements**, each element carrying a checksum and a sequence number, each message an `rpc_message_header_v03_00` envelope plus a payload.

The guest tells us where those rings are once, at boot, through `MESSAGE_QUEUE_INIT_ARGUMENTS` (*kayfabe-gsp/src/boot.rs*:100, read at :1185). After that the **only** hardware event in the whole plane is a single register write that says “*the command queue moved*” — see §3.

1.2 The structures on the wire

structure	what it is	kayfabe [CODE]	ogkm [ogkm]	axis
msgqTxHeader	the ring's producer header: version, element size/count, write_ptr, flags, and the offsets of the rx header and the element array	TxHeader, kayfabe-gsp/src/ring.rs:131-148 (8×u32, 32 bytes, write_ptr at +16)	msgq_priv.h:37	stable
per-element header	prefixes every element: a checksum and a sequence number, plus transport framing	ElementLayout, kayfabe-gsp/src/element.rs:112-233	message_queue_priv.h:43 (580) / :52 (610)	Ø Dg — BREAKS
rpc_message_header_v03_00	the message envelope inside an element: header_version, signature, length, function, rpc_result, rpc_result_private, sequence — 32 bytes	RpcEnvelope, kayfabe-abi/src/view.rs:558-572	—	Dg (additive)
queue pair geometry	binds the guest's command queue and our status queue, including the swapped rx convention where each side publishes its consumption into the <i>other</i> queue's header	MsgqGeometry, kayfabe-gsp/src/ring.rs:407-570	MSGQ_FLAGS_SWAP_RX	stable

□□□ **The element header is the single best worked example of the Dg axis in this tree**, and it is why §0.2 says descriptors rather than branches:

	580.159.04	610.43.02
header size fields	48 bytes authTagBuffer[16], aadBuffer[16], checksum@32, seqNum@36, elemCount@40	16 bytes mctpHeader, nvdmHeader, checksum@8, seqNum@12

⇒ Size *and* every field offset changed between two versions we must both support, and the elemCount field **disappeared**. kayfabe absorbs this as data — ElementLayout carries elem_count_off: Option<usize> and a TransportHdr that is None at 580 and Mctp{..} at 610 — not as a version branch. **[PROPOSE]** every other axis-carrying fact should take this shape.

1.3 The functions

Dispatch: FunctionCodes::classify(), kayfabe-gsp/src/rpc.rs:211-232. The number↔name mapping is **generated**, not transcribed: kayfabe-abi/src/generated/rpc.rs.

□ [ogkm] The enum (src/nvidia/inc/kernel/vgpu/rpc_global_enums.h) is **byte-identical between 580.159.04 and 610.43.02** for every id we use. 610 only *appends*. ⇒ On this plane the *numbers* are Dg-stable and only the *framing* moves.

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
1	SET_GUEST_SYSTEM_INFO	first message after boot: the guest states its driver version, page size and OS, and expects the agreed protocol version back	SERVE	kayfabe-device/src/guestsysinfo.rs:112
64	SET_GUEST_SYSTEM_INFO_EXT	follow-on carrying extra system strings; the real driver returns its status, so it must succeed	SERVE (NV_OK, empty body)	guestsysinfo.rs:122
65	GET_GSP_STATIC_INFO	the guest asks for the big fixed-fact table about this GPU — memory layout, engine list, BAR sizes — before anything else can initialise	SERVE from our device-data model	kayfabe-device/src/staticinfo.rs:182
70	UPDATE_BAR_PDE	the guest hands us the physical root of its BAR2 page directory. □ Only the firmware’s directory is hardware-rooted, so this is the only fact we get about that tree	SERVE (latch it)	kayfabe-device/src/bar2.rs:299
72	GSP_SET_SYSTEM_INFO	a system-description blob (PCI topology and friends), sent early, fire-and-forget	IGNORE — ∅ <i>no reply at all</i> ; echoing would desync the guest’s sequence counter	kayfabe-gsp/src/rpc.rs:318
73	SET_REGISTRY	the guest uploads its entire registry / module-parameter table	IGNORE (no reply)	same
10	FREE	destroy one object handle, or a whole client namespace — RM’s teardown stream	SERVE from the local object graph	kayfabe-rmrpc/src/lib.rs:1095, :1955
21	DUP_OBJECT	alias one client’s object into another client’s handle namespace. ∅ It aliases, refcount++ ; it does not copy — this is the normal UVM flow	SERVE (a graph edge)	lib.rs:1097, :1918
103	GSP_RM_ALLOC	the generic “ <i>create an RM object of this class</i> ” envelope. Nearly every object the guest makes — clients, devices, VA spaces, channels, memory, contexts — rides inside	MIXED, default REFUSE	lib.rs:1229-1276

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
76	GSP_RM_CONTROL	the generic “ <i>run this RM control command</i> ” envelope. Almost every specific query or operation rides inside as a nested NVxxx_CTRL_CMD_* id + params — see §2	MIXED, default REFUSE	lib.rs:1521-1572
71	CONTINUATION_RECORD	∅ not a function: an overflow <i>fragment</i> of a message too large for one element. ∅∅ [CORRECTED w821] “only <i>GSP_RM_CONTROL fragments this way</i>” is false — there are three producers: GSP_RM_CONTROL, SET_REGISTRY (via the <i>async</i>, no-wait path) and ALLOC_MEMORY (page-table descriptor RPC). This tree’s own reasm.rs:12-31 lists all three, while its reassembly (:341) handles only the control case. ∅ GSP_RM_ALLOC genuinely does not fragment — it returns	folded into reassembly; standalone is refused	kayfabe-rmrcp/src/reasm.rs:250
47	UNLOADING_GUEST_DRIVER	NV_ERR_BUFFER_T00_SMALL the guest is about to rmmmod. □ Synchronous — the guest blocks until we reply	SERVE (ack-only) , reply mandatory	kayfabe-device/src/inert.rs:119
202	ECC_NOTIFIER_WRITE_ACK	the guest’s own acknowledgement that it finished writing the ECC notifier buffer	IGNORE	rpc.rs:320
228	INIT_GSP_TRACE_CRASH_BUFFER	the guest hands us a {physical address, size} for a buffer the firmware is meant to fill with crash-trace records	SERVE (ack, no-op) — we write no trace records, so it stays zeroed	inert.rs:119
0x1001	GSP_INIT_DONE (<i>event</i>)	we tell the guest “ <i>boot finished</i> ”; its boot poll waits on this	outbound only	kayfabe-gsp boot FSM
0x1003	POST_EVENT (<i>event</i>)	the generic asynchronous-completion carrier firmware uses to notify the guest	outbound only	—

#	NV_VGPU_MSG_FUNCTION_	what it is, in plain language	we do	[CODE]
0x1004	RC_TRIGGERED (<i>event</i>)	we tell the guest a channel or engine was torn down by robust-channel fault recovery	outbound only	kayfabe-gsp/src/fault.rs
—	everything else (~210 ids)	any other function or event id	Ø REFUSE by name — NV_ERR_NOT_SUPPORTED (0x56), zeroed body	lib.rs:1146

□□□ **The safe default is a named refusal, never an echo.** A chain of `CommandPolicy` links is tried in order (`PolicyChain::respond`, `kayfabe-gsp/src/boot.rs:596`); whatever no link answers falls through to `GspFsm::answer`'s refusal (`boot.rs:1719-1763`). Ø An `Echo0k` policy exists (`boot.rs:541`) but is a **differential-test fixture reproducing the old C's behaviour** and is **not installed in the production chain**. This matters because `0x56` is a status the guest driver forgives — so a generic-ack fallback would let a wrong configuration run on, undetected, which is precisely the failure this project has measured repeatedly.

1.4 □ The reply is always built from local state

Ø **No RPC is a verbatim forward.** Applying an `RmEvent` to the local object graph (`Gpu::apply`, `kayfabe-core/src/gpu.rs:5582`) can *latch* a pending host-side spawn, resolved immediately after by `materialize_pending` (:5591). Real host ioctls happen **asynchronously as a side effect**, decoupled from the synchronous RPC reply.

□ **That decoupling is load-bearing and was paid for:** it previously ran inline under the rank-0 lock and **crashed QEMU** (`gpu.rs:3462-3471`).

□ **[UNVERIFIED]** whether any individual modelled `GSP_RM_CONTROL` / `GSP_RM_ALLOC` arm blocks on a host call before constructing its reply. The top-level dispatch does not; the ~15 per-control handler files were not exhaustively checked. ⇒ This is the same question as the synchronous-verb list in §2.4, and it is the one the owner is waiting on.

2. RM objects and control commands — the plane that rides inside the RPC

2.1 What this plane is

Everything the guest driver *does* is expressed as two verbs against an object graph:

- **allocate an object of class C under parent P** (`GSP_RM_ALLOC`, §1 fn 103), and
- **run control command X against object O** (`GSP_RM_CONTROL`, §1 fn 76).

The object graph is a strict tree rooted at a **client**: `client` → `device` → `subdevice`, and alongside it VA spaces, channel groups, channels and engine objects. ⇒ Our job on this plane is to **be that graph** — to answer as RM would, from our own model, without a host GPU behind most of it.

□ **Default-deny, with a named refusal.** Both verbs refuse anything without an explicit decoder: `AllocClassNotPermitted` / `ControlNotPermitted` / `UnknownControl`. Ø There is no generic-ack fallback. §1.3 explains why that matters — `0x56` is a status the driver *forgives*.

2.2 The object classes we model

class	NVIDIA name	what it is, in plain language	we do
0x0 / 0x41	NV01_ROOT / NV01_ROOT_CLIENT	the handle namespace everything else lives under — a session root	emulate (graph node)
0x80	NV01_DEVICE_0	one physical GPU, under a client	emulate ; <code>deviceId</code> decoded for multi-GPU routing
0x2080	NV20_SUBDEVICE_0	a named sub-unit of a device (1:1 on a single GPU)	emulate

class	NVIDIA name	what it is, in plain language	we do
0x2081	NV2081_BINAPI	∅ an opaque handle whose controls tunnel whole to firmware, uninterpreted by the kernel	allow; ∅ load-bearing for cuInit
0x79	NV01_EVENT_OS_EVENT	the event object userspace binds to an eventfd for completion wakeups	emulate ; matched by a later POST_EVENT
0x7e	NV01_EVENT_KERNEL_CALLBACK_EX	an event the guest's own kernel RM allocates at adapter init	emulate ; ∅ params (a guest-kernel function pointer) deliberately not decoded
0x90f1	FERMI_VASPACE_A	a GPU virtual address space — the page-table root a channel binds to	emulate and allocate for real on the host
0xa06c	KEPLER_CHANNEL_GROUP_A	a TSG : channels scheduled together, sharing one VA space	emulate ; hVASpace decoded
0x9067	FERMI_CONTEXT_SHARE_A	a subcontext — an indirect handle reaching a VA space via its TSG	emulate ; hVASpace decoded
0xc56f	AMPERE_CHANNEL_GPFIFO_A	the command-ring channel object (§4)	emulate and allocate on host
0xc7c0	AMPERE_COMPUTE_B	the compute engine object	emulate and allocate on host
0xc797	AMPERE_B	a CUDA process binds the 3D/graphics engine object — sibling of compute on the same GR engine	emulate
0xc7b5	AMPERE_DMA_COPY_B	the copy engine object	emulate and allocate on host
0xc561	AMPERE_USERMODE_A	∅ the object whose 64 KiB CPU mapping is the doorbell page (Part 1 §2.1)	allocate on host
0xc574	UVM_CHANNEL_RETAINER	a handle UVM takes on a channel it did not create, to keep it alive	emulate ; ∅ never forwarded
0xc076	GP100_UVM_SW	the software placeholder class UVM binds for fault-cancel (§4.2)	∅∅ REFUSED — [MEASURED] all 4 requests in a boot refused 0x56. ∅ It is on the shared allowlist but not routed through classify() — an admitted-but-unreachable class

∅ **DENIED by name, each with a reason** (capability.rs:1579):

class	name	why refused
0x3f	NV01_MEMORY_LOCAL_PRIVILEGED	privileged video memory
0x71	NV01_MEMORY_SYSTEM_OS_DESCRIPTOR	∅ would hand the host a guest-chosen pointer
0x402c	NV40_I2C	no physical board bus exists. ∅ [UNVERIFIED w821] the claim that “ <i>RM's own source expects this alloc to fail</i> ” does not check out — i2capiConstruct_IMPL returns NV_OK unconditionally. The refusal may still be right; its stated justification is not

Beyond these, ~89 classes are **allowlisted but opaque** — admitted past default-deny and tracked as a graph node with no facts extracted (Turing/Ampere/Ada/Hopper/Blackwell engine and codec classes, NV50_MEMORY_VIRTUAL, GT200_DEBUGGER, fabric classes). □ **[UNVERIFIED]** whether any of them ever reaches a real host allocation.

□ **Axis note.** The host-side class profile is per-architecture: AMPERE_CHANNEL_GPFIFO_A / AMPERE_USERMODE_A / AMPERE_DMA_COPY_B on GA10x and AD10x, HOPPER_* equivalents on GH100. ∅ **Only the GA10x profile is validated on real silicon.**

2.3 Control commands

Served locally — the ones we answer from our own model, no host GPU touched:

cmd	NVIDIA constant	what it asks
0x20800a36	INTERNAL_GPU_GET_CHIP_INFO	static chip identity — boot registers, architecture, implementation
0x20800a40	INTERNAL_GET_DEVICE_INFO_TABLE	the per-engine device/instance table
0x20800a41	GET_USER_REGISTER_ACCESS_MAP	□ the bitmap of BAR0 registers userspace may touch — directly relevant to Part 1 §2.1
0x20800a4c	GPU_GET_SMC_MODE	whether MIG partitioning is on
0x20800aac	BIF_GET_STATIC_INFO	static PCIe info
0x20800af3	CONF_COMPUTE_GET_STATIC_INFO	confidential-computing capabilities
0x20800a59	GMMU_GET_STATIC_INFO	MMU geometry
0x20800a61	FIFO_GET_NUM_CHANNELS	how many channels this GPU supports
0x20802a08	CE_GET_FAULT_METHOD_BUFFER_SIZE	□ the size RM will DMA into. An empty capture row decoded this as 0 against a real 20480 — a buffer overrun with a hardware writer
0x2080012b	GPU_PROMOTE_CTX	the guest publishing {VA, PA, size, attr} bindings for channel context buffers
0x00801813/4	DMA_SET/UNSET_PAGE_DIRECTORY	bind or revoke a VA space's page directory on a GPU
0x90f10106	VASPACE_COPY_SERVER_RESERVED_PDES	□ the guest handing us real PDE physical addresses for our reserved window
0x20800a9f	GMMU_COPY_RESERVED_SPLIT_GVASPACE_PDES_TO_SERVER	the same, for a GPU-group-global VA space
0xa06f0103 / 0xa06c0101	GPFIFO_SCHEDULE (channel / TSG)	start scheduling this channel, or this whole group
0xa06f0104	NVA06F_CTRL_CMD_BIND	bind a channel to an engine
0x20801210	GR_SET_CTXSW_PREEMPTION_MODE	set graphics context-switch preemption mode
0x20801702	MC_SERVICE_INTERRUPTS	the guest's interrupt-poll bottom half. □ deliberately refused (0x56) to cancel the polling loop
0xa06c0105	NVA06C_CTRL_CMD_PREEMPT	preempt a channel group. □ decided, not echoed — NV_OK only if no member has a live host twin
0x20808159, 0x20808162, 0x20809001/9/64	<i>unnamed GSS-legacy</i>	∅ no open-source symbol exists. Opaque blobs cuInit/cudart demand; answered from measured values. Their categories are GPU_/CLK_/PERF_LEGACY_NON_PRIVILEGED (ctrl2080base.h)
0x20802209	NV2080_CTRL_CMD_RC_GET_WATCHDOG_INFO	the robust-channel recovery watchdog's state. ∅∅ [CORRECTED w821] this was filed as " <i>unnamed GSS-legacy, no symbol exists</i> " — wrong twice : it is named in ctrl2080rc.h:179 <i>and</i> in this tree's own allowlist (capability.rs:811), and bit 15 of 0x2209 is clear so the GSS-legacy rule would not admit it anyway

Forwarded to a real host ioctl — $\emptyset$ exactly one confirmed live path:

cmd	constant	note
0x906f0106	GET_MMU_FAULT_INFO	scoped to a channel's host twin, one ioctl per ask
0x2080a026/84/97	unresolved 0x2080a0xx family	path built but gated off by default = behaves as REFUSE today

□ **[UNVERIFIED]** classify_control/forward_control exist as a general mechanism but have **one call site**. Whether a broader forwarding path exists elsewhere was not established.

Refused by name, with a reason (capability.rs:1506): GPU_EXEC_REG_OPS (0x20800122) and the perf-counter EXEC_REG_OPS (0xb0cc010a) — arbitrary register peek/poke; ALLOC_PMA_STREAM (0xb0cc0105) — hardware performance counters; the SM-debugger trio DEBUG_SET_MODE_MMU_DEBUG / SUSPEND_CONTEXT / RESUME_CONTEXT (0x83de0307/17/18); GPU_REPORT_NON_REPLAYABLE_FAULT (0x20800177) — the fault mechanism is not modelled; and the fabric/NVLink family (0x00e00102, 0x00f10003, 0x20803083).

Admitted but undispatched — ~135 commands pass the allowlist with no handler and fall to the unserviced ledger, returning NV_ERR_NOT_SUPPORTED. They cluster in: GPU/system enumeration (GET_ATTACHED_IDS, GET_PCI_INFO, GET_UUID_FROM_GPU_ID, P2P caps), capability queries (GR_GET_CAPS, FB_GET_CAPS, FIFO_GET_CAPS, HOST_GET_CAPS, DMA_GET_CAPS), descriptive subdevice queries (GPU_GET_INFO_V2, GPU_GET_NAME_STRING, GR_GET_INFO, BUS_GET_PCI_INFO, MC_GET_ARCH_INFO, TIMER_GET_TIME), and third-party-P2P / ZBC / debugger families.

$\emptyset$ **[RESOLVED w821 — and the resolution is that there was no contradiction.]** A first pass reported “*nine decoder modules with no confirmed dispatch site*” and speculated that either a second dispatcher existed or we had built decoders for controls we answer “*not supported*” to. □ **The second dispatcher exists:** kayfabe-device/src/initables.rs (WantedTable, command map at :1146), and **all ten** are wired through it — eventnotify, businfo, gpu-atomics, fbinfo, cecaps, cepce, grfsinfo, gspfeatures, c2cinfo, pcibars, including EVENT_SET_NOTIFICATION, which the first pass called the sharpest instance. □ **The lesson is about the instrument, not the tree:** “*not in the dispatch table I read*” was reported as “*not dispatched*”. A negative from one grep is a statement about the grep.

□ **Two rule-based admissions that are not enumerable as rows:** any control with the **GSS-legacy mask** bit 15 set (ported from gVisor's nvproxy), and any control on a NV2081_BINAPI object. = Both admit *classes of number*, not numbers. □ An allowlist that admits by rule cannot be audited by listing it.

2.4 □□□□ The synchronous verbs — the list the trap path is judged against

This is the answer to the open question in Part 1 §10.2. “*No RM verb on a vCPU*” cannot be checked until we know which verbs the guest issues and then **immediately consumes a value from**. A verb whose caller reads only a status can be deferred; a verb whose caller reads a **value** it then uses cannot.

○○○ **[CORRECTED w821 — the first version of this section asked the wrong question, and the answer it gave was therefore true of the wrong population.]** I listed what the guest driver's callers consume and then marked each “*served / unserviced*” by us. But most of these controls **never reach us at all**: a control is handled by the guest's **own CPU-RM** unless it is flagged ROUTE_TO_PHYSICAL or its handler issues an explicit RPC. = “*unserviced*” named a gap for verbs we are never asked about.

The table below therefore carries the question that actually matters first: **does it reach us?**

verb	reaches us?	value the caller consumes	our obligation
GPU_PROMOTE_CTX 0x2080012b	□ ROUTE_TO_PHYSICAL	$\emptyset$ none — status-only	answer status
CE_GET_CE_PCE_MASK 0x20802a02	□ ROUTE_TO_PHYSICAL	pceMask	□ served
DMA_SET_PAGE_DIRECTORY 0x00801813	□ explicit RPC	$\emptyset$ none — status-only	answer status
□ GR_GET_CTX_BUFFER_INFO 0x20801219	□ explicit RPC	physAddr, size, aperture, pageSize, bIsContiguous	$\emptyset$ [ADDED w821] value-used and reaches us
□ KGR_GET_CTX_BUFFER_PTES 0x20800a28	□ explicit RPC	the PTEs themselves	$\emptyset$ [ADDED w821] value-used and reaches us
FB_GET_INFO_V2 0x20801303	$\emptyset$ only for indices CPU-RM cannot compute	heapSize, heapStart, ...	□ the four indices UVM asks have local handlers — we answer a <i>different</i> caller

verb	reaches us?	value the caller consumes	our obligation
FERMI_VASPACE_A alloc	□ alloc RPC	∅ status only — □ vaBase/vaSize are overwritten from the local VAS after the call	answer status
GPU_GET_ENGINES	∅ no — local	engineCount, engineList[]	∅ not our gap
GPU_GET_GID_INFO	∅ no — local	GUID bytes	∅ not our gap
FIFO_GET_CHANNELLIST 0x0080170d	∅ no — pure local (chid = pKernelChannel->ChID)	hwChannelId	∅ not our gap
CE_GET_CAPS 0x20802a01	∅ no — local kceGetDeviceCaps	capsTbl	∅ not our gap. (_v2 is 0x20802a03)
GPU_GET_MAX_SUPPORTED_PAGE_SIZE	∅ no — local	page size	∅ not our gap
FB_GET_FB_REGION_INFO	∅ no — local	region limits	∅ not our gap
FAULTBUFFER_GET_SIZE	∅ no — local	buffer size	∅ not our gap

□□□ The two status-only verdicts hold — and they are the result worth keeping

Re-verified at **every** issuer, not just the one originally cited:

- **DMA_SET_PAGE_DIRECTORY** — the caller branches on status and returns `memdescGetPhysAddr(vaspaceGetPageDirBase(...))`, computed **locally**. The params struct has no output field at all (`ctrl0080dma.h:802`).
- **GPU_PROMOTE_CTX** — the three CPU-RM-internal issuers the first version did not check (`kernel_graphics_object.c:130`, the path a CUDA process actually takes; `kernel_graphics_context.c:2195`; `kernel_falcon.c:273`) read back only `bInitialSize/bufferId`, **fields they set themselves**.

⇒ **[PROPOSE]** the rule stands: *a synchronous verb must be answerable from state we already hold, and no synchronous verb may require a host round trip.*

□ **One caveat the first version missed, and it is not about values.** After `SET_PAGE_DIRECTORY` returns, the guest's CPU-RM **commits the root locally and re-enables that VA space's channels**. So *deferrable* was argued purely from value-consumption; **ordering against the next doorbell is a separate argument**, and this document does not make it. ∅ That is the remaining open half of the question, not the value half.

∅∅∅ The “highest-value gap” was not a gap — retracted

[RETRACTED w821.] The first version of this section called `NVC36F_CTRL_CMD_GPFIFO_GET_WORK_SUBMIT_TOKEN (0xc36f0108)` “*the load-bearing gap in the whole document*” because it is value-used and unserved. ∅ **On our plane it never arrives.**

`kchannelCtrlCmdGpFifoGetWorkSubmitToken_IMPL (kernel_channel.c:3181)` forwards to firmware only when `bIsVgpuRpcNeeded`, which requires `IS_VIRTUAL(pGpu)` — a **vGPU** guest. We present as a **GSP-client** guest, so the guest's own CPU-RM computes the token locally via `kfifoGenerateWorkSubmitTokenHal_*` from `{runlistId, chId}` it already holds. The control carries no `ROUTE_TO_PHYSICAL` flag, and the local generator returns only `NV_OK` OR `NV_ERR_INVALID_STATE` — **never 0x56**.

□ **What the doorbell plane actually depends on** — and this is the better statement of it — is that the guest's own `chId` and `runlistId`, **chosen by its CPU-RM and handed to us at channel allocation**, are the values it will write to the bell. Our obligation is at *alloc* time, not at token-query time.

□ **And it leaves a live question in the tree, not in the design:** `kayfabe-device/src/sweep.rs` attributes a bench-observed 0x56 to `kayfabe` refusing this very RPC. If the control cannot arrive, that attribution is wrong and the 0x56 came from a neighbouring verb — `GPFIFO_SCHEDULE (0xa06f0103)` and `NVA06F_CTRL_CMD_BIND (0xa06f0104)` **do** RPC. ⇒ One recorder `grep` for `fn 76, cmd 0xc36f0108` settles it. **This is the kind of error this document exists to catch: a symptom attributed to the first plausible cause on the same page.**

□ The UVM gap closes — in the negative

[CLOSED w821.] The first version flagged “*the UVM kernel module was not searched at all*” as the largest survey gap. Searched: `grep CTRL_CMD kernel-open/nvidia-uvm/*.c` returns **zero hits**. UVM issues every RM control through `nvUvmInterface*` into `nv_gpu_ops.c`. ⇒ `nv_gpu_ops.c` **is the complete UVM verb list**, and the table above is not missing a hidden set.

□ What UVM consumes on the doorbell path: the token it writes to the bell (locally generated, per the retraction above) and the `dmaAddress` from `SetPageDirectory`, which it uses as the PDB for TLB invalidates — and which comes from the CPU-RM's **local** `memdesc`, consistent with the status-only verdict.

3. BAR0 MMIO — reads from DRAM, writes to the vCPU

3.1 The asymmetry, and why it exists

BAR0 is the GPU’s register aperture: ~16 MiB of address space where each 4-byte slot is a control register. A normal emulated device traps **both** reads and writes, and pays a VM exit for each. kayfabe does not:

- **Ø Reads are not trapped.** They are served out of **ordinary DRAM** the guest reads directly, with **no exit and no code running**.
- **□ Writes trap** into `nvkvm_trap_bar0_write` on the vCPU thread.

The reason is the trap contract: a vCPU inside an MMIO exit is **not preemptible**, so every trapped read is a stall charged to a core of the customer’s VM. A register the guest *polls* — and driver init polls several thousand times — would be thousands of stalls for a value that did not change. ⇒ Put the value in memory and let the guest read it at DRAM speed.

[MEASURED] **3 572 of 4 096 BAR0 pages are backable in 12 contiguous runs** (`kayfabe-device/src/plane.rs:2560`), against a read census of **241 722** BAR0 reads.

3.2 The shadow — two structures, and they are not the same thing

□ This is the part most often got wrong: “*the shadow*” names **two** objects.

	RegPlane	ShadowSink / ShadowSegment
what	the model : the source of truth that answers a <i>trapped</i> access	the DRAM the guest actually reads
[CODE]	<code>kayfabe-device/src/plane.rs:1223</code>	<code>kayfabe-qemu-raw/src/shim_unsafe.rs:1656, :1626</code>
shape	ChipProfile + GspModel + ROM image + two RankedMutex state cells + a dead-page bitmap	a Vec of {off, len, base} contiguous byte runs, linearly scanned
guest touches it?	Ø never	□ directly, with no exit

Ø **It is not a flat offset-indexed array and not a page hash map.** It is a small vector of merged contiguous runs — which is why a write to an offset no segment covers is **silently dropped** rather than faulting (`shim_unsafe.rs:1666-1677`).

How a page becomes readable-from-DRAM. `bar0_backable_runs()` (`plane.rs:2579`) sweeps the whole aperture in 4 KiB units and admits a page only if **every dword in it** is answerable without running code: a provably dead dword, a boot-register constant, a ROM byte, a CPU interrupt-tree leaf, the BAR0-window latch, an MMU-invalidate register, or something the GSP model can project. Adjacent admitted pages merge into runs.

□ **The content is computed, never loaded from a capture.** `bar0_shadow_fill` (`plane.rs:2743`) fills each run from the *same* per-arm sources the trapped read path uses, and only then does the shim attach that memory (`:1713`). ⇒ There is no capture table to expire as a vendor regression, which is the failure mode §0.3 forbids.

Write-through is a single funnel. `RegPlane::shadow_write` (`plane.rs:3144`) is the only producer, called from the BAR0-window latch, the MMU-invalidate latches and trigger, the interrupt-tree republish, and the GSP register-group republish (`:3933-4013`).

□ **[UNVERIFIED — a discrepancy worth someone’s attention, not yet a confirmed defect.]** Under `feature = “host-isolates”` a second mechanism, `Regs::back_bar0_dead_runs` (`kayfabe-qemu-raw/src/shim.rs:17527`), independently installs a read-only KVM memslot over `bar0_backable_runs()` backed by a memfd its own comments describe as “*never written... zero resident pages*” — **without calling `bar0_shadow_fill` first**. Its prose still carries the pre-w563 *dead-pages-only* meaning while calling the post-w563 *widened* range, which also includes non-zero boot registers and ROM. The test at `crates/kayfabe-device/tests/bar0_backable_pages.rs:143` explicitly records that w563 changed the invariant from “*the run answers zero*” to “*the run answers what `bar0_shadow_fill` computes*.” ⇒ Either the two paths are gated to disjoint builds, or one of them serves zeros where the guest expects `NV_PMC_BOOT_0`. ØØ **[CORRECTED w821] The premise that this could not be settled was false.** I wrote that the call ordering “*crosses the FFI boundary into the C shim, which is not in this repository*.” **The shim is in this repository** — `qemu/hw/misc/nvkvm/nvkvm.c` — and it calls `kayfabe_shim_bar0_shadow_fill` before `kayfabe_shim_bar0_shadow_attach`, in that order. ⇒ The question is answerable here; what remains unchecked is whether `back_bar0_dead_runs` is also invoked from that file. □ **An “I could not determine this” that was never attempted is worse than an open question — it closes the question while looking rigorous.**

3.3 The trapped-write register table

`RegPlane::write`, `plane.rs:6473`. First match wins; the order is itself load-bearing (the doorbell is classified **before the plane lock is taken**, because its port may block).

Offsets are GA10x/GA106. **Axis** column: what moves the offset or the meaning.

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x001700	NV_PBUS_BAR0_WINDOW	the latch that positions the 1 MiB PRAMIN aperture below. Ø [CORRECTED w821] _BASE is 23:0 with a 16-bit shift ⇒ 64 KiB granularity , not megabyte-aligned; and _TARGET (25:24) additionally selects the aperture — vidmem, coherent or non-coherent system. The window is 1 MiB <i>wide</i> ; it is not positioned in megabytes	store the raw word verbatim (a read-modify-write must see back exactly what it wrote); write through to shadow	A
0xBB0090	NV_VIRTUAL_FUNCTION_ - DOORBELL	□ the bell : a work-submission token saying a command ring has new entries	§2 of Part 1 — the security-critical path. Classified with no lock held	Ø die
0xB830A0	..._PRIV_MMU_INVALIDATE_ - PDB	low/aperture half of <i>which page directory to invalidate</i>	latch only, not acted on	A
0xB830A4	..._PRIV_MMU_INVALIDATE_ - UPPER_PDB	high half of that address	latch only	A
0xB830B0	..._PRIV_MMU_INVALIDATE	□ the TLB-invalidate fire register: the guest writes scope + TRIGGER=1, then spin-polls this same register until TRIGGER reads back 0	decode scope, publish address-space changes here (the guest is quiesced — this is the exact GPU boundary), then clear TRIGGER unconditionally . □ A stuck TRIGGER hangs or resets the guest	A
0xB81000+i*4	..._PRIV_CPU_INTR_LEAF(i)	pending-interrupt bitmap, 32 vectors per leaf, write-1-to-clear	update tree, republish the whole group to the shadow, maybe raise an interrupt. No plane lock	A
0xB81200/1400	..._LEAF_EN_SET/_CLEAR(i)	enable mask for leaf i, set and clear ports	same arm	A
0xB81600/1608/1610	..._TOP(i), _TOP_EN_SET/_CLEAR	top-level pending summary and its enables	same arm	A
0xB81640	..._LEAF_TRIGGER	software interrupt injection (write-only)	same arm. □ tested first , to avoid aliasing with the TOP_EN_CLEAR array bound	A

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x110C00+i*8	NV_PGSP_QUEUE_HEAD(i)	the GSP command doorbell : “a new command is in the ring”. This is the whole hardware surface of the RPC plane in \$1	post and return — increment a counter, no lock, no inline drain. [MEASURED] this fixed a 1.79 s vCPU stall (plane.rs:5196). Drained later by a worker, one command at a time	A, see below
0xBB0080/0084	NV_VIRTUAL_FUNCTION_TIME_0/1	the GPU’s free-running nanosecond timer, read by the driver for timeouts	refused by name and counted — accepting a write would put guest and host GPU on different timebases	die
0x118234	NV_PGC6_AON_SECURE_SCRATCH_GROUP_05(0)	firmware-boot-progress scratch byte the driver polls for “ <i>GFW boot complete</i> ”	drive the boot FSM; republish group	A
0x118128	..._GROUP_05_PRIV_LEVEL_MASK	privilege mask that must be fully lowered before the guest trusts the progress byte	same	A
0x110100	NV_PFALCON_FALCON_CPUCTL (GSP)	the bit that starts the GSP microcontroller running	boot FSM; is_startcpu() classifies	A
0x1100F4	..._HWCFG2 (GSP)	reports/gates whether the falcon’s RISC-V core is enabled	boot FSM	A
0x110118	..._DMATRFCMD (GSP)	the ucode-load DMA command/status register	boot FSM	A
0x110040/0044	..._MAILBOX0/1 (GSP)	the two halves of the LibOS boot-args physical address	boot FSM	A
0x110008/0018/001C	..._IRQSTAT/IRQMASK/IRQDEST (GSP)	falcon interrupt status, mask, routing	boot FSM	A
0x110004	..._IRQSCLR (GSP)	write-1-to-clear falcon interrupt status. the guest’s ISR clears here before draining our status queue	boot FSM; is_swgen0_clear() classifies	A
0x111388	NV_PRISCV_RISCV_CPUCTL	RISC-V core control; the ACTIVE/HALTED bits the guest polls for liveness	boot FSM	A
0x111528/152C	NV_PRISCV_RISCV_IRQMASK/IRQDEST	RISC-V interrupt mask and destination, ANDed for stall attribution	boot FSM	A
0x840100	NV_PFALCON_FALCON_CPUCTL (SEC2)	start register of the security co-processor that loads protected firmware	boot FSM	A

offset	register (ogkm swref)	what it is, in plain language	what we do	axis
0x840040	..._MAILBOX0 (SEC2)	the Booter's argument — distinguishes a Load from an Unload of the protected region	boot FSM; <code>is_booter_unload()</code>	A
0x840118	..._DMATRFCMD (SEC2)	SEC2's ucode-load DMA command/status	boot FSM	A
0x1FA824/828	NV_PFB_PRI_MMU_WPR2_-ADDR_LO/HI	base of WPR2 , the write-protected region where GPU firmware lives. <code>!HI</code> is the guest's " <i>is WPR2 already up</i> " test	boot FSM	0 0 die x dg — see below

0 **Everything not in this table is silently dropped** — counted in `unclaimed_writes`, returning `WriteOutcome::nothing()` (`plane.rs:6733-6737`). That includes writes to boot-register constants and to the ROM window, which are read-only from the guest's side and have no write arm at all.

3.4 000 WPR2 is the worked example of two axes crossing in one register

`gb20x.rs:196-227`: on Blackwell the register **moved** to `0x0088A824/828` (`NV_HUBMMU_PRI_MMU_WPR2_ADDR_LO/HI`) for 610-series drivers — **while still answering the legacy `0x1FA824/828` pair for 580-series drivers**. Both offsets decode to the same internal variant.

= **One register, two axes at once**: the **die** moved it, and the **guest driver version** decides which address that guest will use. Neither axis alone explains the behaviour, and a design that modelled only "*per-die offsets*" would answer the wrong address for a 580 guest on a Blackwell part. 0 This is the concrete argument for \$0.2's claim that axes are not independent and a per-die table is not sufficient on its own.

0 Related, and **[UNVERIFIED]**: `NV_PGSP_QUEUE_HEAD` was once believed Turing-only; `gh100.rs:51` records the correction that **GH100's driver does write it**. = An "architecture X doesn't use this register" belief is exactly the kind of claim that expires.

3.5 Whole apertures, not registers

range	name	treatment
0x700000-0x7FFFFFFF	PRAMIN , 1 MiB	one aperture , classified before the register decode. Every dword resolves through the <code>0x1700</code> latch: <code>fb_addr = (base << 16) + window_off</code> , then reads/writes the guest framebuffer store, or refuses by name if untranslatable. 0 A bring-up aperture, not a running path
0x300000-0x3FFFFFFF	PROM/VBIOS , 1 MiB	one range on read only , served from a byte image. 0 No write arm exists at all — writes fall to the dropped path
0xBB0000-0xBBFFFF	NV_VIRTUAL_FUNCTION — the usermode window, 64 KiB	0 not a pass-through aperture — decoded register by register : the doorbell at <code>0xBB0090</code> , the PTIMER halves at <code>0xBB0080/84</code> . Any other offset inside it is dropped

range	name	treatment
0xB80000-0xBAFFFF	NV_VIRTUAL_FUNCTION_PRIV	likewise register-by-register: the CPU interrupt tree at 0xB81xxx and the MMU-invalidate trio at 0xB830A0/A4/B0. 00 [CORRECTED w821] these two rows were one row reading 0xB80000-0xB8FFFF, usermode/VF window, which is the wrong range for both — and it placed the doorbell and PTIMER inside it, contradicting §3.3 of this same document 00 [CORRECTED w821] a first pass claimed “no MSI-X code exists in this tree at all.” False — IrqSpec::Msix (kayfabe-vmm), signal_msix (kayfabe-vmm-qemu/src/host.rs:374), and msix_init in the device model at qemu/hw/misc/nvkvmm/nvkvmm.c. Whether the table sits inside BAR0’s range is still [UNVERIFIED]
MSI-X table / PBA	—	separate PCI BARs, 0 never trapped — ensuring that is a requirement (Part 1 §2)
BAR1 / BAR2	framebuffer / instance windows	

4. Channels and pushbuffers — the plane unprivileged guest code drives

4.1 What a channel is, for a reader who does not know NVIDIA’s constants

A **channel** is one command stream to the GPU. It has three pieces of memory:

- a **pushbuffer** — a byte array of commands (“methods”), written by the guest;
- a **GPFIFO ring** — an array of 8-byte entries, each pointing at a run of pushbuffer;
- **USERD** — a small per-channel block holding the ring cursors GP_PUT (producer, written by the guest) and GP_GET (consumer, advanced by the hardware).

To submit work the guest appends a GPFIFO entry, bumps GP_PUT, and **rings the doorbell**. The hardware’s front-end then reads GP_PUT, fetches entries, and executes the methods.

□□□ GP_PUT lives in **USERD, which is mapped to the owning process and nobody else**. That is the whole reason a hostile doorbell is harmless on real hardware and dangerous for us: hardware re-reads memory only the owner can write, and we *parse the owner’s pushbuffer*. See Part 1 §2.1.

4.2 The three kinds — and an honest statement of what is built

kind	contract	what it means
Passthrough	RingAndReturn — “no inspection and no work”; may run on the vCPU	the guest’s ring is in real GPU memory; hardware fetches it directly. We never read a byte of it
Translated	ScheduleAndReturn; must not run on the vCPU	every entry’s operands rewritten into our own VA space, then submitted on our host channel. The GPU moves the bytes
Emulated	ScheduleAndReturn; must not run on the vCPU	we implement the function in the VMM

[CODE] GuestChannelKind, kayfabe-core/src/channel_kind.rs:96-158; Contracts at :286-315.

000 The classifier is a two-way switch, not a three-way one, and Translated is never constructed. ProcBoundary::channel_kind() (kayfabe-core/src/project.rs:308-316) is:

```
if self.anchor == SYSTEM_ANCHOR { Emulated } else { Passthrough }
```

⇒ A channel is **Emulated** iff its client namespace is the one reserved *system* component (the guest **kernel**’s own client — UVM, the CeUtils scrubber), and **Passthrough** otherwise. □ **No source location tests a class id to decide the kind**. The class id only matters later, for method decode and engine routing.

⊗ GuestChannelKind::Translated is a fully-typed third variant with complete contracts and exhaustive-match coverage everywhere — and **no production path ever assigns it to a channel**; a Translated birth is refused outright (kayfabe-fwd/src/lib.rs:4324). □ **[CORRECTED w821]** I called channel_kind.rs:355 “a test fixture array”; it is pub const ALL, outside #[cfg(test)]. The substantive claim is unaffected — but the parenthetical was wrong, and it was the kind of detail that makes a reader trust the rest. The design doc that introduced it (the_three_channel_kinds.md, STATUS: DESIGN, w803) says “not yet implemented”, and that is still true. ⇒ **Real kernel-CE work the design says should be Translated is today classified Emulated** and driven through the CPU/host-CE forwarding path in §4.5. □ This is the gap behind the scrub defect in Part 5.

class (name + hex)	what it is	kind today	axis
AMPERE_CHANNEL_GPFIFO_A 0xC56F — user CUDA context	a userspace channel a CUDA process owns	Passthrough	A
AMPERE_CHANNEL_GPFIFO_A 0xC56F — guest-kernel / UVM / CeUtils	a channel the guest kernel driver owns	Emulated (design target: Translated)	A
AMPERE_DMA_COPY_B 0xC7B5	the copy engine class object — does DMA copies and memsets	inherits its channel’s kind	A
AMPERE_COMPUTE_B 0xC7C0	the compute class object — kernel launches	inherits; routed HostGr, real hardware runs it, no in-process executor	A
GP100_UVM_SW 0xC076	⊗ not a real engine . A software placeholder so UVM’s channel allocation succeeds, and a subchannel to hold fault-cancel methods	rides its (kernel) channel → Emulated	stable
VOLTA_CHANNEL_GPFIFO_A 0xC36F, TURING_..., HOPPER_... 0xC86F, BLACKWELL_... 0xC96F	the same channel object, per architecture	kind is per-namespace, not per-class	⊗ A
AMPERE_USERMODE_A 0xC561	⊗ not a channel — the BAR window object that exposes the doorbell register itself (Part 1 §2.1). ⊗⊗ [CORRECTED w821] this row read 0xC361, which is VOLTA_USERMODE_A — contradicting §2.2 of this same document	n/a	A

4.3 The GPFIFO entry — 8 bytes

[CODE] GpfifoEntry, kayfabe-abi/src/submit.rs:1836; decode at :1852. [ogkm] NVC56F_GP_ENTRY0/1, clc56f.h:262-284.

field	bits	meaning
GP_ENTRY0_FETCH	0:0	unconditional / conditional fetch (not modelled)
GP_ENTRY0_GET	31:2	pushbuffer address, bits 31:2
GP_ENTRY1_GET_HI	7:0	address bits 39:32
GP_ENTRY1_LEVEL	9:9	main run, or a subroutine call
GP_ENTRY1_LENGTH	30:10	length in dwords
GP_ENTRY1_SYNC	31:31	PROCEED, OR WAIT — the front-end must drain before fetching

□ LENGTH == 0 is treated as a *control* entry whose low byte is an opcode (GP_ENTRY1_OPCODE, 7:0) rather than an empty run, so GET_HI is not read there. We refuse it rather than decode it (submit.rs:1822). □ **[UNVERIFIED w821]** clc56f.h defines GP_ENTRY1_OPCODE but **does not state the LENGTH == 0 rule**. An earlier version asserted it as fact with a citation that does

not carry it. The behaviour is conservative either way — we refuse rather than act — but the *reason* is inferred, not sourced.

□ **Axis note:** NVC36F (Volta) and NVC56F (Ampere) differ in exactly two bits across the whole entry — C56F **dropped** PRIV (8:8) and **gained** INVAL_SCOPE in MEM_OP_A. ⇒ This encoding is far more A-stable than the register offsets in §3. We model neither bit.

4.4 The method header — one dword

[CODE] method_header_decode, submit.rs:2196. [ogkm] NVC56F_DMA_*, clc56f.h:286-360 — bit-identical to NVC36F_DMA_*.

field	bits	meaning
SEC_OP	31:29	which of 8 encodings this dword is dwords that follow, or the immediate payload itself
METHOD_COUNT / IMMD_DATA	28:16	
METHOD_SUBCHANNEL	15:13	which of 8 bound engine objects this addresses
METHOD_ADDRESS	11:0	□ dword-indexed — multiply by 4 for a byte offset

SEC_OP values: GRP0_USE_TERT(0), INC_METHOD(1), GRP2_USE_TERT(2), NON_INC_METHOD(3), IMMD_DATA_METHOD(4), ONE_INC(5), RESERVED6(6), END_PB_SEGMENT(7).

□□□ **Sizing and decoding are deliberately different jobs.** We *size* (skip correctly past) all eight forms, so the stream never desynchronises; we *decode the payload of* only incrementing runs. ∅ RESERVED6 has no defined format, so it decodes to nothing rather than to a guess. **[PROPOSE]** this split — *always know the length, decode only what you model* — is the right shape for every parser in this system, and it is what lets an unmodelled method be harmless.

4.5 Every pushbuffer method we recognise

method (ogkm constant)	offset	engine	what it does	what we do
NVC56F_SET_OBJECT	0x000	host FIFO	binds a class to a subchannel — this is how the stream says “the next methods are for the copy engine”	bind/clear per-subchannel decode state (ga10x.rs:1762)
NVC56F_SEM_ADDR_LO/_HI, SEM_PAYLOAD_LO/_HI, SEM_EXECUTE	0x5C-0x6C	host FIFO	the channel’s own semaphore release : write a payload to a GPU address when this point in the stream retires. This is how the guest learns work finished	decoded only for OPERATION == RELEASE; observed on the completion queue (ga10x.rs:1160)
NVC56F_MEM_OP_A/B/C/D	0x28-0x34	host FIFO	the TLB invalidate transport, targeted at one page directory	decoded for MMU_TLB_INVALIDATE/_TARGETED with PDB_ONE; counted, and the membar honoured as a hard barrier. ∅ no real invalidate is issued (ga10x.rs:1558)
NVC7B5_OFFSET_IN_UPPER/_LOWER, OFFSET_OUT_UPPER/_LOWER	0x400-0x40C	CE	source and destination addresses of a copy	latched (ga10x.rs:1195)
NVC7B5_LINE_LENGTH_IN, LINE_COUNT	0x418,0x41C	CE	how many bytes per line, and how many lines	latched

method (ogkm constant)	offset	engine	what it does	what we do
NVC7B5_SET_SRC/DST_PHYS_MODE_TARGET	0x260,0x264	CE	which physical aperture (coherent system / non-coherent system / vidmem) a physical operand names	latched; read only when that operand is physical
NVC7B5_SET_REMAP_CONST_A/_B, SET_REMAP_COMPONENTS	0x700-0x708	CE	the constant-fill / component-swizzle parameters — this is how a memset is expressed	latched, consumed by remap decode
NVC7B5_SET_SEMAPHORE_A/B, _PAYLOAD[_UPPER]	0x240-0x24C	CE	the copy engine's own completion semaphore address and payload	latched, consumed inside LAUNCH_DMA
NVC7B5_LAUNCH_DMA	0x300	CE	☐ starts the copy. Carries no operands — every operand was latched by the methods above; the method word itself is all flags	the whole execute decision (ga10x.rs:1453, kayfabe-fwd/src/lib.rs:8724). See below
NVC076_SET_OBJECT, NO_OPERATION	0x000,0x100	UVM SW	routine bind / no-op on the software class	ordinary; explicitly excluded from the tripwire
NVC076_FAULT_CANCEL_A/B/C, CLEAR_FAULTED_A/B	0x104-0x114	UVM SW	UVM cancelling or clearing a real MMU fault	☐ refuses the whole parse — UvmFaultMethodWithoutFaultDelivery (lib.rs:8919). We have no fault delivery, so seeing one means the assumption behind admitting this class broke. ☐ A tripwire, not an error
NVC36F_NON_STALL_INTERRUPT	0x020	host FIFO	☐ not recognised anywhere. grep across all crates returns nothing	falls through to opaque
everything else	—	—	WFI, YIELD, NOP, CRC check, sub-device masks, legacy forms, non-incrementing and immediate runs	sized, not decoded — counted as opaque, stream stays in sync

☐ **NON_STALL_INTERRUPT being unrecognised is a real gap against Part 1 §5.** That section says the interrupt is *requested in the pushbuffer*, and this is the method that requests it. We currently do not see it. ☐ Today that is survivable only because the channels we care about **poll** — UVM spins, and the scrubber's blocking waits loop on the semaphore word — but it is an assumption about the guest, not a property of our design.

☐ **Why LAUNCH_DMA is not in the single-method decoder:** a real copy is **five separate method runs**, and LAUNCH_DMA carries none of the operands. Only the stateful walker that threads an accumulator across the whole run can produce a copy fact (ga10x.rs:1730). **[PROPOSE]** this is the general shape — *a method is not a unit of meaning; a run is* — and any redesign that decodes methods individually will reintroduce this bug.

4.6 What happens when each kind's doorbell rings

Passthrough. The host channel was adopted at **channel allocation**, never lazily (`plan_channel_birth`, `lib.rs:5576`) — mandatory, because RM **zeroes a caller-supplied USERD at alloc time**, so adopting at first doorbell would wipe the cursor that just rang. The doorbell resolves the guest token to the already-adopted host token, runs the ring gate, and writes the real host doorbell. **Ø No pushbuffer byte is ever read** — `device.rs:4110` says so by name, and `ring_content_is_forwardable` returns `false` for this kind. Completions are hardware's own writes to guest memory; we observe nothing and forge nothing.

Emulated. Born lazily at first doorbell if needed. Handed to a worker — must not run on the vCPU. If the bound engine routes to `CpuCe`, the ring is parsed: read the guest's pushbuffer, decode runs into methods, then for each `LAUNCH_DMA` partition the copy into spans by address representability, classify each as a **data copy** (a VA operand) or a **page-table write** (a physical operand — latched and applied to the *owning* process's VA space, which may not be the issuer). Host-CE spans go to the real copy engine and **the real engine writes the guest's completion**. If the engine routes `HostGr`, the doorbell is forwarded but **Ø the ring is never parsed** — “no executor IN THIS PROCESS runs GR work” (`device.rs:8930`).

ØØØ The measured defect, in one line: CPU-executed spans have **zero completion writers** — `write_completion` (`kayfabe-rt/src/cpu_ce.rs:736`; **Ø [CORRECTED w821]** an earlier pointer said `kayfabe-fwd/src/lib.rs:8862`, which is a *comment about it*) has **no callers anywhere in crates/**, pinned by `tests/tests/single_writer_census.rs:360`. A completion on that arm is **silently dropped**. Combined with the scrub being classified `Emulated` rather than `Translated`, this is the cross-client leak in Part 5.

Translated. **Ø** Design only; no doorbell reaches it. The intended path is in `the_three_channel_kinds.md §4`: read `USERD`, advance `GP_GET`, bounded-copy the pushbuffer, rewrite every operand aperture `PHYSICAL → VIRTUAL` into one static VA space, submit on our host channel, and translate the completion **in address only** — the real engine writes the value. An untranslatable operand is meant to be forwarded as a **deliberate fault at a named sentinel VA**, never emulated and never silently dropped. —

5. Ø Where this inventory is knowingly incomplete

ØØØ This section is the point of the document, not an apology. An inventory whose gaps are unmarked reads as complete, and this project's most expensive recurring failure is a plausible value that was never measured. Everything below is a place a survey **stopped**, stated so a reader can argue about it or go and close it.

5.1 Gaps in what we implement

#	gap	why it matters	where
1	Ø Translated is never constructed. The kind exists, is fully typed, has contracts and exhaustive-match coverage, and zero live construction sites	Everything the design says should be <i>translated</i> is today <i>emulated</i> — including the scrub. This is the gap behind the live cross-client leak	§4.2
2	Ø CPU-executed copy spans have no completion writer. <code>write_completion</code> has no call sites	A completion on that arm is silently dropped . A guest waiting on it waits forever or proceeds on stale data	§4.6
3	Ø <code>NVC36F_NON_STALL_INTERRUPT</code> is not recognised anywhere	It is the method by which a guest <i>requests</i> the completion interrupt Part 1 §5 is built around. We survive only because the channels we care about poll — an assumption about the guest, not a property of our design	§4.5

#	gap	why it matters	where
4	∅ No fault delivery exists. <code>NVC076_FAULT_CANCEL_*</code> refuses the whole parse	□ Correctly built as a tripwire rather than an error: it fires exactly when the assumption behind admitting the UVM software class breaks	\$4.5
5	∅ The TLB-invalidate method is decoded and counted, but no invalidate is issued	Modelled structurally with nothing behind it	\$4.5
6	□ Two BAR0 shadow-install mechanisms, possibly overlapping , one of which maps a never-written zero memfd over a range that now includes non-zero boot registers	If both are live, one serves zeros where the guest expects <code>NV_PMC_BOOT_0</code> . Could not be settled — the call ordering crosses the FFI boundary into the C shim, which is not in this repository	\$3.2

5.2 Questions a survey could not answer

#	question	why it was not closed
7	Does any individual <code>GSP_RM_CONTROL / GSP_RM_ALLOC</code> handler block on a host call before replying?	The top-level dispatch does not. The ~15 per-control handler files were not exhaustively checked. □ This is the same question as the synchronous-verb list, and it is the one the owner is waiting on
8	Does the status (reply) queue have its own outbound doorbell, or is it purely guest-pollled?	No explicit statement found; only its absence from the code read. Consistent with the C oracle's <code>IrqRaise == 1</code> with zero interrupt-clear writes, but not proven
9	Is there a top-level RPC function a stock guest sends at init that we fail to classify?	Searched; none found. Absence not proven
10	Does the doorbell token encoding really differ per Blackwell die group?	Carried in this project's notes; not confirmed against <code>gb20x</code> code. The doorbell <i>offset</i> is identical Volta→Blackwell
11	Where does the MSI-X table live, and is it inside BAR0?	∅ No MSI-X code exists in this tree at all — only a vector count. The table and delivery are QEMU's native model on the C side
12	□ <code>sweep.rs</code> attributes a bench 0x56 to kayfabes refusing <code>GPFIFO_GET_WORK_SUBMIT_TOKEN</code> — a control that cannot reach us on a GSP-client guest (\$2.4)	⇒ The attribution is wrong and the 0x56 came from a neighbouring verb (<code>GPFIFO_SCHEDULE / BIND</code> , which do RPC). □ One recorder grep for <code>fn 76, cmd 0xc36f0108</code> settles it
13	□ A too-large SET_REGISTRY would arrive as an ignored fn-73 head plus fn-71 fragments the bridge refuses — i.e. we would <i>reply</i> to a no-wait RPC	∅ Our reassembly handles only the <code>GSP_RM_CONTROL</code> case though three functions fragment (\$1.3). □ Whether a stock guest's registry ever exceeds one element is unmeasured
14	□ <code>GP100_UVM_SW</code> is allowlisted but not routed through <code>classify()</code>	An <i>admitted-but-unreachable</i> class. <code>[MEASURED]</code> all 4 allocs in a boot refused 0x56. ⇒ The allowlist and the dispatcher disagree, and the allowlist is the one that reads as intent

#	question	why it was not closed
15	□ back_bar0_dead_runs vs the C shim's fill-then-attach ordering (§3.2)	Now answerable <i>in this tree</i> — the shim is <code>qemu/hw/misc/nvkm/nvkm.c</code> . What remains is whether that file also invokes the Rust path

5.3 ∅ Axis coverage — the honest statement

Everything in this part was surveyed against **one point of the lattice**:

GPU: Ampere GA106 · **guest OS:** Linux · **guest driver:** 580/610 · **VMM:** QEMU

⇒ The **axis** column in each table says what *should* vary. It does **not** certify that the entry has been exercised at a second point on that axis. Three specific consequences:

- □ **Windows guest has zero coverage.** Every guest-side inference here is Linux-shaped until proven otherwise. The C artifact never ran one either.
- □ **Turing is not covered and is not the easy end.** NVIDIA binds Turing to a *different page-table format family* from the one built here; spanning Turing→Blackwell needs four.
- □ **The cross-axis cases are the dangerous ones.** §3.4's WPR2 register — moved by the **die**, with *which* address in play decided by the **guest driver version** — is the only one this survey found. ∅ It is very unlikely to be the only one that exists.

5.4 What would close these

[PROPOSE], in the order I would do them:

1. **#2 and #1 together** — they are one defect wearing two hats, and they are the live security bug. A completion that is dropped and a channel kind that was never built are both *“the scrub does not really happen.”*
2. **#12** — one grep, and it either confirms an instrument or retires a wrong attribution that is currently sitting in the tree as a comment.
3. **#6 / #15** — the shim is in this repository after all; one read settles whether two shadow installers overlap.
4. **#3** — cheap to decode, and it converts an assumption about guest behaviour into a fact we handle.
5. The axis gaps in §5.3 — □ these are not a task, they are a **standing property** of the product, and they want a second point on the lattice in CI rather than a one-off audit.

5.5 ∅∅∅ What the review of THIS document found, kept as the record

Two adversarial passes over this bundle found **more defects in the document than in the design**, and the pattern is worth stating because it recurs:

- **Two claims were true of the wrong population** — the *“highest-value gap”* that cannot reach us, and *“nine decoders with no dispatch site”* that are all dispatched. Both were built from a correct observation over an incomplete set.
- **One “could not be determined” was never attempted** — the C shim is in this repository. □ That is worse than an open question, because it **closes** the question while looking rigorous.
- **One internal contradiction survived both a table and its own prose** — `0xC361` VS `0xC561` for the same class, in two sections of one file.
- □ **The two headline results survived intact**: both page-directory verbs really are status-only, and the doorbell really is an unprivileged surface. ⇒ The defects clustered in the *supporting* detail, which is exactly where a reader stops checking.

Part 3 — The Windows axis

STATUS: LIVE, 2026-09-20 (w821). First survey of the os axis (\$0 of `THE_ARCHITECTURE_v3.md`), which until now carried **zero coverage**. Sources: ogkm 610.43.02 and 580.159.04 (the shared RM core), plus lawfully published Microsoft/NVIDIA documentation, vendor bulletins, open-source projects and conference talks.

∅ **No leaked source or breach material was used.** Two cited items are published security research that reverse-engineers a *shipped binary* (Google Project Zero; a public patcher repo) — legitimate published work, flagged where used. One conference deck was encountered and deliberately **not** relied on; nothing here depends on it.

0. □□□ Why this is answerable at all — and exactly where the oracle stops

[owner, 2026-09-20] “NVIDIA open-sourced ogkm under MIT/GPL and still had Windows code in it — that’s so valuable. They could have stripped it with `#if` blocks and then stripped when distributing source for Linux, but they didn’t.”

□ **Correct, and the reason is structural rather than an oversight.** `src/nvidia/` is **one RM codebase compiled for several operating systems**. Stripping per-OS for the public drop would mean maintaining a *divergent* public tree — permanently more expensive than shipping the seams. So the seams ship: **40** `RMCFG_FEATURE_PLATFORM_WINDOWS` sites in 610, **45** `NVOS_IS_WINDOWS`, ~86 WDDM/LDDM comments, and Windows-only control commands documented as such in the SDK headers.

⇒ **ogkm is a genuine partial oracle for Windows behaviour** — the only one that exists, since nouveau and envytools document the hardware and the Linux blob and say nothing about `nvlddmkm`.

∅ **And here is precisely where it stops**, because the distinction decides which claims in this document are sound:

we CAN see

□ **Policy and decisions** in the shared core — which branch is taken on Windows, and why Windows-only **control commands and their semantics** (`UPDATE_PDE_2`, `RESERVE_ENTRIES`)

Comments stating Windows behaviour in NVIDIA’s own words

we CANNOT see

∅ **The Windows OS layer.** `arch/nvalloc/win/` is absent; only `unix` is published

∅ **The Windows rmconfig profile** — `rmconfig.h` hard-sets `RMCFG_FEATURE_PLATFORM_WINDOWS 0` and `NVOS_IS_WINDOWS 0` *unconditionally*, so every NVOC table here is the UNIX binding

∅ Every `os*()` value on Windows — `osGetPageSize`, `osQueueDpc`, `osIsRaisedIRQ`

□□□ **The `WindowsFirmwarePolicyArg` in §1 is the exact shape of that boundary, in one symbol:** the **parameter** is public and threaded through the shared policy function, and the **struct it points at is not defined anywhere in the published headers**. ⇒ They stripped the OS layer and kept the seams *into* it. We can therefore see **that** a Windows-specific input exists and **where** it applies — and never **what it contains**.

□ The practical rule this imposes on everything below: **a claim sourced from the shared core is strong; a claim about what the Windows OS layer supplies is inference and is marked as such.**

1. ∅∅∅ THE HEADLINE WAS WRONG — GSP is default-ON for Turing+, on every OS

[RETRACTED w821, within the hour, by the owner: “are you sure the stock Windows driver doesn’t turn GSP on when it’s available? On Blackwell it’s even required.”]

The survey reported “GSP is off by default on a stock Windows guest” from community evidence. ∅ **The shared RM core says otherwise, and it is the authority here** — the policy is computed in `src/nvidia/src/kernel/gpu_mgr/`, which is **not** the Unix layer.

```
gpumgrIsDeviceRmFirmwareCapable (gpu_mgr.c):
```

```
if (!hypervisorIsVgxHyper() && !_gpumgrIsRmFirmwareCapableChip(pmcBoot42))
    { bFirmwareCapable = NV_FALSE; goto finish; }    // capable ⇐ arch >= TU100
...
if (hypervisorIsVgxHyper()) { ...vGPU-specific chips... }
else                          { bEnabledByDefault = NV_TRUE; }    // □ every other case
```

⇒ **For any Turing-or-newer chip outside a vGPU hypervisor, `bEnabledByDefault` is `NV_TRUE`.** □ There is **no OS conditional in this function at all** — the sole platform carve-out is PowerPC. And `gpumgrGetRmFirmwarePolicy` then requests firmware unless the registry *explicitly* says `DISABLED`:

```
*pbRequestFirmware = bFirmwareCapable &&
    (mode == MODE_ENABLED || (bEnableByDefault && mode != MODE_DISABLED));
```

⦿ And the registry key does not mean what the community advice implies

nv-firmware-registry.h: MODE_DISABLED 0x0, MODE_ENABLED 0x1, MODE_DEFAULT 0x2, POLICY_ALLOW_FALLBACK 0x10 — and the **default value is 0x12**, i.e. MODE_DEFAULT | POLICY_ALLOW_FALLBACK.

⇒ Setting EnableGpuFirmware = 1 does **not** flip GSP from off to on. It sets MODE_ENABLED (*force*) and **drops** ALLOW_FALLBACK, because 0x1 does not carry the 0x10 bit.

□□□ **Which supplies a better explanation of the community reports than “off by default”:** GSP was requested, **fell back to monolithic RM**, and setting 1 removed the fallback. The comment in NVIDIA’s own header describes exactly that mode — “*try to enable GPU firmware but fall back if needed... this can result in a mixed mode configuration (ex: GPU0 has firmware enabled, but GPU1 does not).*”

⦿⦿⦿ AND THE RETRACTION ABOVE OVER-CORRECTED — the struct IS defined, and it changes the answer

[CORRECTED AGAIN, same hour.] I wrote that WindowsFirmwarePolicyArg was “*never read in the open tree, and the struct is not defined in any published header.*” ⦿ **The struct is defined**, in generated/g_gpu_mgr_nvoc.h:601:

```
typedef struct WindowsFirmwarePolicyArg {
    NvU32 devId;           // PCI device id
    NvU32 ssId;            // PCI subsystem id
    NvU32 bEnableGpuFirmwareOnWsServerSkus;
    NvBool bIsTccOrMcdm;
} WindowsFirmwarePolicyArg;
```

□□□ **Those four fields say what the Windows default depends on, and it is not uniform:** the **SKU** (devId + ssId), whether this is a **workstation/server SKU** — matching the policy bit NV_REG_ENABLE_GPU_FIRMWARE_POLICY_DEFAULT_ON_WS_SERVER 0x20 in the registry header — and whether the GPU is in **TCC/MCDM** rather than WDDM.

⇒ □ **A per-SKU default with an explicit workstation/server enable is exactly the shape that produces “GSP is on for my Quadro and off for my GeForce.”** The community reports become **plausible again**, for consumer SKUs specifically.

⦿⦿⦿ How I got the retraction wrong — I read a build with the branch compiled out

gpumgrIsDeviceRmFirmwareCapable’s body in this tree **never references** pWinRmFwPolicyArg — it only writes pbEnabledByDefault. I read that body, saw no OS conditional, and concluded “*there is no OS branch in the decision.*”

⦿ **But rmconfig.h hard-sets RMCFG_FEATURE_PLATFORM_WINDOWS 0, so what I read is the Linux-compiled view of a function whose Windows logic is absent from the public drop** — NVIDIA kept the signature and the struct and stripped the body that uses them. □ **The absence of a Windows branch in a build where Windows is disabled is not evidence that no Windows branch exists.** □ Third instance of this tree’s own lesson in one session, in a third disguise: *an empty result is evidence of nothing, not evidence of emptiness.*

□ So what IS established, stated at the right strength

claim	status
Windows runs as a GSP client	□ Proven. RMCFG_FEATURE_PLATFORM_WINDOWS && IS_GSP_CLIENT(pGpu) at gpu_registry.c:224 and gpu_user_shared_data.c:335, selecting a constant named for that configuration : NV_REG_STR_RM_RUSD_POLLING_INTERVAL_WINDOWS_GSP 250 (VS_DEFAULT 500). Plus bGspNocatEnabled, a Windows-only field in the GSP boot RPC itself
The Windows default is per-SKU and per-mode	□ Proven from the struct’s fields — devId, ssId, WS/server, TCC/MCDM
Whether a stock GeForce Windows guest defaults GSP on	⦿ UNRESOLVED — and §1.1 establishes that NOBODY HAS MEASURED IT

claim	status
GSP is required on Blackwell — on Linux	☐ Yes, and the mechanism is in this tree. [owner] “<i>proprietary Linux doesn’t work on Blackwell, so that infers GSP is required anyway.</i>” Confirmed: the open module refuses a non-firmware-capable GPU by name — <code>osapi.c:3721</code> calls <code>gpumgrIsDeviceRmFirmwareCapable</code> and on <code>NV_FALSE</code> prints “<i>installed in this system is not supported by open nvidia.ko</i>”. ⇒ Blackwell + open-modules-only + openrm-is-GSP-only ⇒ GSP required ☐ Does not transfer. <code>nvlddmkm.sys</code> is not the open module, so the Linux chain says nothing about it. ☐ But <code>devId/ssId</code> are policy inputs, so a per-SKU Blackwell default is expressible either way
...and on Windows	

1.1 000 What the public record actually contains — searched w821

[owner] “*maybe find on the internet. GSP Windows Turing+.*” Done. The result is not an answer; it is the discovery that **there is no published answer, and the folklore has no measured basis.**

1. NVIDIA has published nothing about Windows. The authoritative source is the driver README’s GSP chapter, and it is **Linux-only**: “*The GSP firmware will be used by default for all Turing and later GPUs*” — stated for the Linux driver, configured by the **kernel module parameter** `NVreg_EnableGpuFirmware`. ☐ The document **makes no reference to Windows at all**, and NVIDIA documents no Windows registry setting, Control Panel option or app control for it.

2. ☐☐☐ The community claim has no before-state behind it. The primary thread — the one every later citation traces back to — contains **zero documented observations of the default state**. Posters report results *after* setting `EnableGpuFirmware=1`; nobody recorded what `nvidia-smi -q` said **before**. ⇒ “*GSP is off by default on Windows*” is an **inference from the existence of a registry key that people found worth setting**, not a measurement.

3. ☐ And the “N/A means disabled” reading is borrowed from the wrong document. It comes from NVIDIA’s **Linux** README (“*a valid version if GSP firmware is enabled, or N/A if disabled*”) applied to Windows `nvidia-smi`. ☐ That is precisely the misapplication this tree already has a lesson for: an unpopulated field is not a measured zero.

4. ☐ One Blackwell datapoint cuts the other way. A user on an **RTX 5090, driver 581.94** reports a watchdog error that “*immediately stopped after disabling GSP*” — which implies GSP was **on**, and that it is **disableable** on Blackwell/Windows. ☐ Forum-grade, single report, and it does not distinguish “on by default” from “on because they had enabled it”.

5. ☐☐☐ The claim traced to its root — one user, one card, 2023, no vendor reply. The NVIDIA Developer Forums thread every chain leads back to is “*Enable GSP on Windows 11 on my 2080 Ti?*”, **26 November 2023**. The poster observes `GSP Firmware Version : N/A`, finds `gsp_tu10x.bin` in the DriverStore, and writes “*so it is disabled by default, but the driver has GSP binary.*” ☐ **No NVIDIA staff member replied.** One card, no driver version, no before/after, and the conclusion rests entirely on the `N/A` reading. ⇒ **That single post is the origin of the whole claim.**

☐ **But the datapoint is at least ON-POINT, and it is worth saying why.** The RTX 2080 Ti is **TU102 — Turing**, so it sits *inside* the capable set (`arch >= TU100`, `$0.0.1`); pre-Turing would be Pascal and earlier. ⇒ The `N/A` is **not** explained away by “this die has no GSP”. ☐☐☐ And the blob the poster found is named `gsp_tu10x.bin` — firmware for *that very chip family*, shipped in the Windows DriverStore. ⇒ The card was capable **and** its firmware was present **and** it still reported nothing. That is the right kind of observation; what it lacks is replication and a known-positive, not relevance.

6. NVIDIA’s ONLY published sentence connecting Windows and GSP is scoped to a different product. In the vGPU user guide’s “*Disabling GSP Firmware*” task: “*For NVIDIA vGPU deployments on Linux and all NVIDIA vGPU software deployments on Windows, omit this task.*” ☐ That is about **NVIDIA vGPU software deployments** — the licensed vGPU/GRID product — not a stock GeForce Windows guest. ☐ It is suggestive (you would not tell people to skip a disable step for something that is on and problematic), but it does not address our configuration and must not be stretched to.

000 And the precise reason the claim is untested: it has no known-positive

The `N/A` inference may well be **right** — `nvidia-smi` is NVIDIA’s own tool and largely shared, so an empty field plausibly does mean “no GSP version to report”. ☐ **It has simply never been checked against a known-positive**: nobody has published a Windows card reporting a GSP version **without** the registry key, on any SKU.

□ And this tree already names that requirement: `a_census_zero_needs_a_known_positive`. A zero from an instrument nobody has seen produce a non-zero, in that configuration, is not a measurement. □ The guru3D posters who set the key *and then saw a version* prove the **instrument works on Windows** — which is exactly what makes the missing observation cheap to take and inexcusable to keep inferring.

⇒ □□□□ **The convergence is the useful part.** The open tree says the Windows default is decided **per-SKU and per-mode** (`devId`, `ssId`, `WS/server`, `TCC/MCDM`). The public record says **nobody has measured it**. Those two facts fit together exactly: *a per-SKU default is what produces contradictory folklore*, because different people are correctly reporting different cards.

⇒ **[PROPOSE] Stop trying to settle this from sources. It is a one-hour measurement** — a Windows box, a GeForce part, `nvidia-smi -q` **before** touching the registry — and until someone runs it, every claim in either direction is folklore, mine included.

Sources: [NVIDIA 580.65.06 GSP chapter](#) · [guru3D: Enable GSP Firmware on Windows](#) · [guru3D: How to disable NVIDIA's GSP Firmware on Windows](#) · [Overclock.net: RTX 5090 GSP Firmware](#)

□ **And one thing the “Turing+” framing in that search should not be allowed to blur** [owner: “pre-Turing it can't be used”]: **Turing+ is the HARDWARE capability gate** (`_gpumgrIsRmFirmwareCapableChip`, `arch >= TU100`), not a Windows policy statement. Below Turing there is no GSP to enable on any OS. ⇒ Everything uncertain in this section sits **inside** the capable set, in the *default-on policy* — see `THE_ARCHITECTURE_v3.md §0.0.1`.

⇒ □ **The practical position:** design for GSP being present, **detect and refuse the alternative** (§1's fallback detector), and treat “*stock GeForce Windows defaults GSP off*” as an open risk to be measured — not as a settled premise in either direction.

□□□ How the survey got it wrong, and it is this tree's own documented failure

The community evidence rested partly on `nvidia-smi -q` reporting `GSP Firmware Version : N/A`. ○ **That is not evidence GSP is off. It is evidence the field was not populated.** □ This repo already carries that lesson, paid for once: *an empty capture is evidence of NOTHING, not evidence of emptiness* — the `dlen=0` rows whose every checked instance was **contradicted** by real hardware. ⇒ The survey reproduced the exact error the tree documents, from a different direction.

□□ What replaces it, and it matters on every OS

○○ **The default policy ALLOWS SILENT FALLBACK TO MONOLITHIC RM.** That is 0x12's 0x10 bit, and it is the default on Linux too.

⇒ **If a guest's RM falls back, our fake GSP is never used at all** — monolithic RM drives our emulated registers directly, and every RPC-based assumption in this document evaporates **with no error and no message**. □ **[PROPOSE]** we must be able to detect that and refuse it by name. The detector is cheap: a guest that has fallen back **never publishes a message queue** and never sends `SET_GUEST_SYSTEM_INFO`, so “*boot progressed past the point where an RPC was due, and no queue exists*” is a nameable, checkable state rather than a hang.

⇒ **Net effect on the Windows question:** §1 no longer argues that the premise fails. It argues that GSP is requested on Turing+ regardless of OS, that a **fallback** path exists and is enabled by default, and that whether NVIDIA's Windows driver takes it is **[UNVERIFIED]** and gated by a struct we cannot see. □ **Sections 2-4 below are unaffected** — they are about WDDM's ownership of the page tables, TDR, and the absence of UVM, none of which depends on how GSP is enabled.

2. Under WDDM the OS owns the page tables — and we never see the writes

Microsoft, verbatim: “*VidMm manages the GPU virtual address space of all processes. VidMm is also responsible for allocating, growing, updating, ensuring residency, and freeing page tables.*” NVIDIA's side matches: VA spaces are `SHARED_MANAGEMENT` (“*management of the VA space is shared with another component (e.g. driver layer, OS...)*”, `nvos.h:1384`), and ranges are `NVOS32_ALLOC_FLAGS_EXTERNALLY_MANAGED` — “*Page tables for this allocation will be managed outside of RM.*”

○○○ **And the transport for individual PDEs is invisible to us.** `NV0080_CTRL_CMD_DMA_UPDATE_PDE_2` (0x80180f) is documented “**only available on Windows and MODS platforms**” and **formats the PDE into the caller's own buffer** rather than RM's directory. Critically, it is **handled purely in guest CPU-RM and is NEVER RPC'd to GSP for a GSP client** (`dma.c:320-324` RPCs only under `IS_VIRTUAL_WITHOUT_SRIOV`; `:340-355` does the work locally). Same for `NV90F1_CTRL_CMD_VASPACE_RESERVE_ENTRIES` (`gpu_vaspace.c:4109`).

⇒ **The address model in Part 1 §4 assumes RM authorship of page tables. On Windows that assumption is false**, and our fake GSP hears nothing.

□ **One thing survives:** `SET_PAGE_DIRECTORY` is RPC'd to GSP when `IS_GSP_CLIENT` (`dma.c:507-521`). □ But with different semantics — aperture typically `_VIDMEM` not `_SYSTEM_COH`, arriving per device creation rather than per UVM VA space.

□□□ **And the silver lining, which is real:** unlike Linux, Windows *tells the driver every PTE it wants written, explicitly and in order*, through `DxgkDdiBuildPagingBuffer` ops (`UpdatePageTable`, `FlushTlb`, `MapApertureSegment`) submitted as **GPU-executed paging buffers**. ⇒ If we can see paging-buffer submissions, that is a **richer** signal than the doorbell-time page-table sweep the C artifact used. ∅ **[UNVERIFIED]** whether NVIDIA's WDDM path routes those through CE (visible to us), BAR1 CPU writes, or `UPDATE_PDE_2` (invisible). **This is the single most important open question on this axis.**

3. TDR — a hard, OS-enforced 2-second deadline

	Linux	Windows
graphics timeout	4 s	□ 2 s , OS-enforced
compute timeout	30 s	2 s (5 s <code>TdrDdiDelay</code>)
consequence of exceeding	the driver soldiers on with stale TLBs	adapter reset ; <code>VIDEO_TDR_FAILURE</code> (0x116)
repeat	—	∅ five hangs in one minute ⇒ bugcheck 0x117

RM's own comment concedes it: *"90 % of the overall hard limit of 2.0 seconds, imposed by WDDM driver rules"* (`os.c:2116`).

⇒ ∅ **On Linux a slow emulator is merely slow. On Windows a completion path slower than 2 s is diagnosed as a hung GPU.** □ Cold boot is probably safe — device init gets a 60 s exception (`thread_state.c:331`) — but **runtime is not**, and our data plane is currently a CPU copy loop.

□ Related and worse: Windows has a **first-class page-fault interrupt** carrying `{FaultedVirtualAddress, PageTableLevel, FaultErrorCode, FaultedProcessHandle}` and the OS *expects* an accurate fault path, using it to kill the offending context rather than the adapter. ∅ The C artifact went green *"without servicing or forwarding a single GPU fault."* That is a **Linux-only luxury**.

4. Four more that matter, briefly

- ∅ **No UVM on Windows.** *"Currently, we don't support UVM on Windows/MODS"* (`pool_alloc.c:488`); in WDDM mode CUDA contexts are `dxgkrnl` contexts (`D3DKMT_CLIENTHINT_CUDA`). ⇒ The `UVM_*` ioctl stream **disappears** — and `UVM_MAP_EXTERNAL_ALLOCATION` is the lockstep boundary of our `nvdiff` oracle (221 of `cuCtxCreate`'s 479 ioctls).
- ∅ **The OS manages the framebuffer** on non-TCC Windows — RM verbatim: *"Disabling in Non-TCC windows because the OS manages FB"* = **no scrub-on-free**, and RM stops choosing PTE kinds (`bRmToChooseKind = NV_FALSE`). ⇒ The scrub path in Part 5 is one a Windows guest largely does not take.
- ∅ **System GPU mappings become 4 KB-granular** where Linux uses 64 KB/2 MB — a 16-512× PTE-count increase. □ And Windows **guarantees** VA/PA 64 KB congruence and lets the driver *rely* on it, so a violating emulator faults with no obvious cause — the same class as this tree's existing congruence trap.
- □ **DEV_xxxx must already be in NVIDIA's signed INF.** PnP binds by hardware id from a catalog-signed package, and *"even a single-byte change... invalidates the digital signature."* ⇒ **A gate before any of the above matters.**

□ **And one that lands directly on w821's Hopper work:** RM allocates the BAR1 doorbell region **downwards from the top of BAR1 on Windows** — *"WAR for Bug 3564398, need to allocate doorbell for windows differently"* ⇒ `BUS_MAP_FB_FLAGS_MAP_DOWNWARDS` for `MEMDESC_FLAGS_MAP_SYSCOH_OVER_BAR1` (`mapping_cpu.c:506`), **the exact flag Hopper+ sets on the usermode/doorbell region** (`kernel_fifo_gh100.c:123`). ⇒ Our BAR1 doorbell-page identification (Part 1 §2.1) must not assume Linux allocation order.

5. □□□ The escape hatch — TCC, and it collapses most of the table

TCC mode is not WDDM at all — it *"uses the Windows WDM driver model"* and disables graphics (`nvidia-smi -dm 1`). □ And nearly every Windows branch found in `ogkm` is gated on `!tcc`: FBSR-WDDM mode, PMA client page tables, P2P mailbox, scrub-on-free. `nv_gpu_ops.c:2327` states it directly: ***"Non-TCC mode on Windows implies WDDM mode."***

⇒ In TCC: scrub-on-free returns, PMA-managed client page tables return, `EXTERNAL_HEAP_CONTROL` is off, FBSR-WDDM is off, and `dxgkrnl/VidMm/VidSch` are entirely out of the loop. §§2, 4 largely evaporate. □ TCC is not available on GeForce parts. (MCDM is the middle option — render-only, no display, and it **requires an MMU**.)

[PROPOSE] If a Windows guest ever becomes a target, **TCC is dramatically the smallest one**, and the order is: TCC → MCDM → WDDM.

6. □ What is the SAME — this bounds the work, and it is a lot

1. **The RM API ABI is identical** — NV0521/54/32/33/38, handle semantics, class ids, control numbers. No Windows variants in `common/sdk/nvidia/inc/`.
 2. `NV01_EVENT_WIN32_EVENT` **is** `NV01_EVENT_OS_EVENT` — literally a `#define`, value `0x79`.
 3. □ **The GSP boot sequence is OS-agnostic** — **zero** Windows conditionals in the bootstrap path; FWSEC/WPR2/booter/LibOS-args/msgq/GSP_INIT_DONE all shared. The only platform conditional makes Windows hold the API lock across *all* of GSP init ⇒ **strictly more serial, the easier case for us**.
 4. `GspSystemInfo` **layout is identical**; 610 vs 580 differ by four fields — a version difference, not an OS one.
 5. `SET_REGISTRY` **format is the same**; `nvrml_registry.h:24` says outright “*shared between Windows and Unix*.”
 6. □□□ **The doorbell / work-submit path has NO Windows conditional in RM** — usermode region, `workSubmitToken`, `kfifoUpdateUsermodeDoorbell_*`, `USERD`, `GPFIFO`: **zero** Windows branches across `gpu/fifo/`. ⇒ **Part 1 §2 is OS-independent**, modulo §7’s two unknowns.
 7. **GMMU formats, PTE/PDE encodings, apertures, big-page selection** — chip-keyed, not OS-keyed.
 8. **Hypervisor detection is shared and CPUID-based**, and **∅ there is no error-43-style refusal anywhere in the open RM core**.
 9. **Robust channels / RC are enabled on Windows, GSP and Unix alike**.
 10. **PMA is supported on all platforms** — WDDM disables *client page tables under PMA*, not PMA.
-

7. □□□ A free gift — detecting a Windows guest from the FIRST RPC

`GSP_SET_SYSTEM_INFO` carries `NvBool bGspNocatEnabled`, and the **only** assignment in the entire tree is:

```
if (RMCFG_FEATURE_PLATFORM_WINDOWS) { rpcInfo->bGspNocatEnabled = NV_TRUE; }
```

`kernel_gsp.c:4624` (610) / `rpc.c:10650` (580) — present and identical in both generations.

⇒ `bGspNocatEnabled == NV_TRUE` **in the first RPC we ever receive** ⇒ **the guest RM was built for Windows**. □ It is an *enable* flag, not an OS field — **assert on it, do not build on it**. There is no explicit OS identifier in `GspSystemInfo`.

□ [PROPOSE] **Implement this now, as a refusal, before any Windows work happens**. It costs one comparison and converts “*a Windows guest silently does something strange*” into a named refusal at the first message — which is exactly what `support_matrix_asymmetry` demands of an out-of-band configuration.

8. □ A correction to this project’s own framing

`CLAUDE.md` describes `nvkvm` as “*WSL2-style NVIDIA GPU ioctl/RPC forwarding*.” ∅ Publicly, WSL2’s mechanism is **D3DKMT-shaped thunks over VM Bus to a host `dxgkrnl`** — the Windows Graphics Kernel lead, XDC 2020: “*Level of abstraction is the WDDM interface... Not a straight pass-through... No data copy — only control information exchanged over VM bus*.” ⇒ **Mode 1 resembles WSL2’s shape, not its protocol**. The phrase should be corrected where it appears, because it invites the assumption that a documented design exists for what we do.

□ Related, and worth knowing: Microsoft’s own answer to our problem statement is **not to run the vendor KMD in the guest at all** — GPU-PV puts “*no KMD in the guest, only UMD... no video memory manager (VidMm) or scheduler (VidSch) in the guest*.” ⇒ **The industry cut line is at D3DKMT, not at the hardware**. We are deliberately below it, and that is the thing that makes this project unusual — and the reason no prior art exists for emulating an NVIDIA GPU to a Windows guest.

9. What could NOT be determined, and the first measurements to take

question	status
The Windows OS layer itself (osGetPageSize, osQueueDpc, osIsRaisedIRQL, ...)	∅ arch/nvalloc/win/ is not published. The 4 KB page-size finding rests on an inferred osGetPageSize()==4096
The Windows rmconfig profile — which chips, classes, engines and HAL bindings	∅ openrm hard-sets RMCFG_FEATURE_PLATFORM_WINDOWS 0; every NVOC table here is the UNIX profile
Is the GSP firmware image byte-identical between Windows and Linux packages?	∅ Nothing speaks to it. If it differs, our WPR2/radix3 sizing constants do too
Is the GSP-RM RPC protocol the same on Windows?	● inference only (one RM codebase). ∅ Do not assume our msgq work transfers without measuring
What do dxgkrnl/VidMm actually issue, and in what order — CE, BAR1 CPU writes, or UPDATE_PDE_2?	∅ The most important open question on this axis (\$2)
Is display mandatory for nvlddmkm to load?	∅ Cosmetic on Linux, plausibly fatal on WDDM. TCC/MCDM sidestep it
Does NVIDIA enable WDDM 3.2 user-mode doorbells ?	∅ If so, the guest rings a doorbell from user mode with no kernel transition — a trap-per-doorbell design becomes a per-write exit on a user-mode hot path
Was the Code-43 CUID check removed at R465, or gated?	∅ NVIDIA said a narrow configuration is <i>supported</i> ; it never said the check was removed

□ **[PROPOSE]** The first two measurements, in order: 1. D3DKMT_QUERY_GPUMMU_CAPS on real hardware — VirtualAddressBitCount, PageTableLevelCount, 4 K vs 64 K segments, GpuMmu vs IoMmu. Cheap, and it decides \$2 and the page-size finding. 2. **Check our emulated DEV_XXXX is already in NVIDIA's signed INF.** If it is not, nothing else on this axis is reachable with a stock guest.

Part 4 — The machine, as it is

STATUS: LIVE, 2026-09-20 (w816). Companion to `THE_ARCHITECTURE_v2.md` (the proposal) and `THE_SURFACE_v2.md` (the per-item inventory). **This** document is the one an engineer should attack: it describes the system as it actually runs, from a `file:line` survey, and marks every place I propose to change it.

0. How to read this

Three markers, used consistently:

marker	meaning
[MEASURED]	a number from a real boot, with the run that produced it
[CODE]	a fact read out of the tree at <code>w749-fable-legb</code> , with <code>file:line</code>
[PROPOSE]	my design opinion. Argue with these.
□ [UNVERIFIED]	the survey could not establish it. Not filled in with a guess.

∅ Two contradictions found while writing this are left **in**, unresolved, because only one side of each can be true and I could not determine which:

1. **The BQL.** `shim.rs` and `docs/design/blocking_and_completion_model.md:16` both state “*every guest MMIO write arrives with the QEMU BQL held.*” But `qemu/hw/misc/nvkm/nvkm.c:1118` calls `memory_region_enable_lockless_io` on **every** region, the realize-time self-check *refuses the device* if the flag is missing (`nvkm.c:1568`), and `nvkm.c:392` states as measured fact: “*these callbacks run CONCURRENTLY on every vCPU with NO BQL.*” □ If the lockless path is live, every “the whole VM is frozen during a trap” argument in this tree is stale, and the shared-lock findings in §4 get much sharper.
2. **Vas is keyed by** (`GpuId`, `ResourceKey`) [`CODE gpu.rs:1761`], but at least three doc comments still say (`GpuId`, `Pdb`) (`gpu.rs:1760`, `:250`, `:275`). □ I did not determine whether any reachability argument still depends on the stale key.

□ And a third, which I fixed while writing: `ScratchpadArm:0ff` is documented “*The default*” while `scratchpad_from(None)` returns `Measure`. That stale line made one of my own surveys report that the single store is off by default — i.e. that the central memory design does not run. It is the **third** stale-default of the day, after `IsolatePlane::Stillborn` and `KAYFABE_VAS_OWNER=isolate`. ## 1. Vocabulary — what the nouns actually are

∅ Written for a reader who does not know NVIDIA’s constant names. Every term below appears in the rest of the document and in the code; this is the only place they are defined.

GPU, host, guest. The *host* is the machine with the real card. The *guest* is a VM running a **stock, unmodified** NVIDIA driver. The guest believes it owns a GPU. It does not — it owns our emulation of one, and real work is carried out on the host’s card by unprivileged host processes we control.

RM (Resource Manager). The core of NVIDIA’s driver. Everything the driver does is “allocate an object of class X under parent Y”, “call control command Z on object W”, “free it”. Objects form a tree rooted at a *client*. This object protocol is the control plane, and it is what we emulate.

Client / handle / object. A *client* is a driver-level session (roughly: one open of `/dev/nvdiactl`). A *handle* is a 32-bit number the guest picks to name an object **within** its client. Handles are per-client, reusable after free, and therefore not identities — two different objects can wear the same number at different times. This is why the code keeps “incarnations” rather than trusting handle values.

GSP (GPU System Processor). On modern cards, a RISC-V core **on the GPU** runs a firmware that does most of RM’s privileged work. The host driver stops touching hardware directly and instead sends the GSP messages over a queue in memory. □ **We impersonate the GSP.** That is the central trick: we do not emulate a GPU register-by-register, we answer the driver’s *messages* the way its firmware would.

RPC. One of those messages. Each has a numeric function id (e.g. `0x67` = “allocate an object”). The guest writes a message into a ring buffer in its own memory and pokes a register; we read it, act, and write a reply back into the ring.

Control command. A sub-kind of RPC: “run command `0x20800102` against object H, here are the parameters.” These are the bulk of the driver’s vocabulary — hundreds of them, each with its own parameter struct. Most are *queries* (“how much framebuffer is there?”, “what are this copy engine’s capabilities?”).

BAR0 / BAR1 / BAR2. PCI memory windows the card exposes to the CPU. - **BAR0** is the register aperture — 16 MiB of control registers. A guest read or write here traps to us. - **BAR1** is a window onto video memory, so the CPU can read/write VRAM. - **BAR2** is a second window, used for *instance memory* (the structures describing channels).

Framebuffer / VRAM / vidmem. The card’s own memory. A *framebuffer offset* is a byte offset into it. □ In our design guest vidmem is **one** big host allocation, and a guest framebuffer offset is simply an offset into that allocation — the identity is the entire memory model.

Sysmem. Ordinary system RAM the GPU can also read, via an IOMMU-ish mapping. For us this is the guest’s RAM, which the hypervisor hands us as a shareable file descriptor (`memfd`).

GPU virtual address (VA) and the page tables. The GPU has its own MMU. Work submitted to it uses *virtual* addresses, translated by page tables the driver builds in memory. A **PDB** (page directory base) is the physical address of the root of one such tree — i.e. “which address space”. The GPU’s TLB caches translations, so after changing tables the driver issues a **TLB invalidate**.

VA space. One page-table tree = one address space. A guest process typically has one; the kernel driver has others. Two channels sharing a VA space see identical translations.

Channel / GPFIFO. A *channel* is a submission queue to an engine. The driver writes command packets into a **pushbuffer** in memory, then appends an entry to a ring called the **GPFIFO** pointing at that pushbuffer. Hardware consumes the ring.

USERD. A small per-channel structure holding the ring cursors: `GP_PUT` (how far the driver has filled) and `GP_GET` (how far hardware has consumed). The driver writes `GP_PUT`; hardware writes `GP_GET`.

Doorbell. After updating `GP_PUT`, the driver writes one 32-bit word to a register to say “there is new work on channel N”. That write is a *doorbell*. It is the hottest trap in the system — and on a real card it is a posted write that costs the driver almost nothing.

Engines: CE and GR. - **CE (Copy Engine)** — a DMA engine. It moves bytes between addresses. `LAUNCH_DMA` is its “go” command; operands (source, destination, length) are loaded into it by preceding commands. - **GR (Graphics/Compute)** — runs shaders/kernels. CUDA work lands here.

Semaphore release. How software learns an engine finished. The driver puts a “when done, write value V to address A” command in the pushbuffer, then **spins** reading address A until it sees V. ∅ This matters enormously for us: the wait is the *guest’s own* spin on *its own* memory, so if the value appears the guest proceeds, and if it never appears the guest hangs. We must never forge V for work that genuinely had to happen.

Scrub. Before RM hands a page of video memory to a different client, it clears it — with a copy-engine fill, on a kernel-owned channel. This is what stops one process reading another’s old data. □ The guest does this itself and orders it correctly; our only job is not to lie about whether it happened.

WPR2 (Write-Protected Region 2). A region of VRAM the GSP firmware is loaded into, fenced off by hardware. The driver checks registers to see whether it is already up. It only resets on a full device reset, which is why each clean run needs a fresh VM boot.

CeUtils. RM’s internal helper that owns the kernel-side copy-engine channels — the ones used for scrubbing and other housekeeping. Its channels belong to no guest process.

UVM (Unified Virtual Memory). A second guest driver module (`nvidia-vm`) providing managed memory. It has its own ioctl surface and its own channels, and CUDA uses it heavily.

Xid. An NVIDIA error code reported by the driver when hardware faults. `xid 31` is a GPU memory-access fault — the engine touched an address the page tables did not map. ∅ For us an Xid on the host is a signal we got a mapping wrong.

2. Processes and threads

2.1 The processes

A full Mode-2 boot is **one QEMU process** with our Rust archive statically linked in as a QOM device, plus **N+1 sandboxed child processes**.

process	count / lifetime	opens <code>/dev/nvidia*?</code>	what it is for
VMM (QEMU)	1 per VM	No [CODE]	owns the guest, KVM fd, all guest RAM, the MMIO traps, the pure core, the doorbell table, the BAR mirrors, and the isolate factory

process	count / lifetime	opens /dev/nvidia*?	what it is for
Scratchpad isolate	1 per (VM, GPU), spawned at PCI realize	Yes	holds the ONE reserved video-memory object, the walk kernel and (when armed) a CUDA context that guest process's RM client, its channels, its VA spaces
Per-proc isolate	1 per (guest process, GPU), lazy	Yes	

[CODE] Every `/dev/nvidiactl` open in the workspace is in `kayfabe-isolate-host` (the child side) or the standalone ladder tool. **The VMM issues no RM ioctl.** It holds descriptors the children hand up — sealed memfds and armed device nodes — which it only `mmap`s, never `ioctl`s (`export.rs:31-42`).

□ **The “birth client B” is not a process.** It is an RM client handle plus two character-device descriptors, minted **inside a per-proc isolate** (because RM stamps `pClient->ProcID` from the creating task) and passed to the scratchpad by `SCM_RIGHTS` [CODE `child.rs:722-740`].

[PROPOSE] Isolates stay. Their founding reason is **VA identity** — NVIDIA, and UVM especially, treats host VA and guest VA as one address, so each guest process needs its own host address space. Forgetting this is what let bug #14 (identical-VA collision between two guest processes) back into the C. But under the single store they stop being *memory owners* and become **address-space containers**: `Publish`, `PublishVidmem`, `JoinFbLeaf`, `AliasFbLeaf`, `FbJoinPeek` and `ExportBacking` all delete, leaving `MapStoreSlice`, `AdoptVaSpace`, `channel birth`, `doorbell` and `control`. □ **[UNVERIFIED]** Route K already births channels in the *scratchpad's* process — if that holds for UVM's channels too, VA identity has already moved and this conclusion changes.

2.2 The sandbox — two stages, forced order

Stage A, taken by the parent at `clone` [CODE `spawn_unsafe.rs:1009`]: `CLONE_NEWUSER|NEWPID|NEWNET|NEWIPC|NEWUTS|NEWNS`, `rAw SYS_` `clone`, then rootless uid/gid maps (`setgroups=deny` first, as the kernel requires). `execveat(AT_EMPTY_PATH)` on a **sealed memfd**, so the image cannot be substituted. `PR_SET_NO_NEW_PRIVS` before `exec`.

Stage B, taken by the child in its own `main`, before any worker thread exists [CODE `sandbox_unsafe.rs:770-891`] — and **the order is the security property**, which is why it is one call and not three: nested user `ns` → `MS_REC|MS_PRIVATE` / → 256 KiB `tmpfs root` (`nosuid,noexec`) → bind the two device nodes → `pivot_root(".", ".")` + `umount2(MNT_DETACH)` → **only now** open the `/dev` `dirfd` → remount `root` read-only → drop every capability and **read the result back**, a single surviving bit being a refusal.

□ **Three things any security claim must not overstate:** 1. **seccomp is NOT installed** [CODE `lib.rs:57`]. The fixed fd numbers exist *so that* a future filter can hardcode them; that filter does not exist. Any document saying the isolate is syscall-filtered is wrong. 2. **The CUDA scratchpad has a full filesystem view** for the duration of its CUDA bring-up, which runs *before* `sandbox::enter` [CODE `child.rs:355-374`]. Constraint 20 must be read as “*it ends with the same reach*”, not “*it never had more*”. 3. That bring-up makes the child **multi-threaded** before `sandbox::enter`, and `unshare(CLONE_NEWUSER)` is refused to a multi-threaded process — so the scratchpad depends on the fallback arm under inherited `CAP_SYS_ADMIN`.

□ [CODE `isolate.rs:20-32`] `fd 4` is **deliberately vacant**. It used to carry an `o_PATH /dev` `dirfd`, and `openat(4, "../etc/shadow")` **opened** — `o_PATH` places no restriction on ...

2.3 The threads

Exactly two production threads are created by `kayfabe` inside the VMM [CODE]:

thread	loop	waits on
<code>kayfabe-doorbell-publish</code>	<code>doorbell_publish_loop</code>	Condvar on the publication queue
<code>kayfabe-completion-observer</code>	<code>observer_loop</code>	<code>epoll_wait</code> , 250 ms tick, woken early by an <code>eventfd</code>

Plus QEMU's own `main` and `vCPU` threads, on which our trap code runs.

Inside each isolate child: a `main` thread, one **control** thread blocked in `recv(2)` on a datagram socket, and `--workers` (default **4**) pool threads each blocked reading its own socket. □ **None of the child's threads are named**, so `/proc/<pid>/task/*/comm` shows `kayfabe-isolate` for all of them.

□ The control thread is separate for one reason: a cancel must be serviced **while every worker is busy**, which is the only state in which a cancel is ever wanted. It signals with `tgkill(SIGUSR1)` whose handler is installed with **no SA_RESTART** — “the single most consequential line in the file” [CODE `signal_unsafe.rs:63`] — because that is what makes an in-flight RM `ioctl` return `EINTR`.

□ **[UNVERIFIED]** `observer_loop` declares no `ThreadClass`, so it defaults to `Coordinator`, under which long blocking sections are a reported violation — yet it deliberately performs millisecond-scale budgeted drains. The publication worker *does* declare `Worker`. There is no comment either way.

3. The vCPU trap path, step by step

3.1 How a guest access reaches Rust

```
KVM vmexit
→ nvkvm_trap_write()          qemu/hw/misc/nvkvm/nvkvm.c:591  (MemoryRegionOps.write)
→ kayfabe_shim_regs_write()    shim_unsafe.rs:1371      (extern "C")
    mark_vcpu_thread()          :1384                thread-local
    TrapGuard::enter_at(bar<56|off) :1453            thread-local depth
→ Regs::write()                shim.rs:18576
→ RegPlane::write()             plane.rs:6473
```

Reads are shorter: `nvkvm_trap_read` → `kayfabe_shim_regs_read` → `Regs::read` → `RegPlane::read`.

∅ BAR0 is **cut into pieces**: 4 KiB pages whose every dword is “dead” get a RAM-backed alias so the guest’s reads of them never exit at all; the rest stay live trap pieces.

3.2 A DOORBELL write (BAR0 0x00BB0090) — the hot path

Defaults in force: `doorbell_async = 0n` (defers), `doorbell_inline = Off`.

1. `RegPlane::write` — two atomics (writes, `mmio_in_flight AcqRel`), then the doorbell arm matches and **returns before the rank-0 plane mutex is ever reached** [CODE `plane.rs:6525`].
2. `ring_doorbell` — bumps a counter; takes `mmu_inval.inner` briefly; takes `plane.doorbell (read)` across the call.
3. `SharedDoorbell::ring` — the passthrough inline short-circuit is off by default; `shadow_route` is pure arch math + one Relaxed load on an atomic slot array. **No lock**.
4. `pubqueue.offer(MapPublication::for_doorbell(token))` — takes `pubqueue.inner`, coalesce check, cap check, `VecDeque::push_back`, **drops the guard**, then `notify_one`.
5. Back out: `dbcost` (atomics), `dbledger.seen` (a mutex, ≤64 tokens), and the report is Scheduled — SO `announce_completion` is skipped and `state/cpu_intr` are **not** taken.
6. The tail of `Regs::write` runs on every write: a few Relaxed epoch swaps, each of which may emit one further offer.

Net on the vCPU: ~6 atomics, **4 short lock acquisitions**, one `VecDeque` push, one `notify_one`. No rank-0 plane lock, no device lock, no IPC, no blocking.

Deferred to the worker: everything — mirror revalidation, isolate spawn, ring adoption, the GPFIFO read, the host forward or CE execution, the retired-object drain.

3.3 An MMU_INVALIDATE write (BAR0 0x00B830B0) — the one the guest spin-polls

1. `RegPlane::write` → the invalidate arm. Writes to the two PDB latches are pure latching.
2. On the **trigger**: `note_trigger` takes `mmu_inval.inner`, decodes, then `pending.swap(true, AcqRel)` — *this is the atomic the guest polls* — and bumps `issued`.
3. `shadow_write` pushes the new trigger value into the BAR0 backing page: takes `plane.read_shadow (read)` then `ShadowSink::segments`, then a `write_volatile`.
4. `Regs::write` offers `MapPublication::for_invalidate` onto the same queue.
5. **The vCPU returns. It never blocks.**

Where the guest blocks: in its own loop. `ogkm` spin-polls the trigger until it reads false, with a **4 s** (graphics) / 30 s (compute) timeout. If the containing page is RAM-backed the poll costs **no vmexit at all**; if not, each poll is a read trap served by a deliberately lock-free arm. □ **[UNVERIFIED]** whether page `0xB83000` is backed for the shipped profile — the predicate is per-dword over the whole page and the survey did not execute it.

What clears TRIGGER: the worker, at the end of its pass — `complete_through_unmaps(seq, ...)`, which **withholds** completion if a newer trigger arrived or if unmaps are outstanding (Constraint 27: completing early would let the guest reuse a page still reachable through a slice we have not torn down), then `pending.swap(false)` and a `write_volatile` of 0 into the shadow.

□ [CODE shim.rs:18726] the completion is **no longer a Drop guard**. In the code’s own words: *“complete() is now the ONLY thing that clears the trigger, and a worker that dies without calling it hangs the guest by construction.”*

3.4 Back-pressure — the rule that makes deferral safe

The queue never drops work silently. A refused offer converts a **notification** into a conservative **full sweep**: a dropped doorbell arms an emulated-channel sweep; a dropped invalidate arms a full rescan; a dropped RPC bind force-releases held replies. Owner’s rule, quoted in-source: *“a lost NOTIFICATION becomes a conservative FULL RESCAN ... losing one may only ever cost time, never coverage.”*

□ **Coalescing is OFF for doorbells, and that is a measured verdict.** [MEASURED] on real GA106 the coalescing arm produced `cuCtxCreate` → unknown error (999) and two host Xid 31S (CE2 FAULT_PDE, GR0_PBDMA0 FAULT_PTE); the `nocoalesce` arm was clean. *“Deferring is fine and coalescing is not.”*

4. Locks, and the blocking rules

4.1 The ranked order

[CODE lock.rs:56] *“The total, one-way lock order ... Every lock declares its rank at construction; a thread may acquire only in strictly increasing rank, at most one lock per rank.”*

rank	lock	protects
0	<code>RegPlane::state</code>	the GSP FSM, guest-RAM port, command policy
1	<code>RegPlane::mem</code>	the emulated framebuffer store + GMMU format
2	<code>SharedDevice::state</code> (RwLock)	the spine, routing maps, delivery pump
3	per-Proc mutex	one guest process’s channels/VA spaces
4	leaf (<code>cpu_intr</code> , executor inbox, recorder)	

Enforcement is real: `check_acquire` **panics before the OS acquire**, always on, not a `debug_assert`.

□ Why the plane sorts *below* the device despite being further out: the shipping order is plane→core and ranks must strictly increase, so the plane must sort first *“or every vCPU MMIO trap in the device would panic.”* The ABBA this fixed was **guest-buildable** — *“A guest could build the deadlock by ringing a doorbell on one vCPU while touching a register on another.”*

4.2 The blocking rules — what is actually enforced

rule	mechanism	teeth
R1 no blocking under a ranked lock	<code>assert_lock_free</code>	runtime panic, always on
R0 no blocking on a vCPU	<code>assert_not_on_vcpu</code>	□ census by default — panics only under <code>KAYFABE_VCPU_BLOCK_FATAL</code>
no publishing from a trap	<code>OffVcpu</code> token, taken by value	□ compile-time ; minted in exactly one place
INLINE-SAFE(site) predicate	prose only	∅ (a) and (b) have no mechanism anywhere

□ The `OffVcpu` doc is the clearest statement of the principle in the tree: *“A constraint that lives only in prose is re-litigated by every new path, and the reviewer who has to notice it is the weakest link in the system. **The type is the reviewer.**”*

□ **The read path does not mark the thread as a vCPU.** `kayfabe_shim_regs_read` installs only the `TrapGuard`; a thread that has never done an MMIO write is not `is_vcpu()`. A second, independent detector does catch it — *“the trap witness is measured, a declaration is a claim, and where they disagree the measurement wins.”*

4.3 ☐ The gate that is RED right now

tests/tests/unranked_locks.rs enumerates every **unranked** lock a vCPU can hold, because `assert_lock_free` masks only *ranked* locks and therefore **cannot see these at all**. Its own header is honest about what it buys: “*ENUMERATED, NOT ENFORCED ... Nothing fires when one is taken in a bad order.*”

[MEASURED 2026-09-20] it FAILS with **21 undeclared** unranked vCPU-path locks. Two matter most:

- `dbtable.rs Mutex<BTreeMap<u64, [u64; 4]>>` — **the doorbell ledger, taken on every doorbell**. ☐ Mine, added w801.
- `shim_unsafe.rs Mutex<Vec<ShadowSegment>>` — taken by the **vCPU** in `shadow_write` *and* by the **worker** in `publish_invalidate_trigger`. That is a lock genuinely shared between a trap and a worker, which is **precisely clause (c)** of the inline-safe predicate.

□ And if the BQL contradiction in §0 resolves toward *lockless*, two vCPUs can be in these paths simultaneously, which makes the shared-lock case sharper still.

[PROPOSE] Classify all 21 with real rulings — not to turn the gate green, but because the answer to “*can anything block beneath this?*” is a fact the v2 design needs anyway.

4.4 Where we block on purpose

[CODE `isolate.rs:454`], stated at the site: “*this is a synchronous write-then-blocking-read over a unix socket to the isolate child, and every production caller is inside the vCPU’s MMIO trap under the QEMU BQL ⇒ whatever the child spends here, the guest pays.*”

⇒ So the slogan “no blocking work in any MMIO trap” is **not** what the code does. There is one sanctioned blocking site and it is the isolate round trip. The store-map path explicitly refuses to be one of these, declining by name on a vCPU rather than stalling.

[MEASURED w470] the worst recorded stall was an isolate **spawn** on a vCPU — `Regs::write → materialize_pending → spawn_host → read_frame → __recv`, **1.62 s** inside an MMIO exit. That is why the scratchpad is brought up at realize, before the guest runs.

[PROPOSE] v2 should make this structural rather than sanctioned: no RM verb on a vCPU at all. Every such call becomes an offer onto the queue, and the guest waits on its own primitive — which §1 of the architecture says it already tolerates.

5. Data structures

5.1 The object graph — identity vs label

The whole file is built on one distinction, and it is the right one: **a guest handle is a label, not an identity**. RM recycles handle values with no quarantine and no free list, so the same 32-bit number names different objects over time.

type	what it is
<code>NodeKey { client, handle }</code> <code>ResId(u64)</code> <code>ResourceKey { origin, incarnation }</code>	a handle — per-client namespace slot. A label. stable identity, minted at the origin alloc, never reused the reportable identity; ordered origin-first so maps iterate deterministically
<code>HandleRef::{Origin, Alias}</code>	<i>how</i> a live handle came to reference its resource — recorded, never re-derived

```
struct Resource {  
    node: RmNode, refs: BTreeSet<NodeKey>, pdb: Option<Pdb>,  
    pdb_aperture: Option<Aperture>, gpu: Option<GpuId>, map_refs: usize,  
    owner_kind: ClientKind, owner: ClientId, owner_incarnation: u32,  
}
```

Liveness is `refs non-empty or map_refs > 0`. `next_incarnation` is “*the smallest ordinal not held by a live resource at this handle value*”, computed as a **range query**, not a scan — explicitly to deny a complexity DoS.

☐ `RmGraph::apply` **clones the whole graph** for rollback, because three faults are raised after mutation. The comment records it: ~24 % of an O(live objects)-per-event control plane. **[PROPOSE]** this is a named, deferred cost and v2 should fix it by validating before mutating.

5.2 The runtime model

```
Gpu { spine: Spine, system: Proc, procs: BTreeMap<ProcId, Proc> }
```

□ The concurrency contract is in the type, not a comment: a **per-proc** op takes the device *read* lock plus that proc’s mutex; a **spine** op takes the device *write* lock plus exclusive access to every proc, expressed as a `ProcSet` argument.

Proc is not a hardware concept. `ProcId` is “purely the label of one dup-connected component of declared user clients in the RM graph.” The stable name is `ProcAnchor` — the smallest client *declaration* in the component.

□□□ The distinction that earned its keep: `clients (ClientKey)` **labels** a component; `client_ids (ClientId)` **matches** one. Matching on handle values alone let a re-declared namespace **adopt the previous tenant’s proc** — its isolate, arena, host VAS and pending-release queue.

`Vas` carries the address plane; `Channel` carries the exec plane. `Channel::method_state` is the one guest-driven state and is bounded **structurally** (a fixed array), not by a check.

5.3 The address table — what v2 deletes

```
pub struct AddressTable { map: IntervalMap<Row>, owner: Option<Pdb>, generation: u64, blockage }
```

```
pub struct Binding { kind: RegionKind, phys: u64, aperture: Aperture, host: Option<HostBacking> }
```

All four `Binding` fields are private with exactly two constructors — `declared_by_guest` (no host object) and `real_gpu_memory` (the only kind carrying one). Rows are **ranges**, not pages.

`resolve(pdb, va)` checks identity **first** (owns), then a predecessor lookup; a miss is a **fault**, never a heuristic. `bind` checks identity, then host-VA identity, then extent agreement, *before* anything enters the map.

[PROPOSE] This whole structure goes. Under v2 the walk produces a diff, the diff is `mmap/munmap`, and the **host page tables are the model**. Every field above is already in hand at the moment you act: the target is identity (`store offset == FB offset`), `system` comes from a static layout, permissions come from the PTE just walked, and “it went away” is `munmap`.

□ The machinery is **already built and unwired**: `kayfabe_mmu::walkdiff::diff` exists fully implemented with **zero production callers** (its only caller is a `#[test]`), `MapOp::Map, Unmap, Remap` exists, and `StoreMapPort::apply_ops` takes a **list** and does unmaps-before-maps — while its one production caller always passes a single-element `Map`. The source even says “*when the delta lands, this call site does not change.*”

○ **Consequence today:** the only production unmap is a same-VA re-point. `release_vas` has zero callers. So `StoreMapPort::outstanding` grows **monotonically over the VM’s life** — mappings only ever accumulate, which is the other half of the cross-client leak.

5.4 Crate layering

`util` → `arch` → `abi` → {`mmu`, `core`, `fwd`, `rmrpc`, `device`, `rt`} → `qemu-raw`, with `vmm` and `isolate` as ports.

□ One layering fact worth arguing about: **kayfabe-mm**u depends on **kayfabe-isolate**, because `Binding` carries a `HostHandle`. So the address plane sits *above* the sandbox port rather than beside it. **[PROPOSE]** deleting the address table inverts that and makes `kayfabe-mm`u a leaf crate — which is a good structural signal that the deletion is the right shape.

5.5 The isolate boundary — two layers, not one

□ A correction to a premise I held: `VerbPlan/VerbReply` are the **in-process** typed plan and reply. The **IPC** is `Request/Reply/Envelope`.

- **Transport:** one `SOCK_STREAM` socketpair **per pool worker**, granted at fixed fd numbers.
- **Framing:** `u32 LE` length then body, in **one** `write_all` — because a length written separately from its body permanently desynchronises a signal-interrupted channel.
- **No multiplexing, no demux, no txn correlation.** Each worker owns its socket and the channel is 1-deep. An explicit divergence from the C, which multiplexed 16 workers over one fd.
- ○ Two `RmError` variants deliberately have **no wire form**: `ForeignHandle` (our own gate produces it before a request is written — a compromised child must not be able to claim our invariant broke) and `Wedged` (a child that could send it would be reporting that it never answered).
- **Cancel** travels out-of-band on a separate datagram socket, guarded by a `txn` the worker publishes immediately before the verb and clears immediately after — without it a cancel races the completion and lands on an unrelated later operation.

6. Installing a mapping, and observing a completion

6.1 The mapping chain, end to end

The decision is made by a **publication pass**, not by a fault — we map ahead, we do not wait to be asked.

```
doorbell_publish_loop          (worker thread; mints the one OffVcpu)
→ publish_vas_rows(token, seen, OffVcpu) shim.rs:13259  - takes OffVcpu BY VALUE
→ vas_publish_census(...)      - produces candidate (va, len, phys)
→ join_one_fb_leaf(...)        shim.rs:15099
→ plane.fb_join_plan(phys, len) → FbJoinPlan::DeviceBacked { at: phys }
→ map_store_slice_for_leaf(...) shim.rs:14765
    step 0 decline if on a vCPU or in a trap      :14787
    step 1 vaspaces_handover → scratchpad dups the space (once per Vas)
    step 2 apply_ops(store_vas, [MapOp::Map{ va, gpga: at, len }])
→ StoreMapPort::map            storemap.rs:559
→ iso.with_worker(|w| w.with_rm(|rm| rm.map_store_slice(...)))
→ ProxyRmBackend::map_store_slice → write_frame / blocking read_frame (process boundary)
→ child: worker_loop → serve_one → HostRmBackend::map_store_slice
→ RmConnection::raw_map_dma_slice      rm.rs:4425
→ ioctl(NV_ESC_RM_MAP_MEMORY_DMA)      rm.rs:4509      ← THE SYSCALL
```

□ **The syscall happens in the scratchpad isolate child.** The VMM performs no RM ioctl for a store slice; it writes and reads one socket frame.

Two details that are load-bearing and easy to lose: - ∅ The code takes **RM’s answer, not its own request** (done.placed.first()). The comment records that substituting the requested address here regressed the guest to NEVER RETIRED. - ∅ **Constraint 28:** if RM places the mapping anywhere other than where we asked, the mapping is **torn down** and the call refuses — a fixed map that silently moved is worse than none. - ∅ status == 0x51 on a FIXED map means *already mapped there* (occupancy, not capacity) and is re-reported as such.

Guest RAM takes a different route: the memfd arrives **at spawn on a fixed fd**, not on a request, so a future seccomp filter can hardcode it. The child mmap’s it and wraps it in an OS_DESCRIPTOR; ∅ RM pins those pages and releasing the mmap does **not** unpin them.

6.2 The single store

[CODE] Allocated at **PCI realize**, in the scratchpad, by a **bisecting probe**: halve to a floor, then bisect to 64 MiB grain, each attempt allocated and freed. First try contiguous + 1 GiB aligned; fall back to non-contiguous + 4 KiB, latching which — and that latch is what later picks the page-size flag for every mapping.

[MEASURED] **11904 MiB**. □ Not a constant anywhere: the compiled advertisement is 12288 MiB and is **rebound to whatever the probe reserved**. □ The identity guest FB offset == store offset is only safe *because* of that rebinding.

∅ Deliberately **not** alloc_vidmem: that verb is contiguous-with-alignment = len, so an 11.8 GiB request refuses on **fragmentation** — a refusal that is not about capacity, on the one call whose refusal is supposed to mean “there is not enough video memory.”

6.3 Completions — two mechanisms with opposite writers

path	who writes the value the guest polls	how we learn
GR / compute , forwarded	the GPU	we only <i>read</i> : epoll_wait at a 250 ms floor, woken early by an eventfd; the watch list is declared on the vCPU and swept on the observer thread
emulated CE , served locally	we do	our executor writes the guest’s own literal payload, at the address the guest’s own page tables resolve to, after the bytes moved

□ The completion-watch module **never writes, never raises, never resolves** — the capability it is handed is a reader, by construction. □ The CE writer is a **single-writer discipline** with a compile-fail test and a workspace-wide census, because the C had a second, lagging writer that corrupted a live payload.

∅ AWAKEN_ENABLE = 0 in the measured cuCtxCreate stream: the guest asked for **no interrupt at all**. The completion is a value appearing at an address. That is why polling — not interrupt delivery — is the mechanism here.

6.4 Interrupts — what gets announced

Two independent gates, both in `ring_doorbell`: - **non-stall vector**: only a **locally served** doorbell announces. A forwarded one does not — the work finished on a host engine at an instant this device was not standing at. - **OS-event wake**: fires for **served or served-locally**. [MEASURED w686] gating this on locally-served alone meant **40 of 44** completions on passthrough channels announced nothing and the guest sat in `MC_SERVICE_INTERRUPTS`.

⊗ `LEAF_TRIGGER` is the **guest's own** self-trigger register, not our announcement path. Masking is **recorded, not acted on**.

7. The proposal, and the arguments against it

This is the part to attack. Each item has the counter-argument I can see.

P1. Mirror, not adopt

There are two ways to make the GPU honour the guest's translations.

- **Adopt** — point the host channel's instance block at the **guest's own** page-directory root. No walk, no diff, no syscalls at all. Most of v2 evaporates.
- **Mirror** — walk the guest's tables, replicate into a host VA space.

I propose mirror, and I want this one argued. My objection to adopt: the guest's PTEs contain **guest** physical addresses, which are host-valid only where the backing is identity — true for the store, false for system GPAs. □ If adopt can be rescued (e.g. by making system identity too), it is strictly better and I would drop mirror.

P2. No address table; the host page tables are the model

Already argued in §5.3. The strongest evidence it is right: **the diff machinery already exists and is unwired**, and the reconciliation bugs that dominated 2026-09-19/20 are all of the form “*two copies of one fact disagreed.*”

⊗ **Argument against**: syscall volume. N changed pages $\Rightarrow$ N mappings unless runs are coalesced. **Unmeasured**. The walk is 205.7 μ s and page tables change far less often than doorbells ring, so I expect small bursty diffs — but that is a prediction.

P3. Lazy everywhere; strict only at recycle — and recycle is the guest's job

A doorbell **never** waits for a refresh. `ogkm` already accepts that addresses are *undefined* until an invalidate completes: old page or new page, never a page neither mapping referenced.

$\Rightarrow$ We own no scrub policy. Our single failure is that we **forge the scrub's completion** (`forwarded=0` on the kernel tokens), so the guest reuses a page it believes was cleared. **That is the cross-client leak**. Fix: let the work reach the GPU. The gate already exists.

P4. No RM verb on a vCPU, structurally

§4.4 shows the current design **sanctions** one blocking site and pays for it with a measured 1.62 s stall. **[PROPOSE]** make it impossible instead: every RM verb becomes an *offer*, and the guest waits on its own primitive.

⊗ **Argument against**: some verbs are genuinely synchronous in the guest's eyes — a control command whose reply the guest reads immediately. Those need an answer *now*. □ I do not have a clean answer for that case and it is the main hole in P4.

P5. Deletion order

1. **The scrub** — live security bug, gate exists, smallest change.
2. **Pdb-as-identity** — still costing diagnoses; split into `VasId` + `Option<Pdb>`.
3. **The address table and joins together** — they are one mechanism.
4. **The CPU executor last** — the scrub result tells us which half of that plane survives.

P6. What must NOT be deleted

The instruments. The doorbell ledger's per-token `forwarded=`, the censuses, refusals **by name**, the bare-metal baseline, the hardware gate. [MEASURED 2026-09-20] the bare-metal baseline is what turned nine guest failures from opinion into adjudication; without it, a rewrite cannot be graded, and a rewrite you cannot grade is a rewrite you cannot land.

8. Open problems — named, not designed away

1. **Where the walk's root comes from.** `MAP_MEMORY_DMA` is a HAL stub on GSP parts, so a client-allocated VA space is never declared — [MEASURED] **zero** `SET_PAGE_DIRECTORY` events in a whole boot. Candidates, ranked: the **instance block** (which we author, and which is how hardware itself finds the root); `COPY_SERVER_RESERVED_PDES` (arrives, but only for RM-managed spaces); observing the guest's own page-table writes.
2. **Fault delivery becomes load-bearing.** v2 lets the GPU fault instead of pre-validating operands — but guest fault-recovery methods are currently **refused by name** precisely because we do not deliver faults. That refusal is the marker for this work.
3. **The BQL question** (§0). It changes the concurrency argument wholesale.
4. **The 21 unranked vCPU-path locks** (§4.3).
5. **Multi-GPU and non-GA10x.** The chip table holds exactly one profile.
6. **Syscall cost of the diff** (P2).

Part 5 — The surface we present (v2 plan-level)

STATUS: LIVE, 2026-09-20 (w815). Companion to `THE_ARCHITECTURE_v2.md`.

∅ Every row below was read out of the tree at `w749-fable-legb`, with `file:line`. Where the survey could not establish something it says so — **an unverified row is marked `□`, never filled in**. This document is the inventory the v2 rewrite is scoped against: the **plan** column is the decision, and `[DELETE]` means the mechanism goes away, not that the behaviour does.

1. GSP RPCs

The wire table is generated from `rpc_global_enums.h` (`kayfabe-abi/src/generated/rpc.rs`); classification is `FunctionCodes::classify` (`kayfabe-gsp/src/rpc.rs:209`); the one dispatch is `translate()` (`kayfabe-rmrpc/src/lib.rs:1073`).

1.1 The object-model RPCs — KEEP, unchanged

`GSP_RM_ALLOC (0x67)`, `GSP_RM_CONTROL (0x4c)`, `FREE (0x0a)`, `DUP_OBJECT (0x15)`.

These four are the whole guest→us object protocol: the guest tells us what it allocated, what it freed, what it aliased, and what it wants done. We decode them into `RmEvents` and keep a graph of the guest's objects. **Plan: unchanged in v2**. This is the control plane, it is version-sensitive, it is heavily tested, and none of v2's deletions touch it. An unmapped alloc class is refused by name (`UnmappedAllocClass`) and that stays — a class we do not model is a thing we must not pretend to have created.

1.2 `UPDATE_BAR_PDE (0x46)` — KEEP, but it is the BAR2 root

Inert to the object model; acted on at `kayfabe-device/src/bar2.rs:299`, where it latches the root PDE that BAR2 translations are rooted at, and replies `NV_OK` with an empty body. **Plan: keep**. BAR2 is an instance-block window and the guest legitimately re-roots it. `□` This is one of the few places v2 still walks page tables *to answer a read*, rather than to produce a diff — see §3.4.

1.3 `SET_GUEST_SYSTEM_INFO (0x01)` / `_EXT (0x40)` — KEEP

The version handshake. We answer with the version we agreed to speak; the `_EXT` form is answered `NV_OK` with an empty body. The guest's firmware string is latched read-only. **Plan: keep** — this is where the support matrix is enforced, and it must stay strict.

1.4 `GET_GSP_STATIC_INFO (0x41)` — KEEP, forged by construction

We synthesise an entire `GspStaticConfigInfo` from the chip and driver tables (`staticinfo.rs:183`). There is no host value to forward: on real hardware this comes from GSP firmware we are replacing. **Plan: keep forged**. `□` This is the single largest fabricated structure we hand the guest and it is derived per-die; it is the first thing to re-derive when adding a chip.

1.5 The inert set — KEEP as inert, they are load-bearing

`INIT_GSP_TRACE_CRASH_BUFFER (0xe4)`, `UNLOADING_GUEST_DRIVER (0x2f)`, `GSP_SET_SYSTEM_INFO (0x48)`, `SET_REGISTRY (0x49)`, `ECC_NOTIFIER_WRITE_ACK (0xca)`.

Accepted, no state moves. ∅ Two of these matter more than “inert” suggests: `UNLOADING_GUEST_DRIVER` **requires a reply** — a missing one hangs `rmmod` — and the two `NoReply` ones must **not** be answered, because an unsolicited reply desynchronises the queue. **Plan: keep, with the reply disposition as the thing under test**.

1.6 Events we post — KEEP

`EVENT_GSP_INIT_DONE (0x1001)`, `EVENT_POST_EVENT (0x1003)`, `EVENT_RC_TRIGGERED (0x1004)`. We send these; a guest-sent one is refused `EventFromGuest`. **Plan: keep**. The refusal direction is a real boundary — the guest must not be able to inject its own completions.

1.7 Everything else — refused by name, and that is the design

Other(code) ⇒ UnknownFunction { code } — *replied to, never dropped*. Includes ids present in the generated table with no dispatch arm: NOP, ALLOC_ROOT, ALLOC_MEMORY, MAP_MEMORY_DMA (**0x0e**), UNMAP_MEMORY_DMA (0x0f), EVENT_GSP_RUN_CPU_SEQUENCER.

□□□ MAP_MEMORY_DMA is the important one. It is a HAL stub on GSP-client parts, so the guest never sends it — which is why [measured w811] a whole boot carries **zero** SET_PAGE_DIRECTORY events and a client-allocated VA space is never declared to us. **Plan: this is open problem #3 in THE_ARCHITECTURE_v2.md §7**. v2 needs a root to seed a walk and this RPC will not provide it.

2. RM control commands

2.1 Page-directory controls — [DELETE the concept, keep the decode]

cmd	name	today
0x00801813	DMA_SET_PAGE_DIRECTORY	→ RmEvent::SetPageDir
0x90f10106	VASPACE_COPY_SERVER_RESERVED_PDES	→ RmEvent::SetPageDir
0x20800a9f	GMMU_COPY_..._PDES_TO_SERVER	same params as above
0x00801814	DMA_UNSET_PAGE_DIRECTORY	refused PageDirControlNotModelled

These tell us a VA space’s page-directory base. **Plan: keep decoding them, but demote what the answer is used for.** In v2 a root is needed **only to seed a walk** — never as an address space’s identity. [ROT] today Pdb is both, and unwrap_or(Pdb(0)) at eight sites makes Pdb(0) mean “declared at offset 0” and “not declared”; that collision cost four wrong diagnoses in one session. □ Note both arms refuse a **sysmem** root today (SetPageDirRootAperture / PublishedPdesRootAperture) even though a real GA106 was measured with one — that refusal is on the v2 review list.

2.2 GPU_PROMOTE_CTX (0x2080012b) — KEEP

The guest tells us where it placed a context buffer. Decoded, ≤16 entries, entryCount > 16 refused. **Plan: keep.** □ The C rounded every promote-derived mapping **up to 64 KiB** (asize = (size + 0xffff) & ~0xfffffull); this port binds at the declared length and that difference produced sub-page holes. Under v2 the mapping comes from the **walk**, not from the promote, so this becomes an observation rather than a mapping source — which dissolves the rounding question instead of answering it.

2.3 Channel controls served by ObjectPolicy — KEEP

GPFFIFO_SCHEDULE (0xa06f0103 / 0xa06c0101), BIND (0xa06f0104), SET_CTXSW_PREEMPTION_MODE (0x20801210), MC_SERVICE_INTERRUPTS (0x20801702), GET_MMU_FAULT_INFO (0x906f0106), PREEMPT (0xa06c0105).

The channel lifecycle: schedule it, bind it to an engine, preempt it, ask why it faulted. **Plan: keep all.** ∅ GET_MMU_FAULT_INFO is **relied to the host channel verbatim, never synthesised**, and that becomes *more* important in v2: if the GPU faults because our diff was late, the guest must receive the hardware’s own answer.

2.4 The 47 forged tables — KEEP forged, but they are a per-die liability

FIFO_GET_DEVICE_INFO_TABLE, INTERNAL_GPU_GET_CHIP_INFO, CE_GET_CE_PCE_MASK, FB_GET_INFO_V2, GR static caps, the cudart-init group, and ~40 more (kayfabe-device/src/inittables.rs:1085). Each is answered with a struct built from our chip and driver tables; a paramsSize mismatch is refused.

Plan: keep, and derive per die rather than maintain per die. □ This is the largest correctness surface we own and the one with the worst oracle: [the C artifact] 11 of 56 captured rows had **aten = 0** and every empty row checked against real hardware was **contradicted**. CE_GET_FAULT_METHOD_BUFFER_SIZE decoded from an empty row as **0** where a real GA106 returns **20480** — and RM DMAs fault records into a buffer of that size, so trusting it was a buffer overrun with a hardware writer.

2.5 Input-only controls — KEEP

NV2081_BINAPI, DEBUG_SET_EXCEPTION_MASK, SET_TIMESLICE, two CUDA-limit controls. The ack is the whole verb; we echo the guest’s own bytes behind a size check. **Plan: keep.**

2.6 Denied by capability — KEEP DENIED

IMPORT_MEM, DISABLE_IMPORTERS, NVLINK_GET_PLATFORM_INFO (fabric management); GPU_EXEC_REG_OPS, NVB0CC_EXEC_REG_OPS (raw register access); REPORT_NON_REPLAYABLE_FAULT; the SM-debugger trapping set; ALLOC_PMA_STREAM (performance counters).

Plan: keep denied, permanently. These are the guest asking for a capability the host process does not have and must not acquire. □ This is a *capability allowlist* (160 rows), not a served set: admitted-but-undecoded still refuses GspRuleControlUnserviced. □ The survey did not transcribe all 160 rows — that is a mechanical dump if the full admitted set is ever needed.

3. MMIO registers

□ **GA106 only.** The chip table holds exactly one profile; ad10x, gh100, gb20x exist in kayfabe-chips and are not in it. □ The BAR0 aperture is 16 MiB and ~87% of its pages hold no register at all — “everything else ⇒ Unclaimed” is the complete statement for the remainder.

3.1 Boot constants — KEEP

NV_PMC_BOOT_0/1/42, NV_PTIMER_TIME_PRIV_LEVEL_MASK, NV_USABLE_FB_SIZE_IN_MB, NV_XVE_LINK_CAPABILITIES. Constants the driver reads to identify the part. **Plan: keep, per-die.** ∅ One is an advertised fiction — the access-counter notify buffer size carries an entry count only.

3.2 The GSP boot FSM — KEEP

GFW_BOOT_PROGRESS/PLM, the PGSP/PSEC falcon registers (CPUCTL, HWCFG2, DMATRFCMD, MAILBOX0/1, IRQS-TAT/MASK/DEST/SCLR), NV_PRISCV_*, WPR2_ADDR_LO/HI.

This is the fake GSP’s entire visible boot: the driver pokes STARTCPU, watches progress reach 0xFF, reads mailboxes for boot args, and finds WPR2 where it expects. **Plan: keep.** ∅ WPR2 state only resets on a full QEMU restart, which is why each clean run needs a fresh boot.

3.3 Queue doorbells and the interrupt tree — KEEP

NV_PGSP_QUEUE_HEAD(i) (0x110C00 + i*8) is the command doorbell: posted to a worker, no state touched inline. CPU_INTR_LEAF(i) / EN_SET / EN_CLEAR, CPU_INTR_TOP*, and CPU_INTR_LEAF_TRIGGER (write-only, latches a vector and raises the CPU interrupt).

Plan: keep. ∅ [measured w684] 40 of 44 completions were never announced (no_engine); the interrupt **arming** model is the live design question here, not the register decode.

3.4 The MMU invalidate trio — [DELETE most of what it triggers]

MMU_INVALIDATE_PDB (0xB830A0) and _UPPER_PDB latch; MMU_INVALIDATE (0xB830B0) **triggers** a publication, and the guest **spin-polls** its TRIGGER bit for completion.

Plan: keep the register, delete the publication behind it. In v2 an invalidate is a hint that the guest’s tables changed — a good moment to walk and diff — and **nothing waits on it** (§1: the guest already tolerates undefined-until-complete). ∅ Today this register is the entry point to the epoch/dirty-gate/publication machinery that §3 of the architecture deletes. □ RM cannot express a narrow invalidate (ALL_VA is hard-coded) and its own timeout is **4 s**, after which it proceeds with stale TLBs — so the guest is *already* designed for us to be slow here.

3.5 The framebuffer windows — [DELETE]

NV_PBUS_BAR0_WINDOW (0x1700) plus the PRAMIN moving window (0x700000–0x7FFFFFFF); BAR1 (bus_bar::FB) and BAR2 (bus_bar::INST), both GMMU-translated on access; FbTrapPolicy.

Plan: BAR1/BAR2 become device views of the one store and stop being translated windows; PRAMIN stays as the bring-up aperture it is, not a running path. ∅ [measured] vidmem CPU reads are **~48 MiB/s flat** — any design that routes real traffic through these is slow by construction, which is precisely why the store exists.

3.6 The doorbell — KEEP, it is the hot path

NV_VIRTUAL_FUNCTION_DOORBELL (BAR0 0x00BB0090). Rung off-lock; drives channel submission. NV_VIRTUAL_FUNCTION_TIME_0/1 beside it are the PTIMER: read-only, **writes refused by name** (the guest reads the host’s counter and may not move it).

Plan: keep all three exactly. ∅ §41 bounds what this trap may do: update a queue, wake a worker, one dword for a passthrough doorbell, or a synchronous read-register write. Nothing else, ever.

4. Channel / pushbuffer operations

Decoder: `Gal0xPushbuffer` (kayfabe-chips/src/gal0x.rs:1595); vocabulary `PushMethod` (kayfabe-arch/src/lib.rs:949); consumer `apply_pushbuffer` (kayfabe-fwd/src/lib.rs:8721).

4.1 EMULATED — we implement the effect

method	addr	why it is ours
SET_OBJECT	0x0000	pure bookkeeping — binds a subchannel class, clears its slots
host-FIFO semaphore run (SEM_ADDR_LO/HI, SEM_PAYLOAD_LO/HI, SEM_EXECUTE)	0x5C–0x6C	RELEASE only ; acquires and reduction decode <code>Opaque</code>
MEM_OP_A..D TLB invalidate	0x28...	recorded and censused; PDB_ALL and non-invalidate ops $\Rightarrow$ <code>Opaque</code>
LAUNCH_DMA with DATA_TRANSFER_TYPE_NONE	0x0300	a release carrying no copy — there is no transfer to forward

Plan: keep all four emulated. Each is either bookkeeping or a completion, never a byte movement. $\emptyset$ In v2 the TLB-invalidate record becomes a *walk trigger* and loses its publication side effect (§3.4).

4.2 TRANSLATED — operands rewritten, the GPU moves the bytes

LAUNCH_DMA (0x0300) is the only true member, fed by fourteen latched operand slots: `OFFSET_IN/OUT_UPPER/LOWER`, `LINE_LENGTH_IN`, `LINE_COUNT`, `SET_SRC/DST_PHYS_MODE`, `SET_REMAP_CONST_A/B`, `SET_REMAP_COMPONENTS`, `SET_SEMAPHORE_A/B/PAYLOAD/_UPPER`.

Today we re-resolve `src/dst` through the issuing channel’s **address table** and partition into spans. **Plan: [DELETE the translation]**. Under v2 the guest’s VA is the host’s VA (route K) and the store is identity-mapped, so there is nothing to rewrite — the GPU resolves the operands through the page tables we applied, and faults if we were wrong. $\emptyset$ This deletes `TableOperands`, the span partitioning, the operand gate, and `CeExecutor::Ours` with it.

$\square$ `SET_SRC/DST_PHYS_MODE` is the one that must **not** be waved through: `PHYSICAL` bypasses the MMU, and a physical-mode copy passed straight to hardware is an escape class. It stays inspected — that is exactly what a **Translated** channel is for, and why `Passthrough` is not allowed to carry it.

$\emptyset$ `SET_REMAP_COMPONENTS` is **required, never defaulted** — an unlatched remap refuses the launch. Keep that; a defaulted remap silently writes the wrong bytes.

4.3 REFUSED — the fault tripwire

`GP100_UVM_SW_FAULT_CANCEL_A/B/C` (0x0104/0108/010C) and `CLEAR_FAULTED_A/B` (0x0110/0114) abort the whole parse with `UvmFaultMethod-WithoutFaultDelivery`. `SET_OBJECT/NO_OPERATION` on the same class pass through as routine.

Plan: keep refusing — and revisit when v2 lands. These say the guest is doing fault *recovery*, which presumes a fault-delivery mechanism we do not implement. $\square$ v2 deliberately lets the GPU fault (§3), so fault delivery moves from “not modelled” to “on the critical path”, and this refusal is the marker for that work.

4.4 PASSED THROUGH

Everything else $\Rightarrow$ `PushMethod::Opaque`, counted, untouched. `Non-Incrementing` method forms (`NON_INC`, `ONE_INC`, `IMMD_DATA`) are deliberately not latched. **Plan: keep.** $\emptyset$ Parsing — not placement — is what is forbidden on a passthrough ring; this is the rule in code.

5. $\square$ Where this inventory is knowingly incomplete

1. The 160-row capability allowlist is not transcribed (gates admission, not service).
2. Registers are **GA106 only**; three other chip modules exist and were not enumerated.
3. There is no enumerable table of *unhandled* BAR0 offsets — ~87% of pages are Unclaimed.
4. `kayfabe-rt/src/ceutils.rs` holds a **second** `PushMethod` consumer; it was confirmed to use the same enum but **not** audited for additional method addresses.

5. `kayfabe-isolate-host/src/rm.rs` *emits* CE pushbuffers using the same constants; classified as a producer, not audited as a decoder.
6. Roughly a dozen `kayfabe-device` modules were not read in depth; two of them gate on `RpcFunction::RmControl` and may reference control ids not listed here.

Part 6 — The scrub question (open owner ruling)

STATUS: OPEN — awaiting an owner ruling on §46 vs §12.26. Measured 2026-09-20 (w813) on a GA106 / 580.159.04 guest, branch w749-fable-legb. Nothing here has been implemented; the `SystemDataPlane` rule is untouched.

What is measured

The `DOORBELL-LEDGER`, per token, on a thin-guest run whose every ledger row verified:

token	class	emulated	forwarded	reading (the ledger's own rule)
0x00000003-0x00000007	the guest's CE channels	1	5	reached hardware
0x00000009	guest	2	2	reached hardware
0x00010001	kernel / system	74	0	never reached hardware
0x00010004	kernel / system	66	0	never reached hardware

"forwarded=0 with emulated>0 means it never went to hardware; forwarded>0 means it did" — the ledger's own text.

⇒ **The guest's copy work runs on the GPU. The kernel's does not, and it is 140 doorbells.**

Why this is a decision and not a bug

The kernel tokens are `forwarded=0` **by design today**. `kayfabe_fwd::FwdFault::SystemDataPlane` implements `l1_concurrency.md §12.26`:

*"The `SYSTEM` proc has no data plane ... Guest-kernel work that would need a backing — **the CeUtils scrub**, the GR golden capture — is **forged** to the system proc's completion queue, **never forwarded**, so the system proc never mints host memory."*

□ §46 (owner, 2026-09-19) names that **exact** example and reverses it:

"okay no real work kernel channel except the no-op or the ones that we must stub, so the scrub and CE, is never executed on CPU. Its always executed on GPU under the single store" and, when I mis-paraphrased it: "Ceutils scrub is not a no-op, it genuinely needs to clear memory. Same for kernel ce".

Two rules, one workload, opposite answers. The date is part of the citation, and §46 is the later one — but §12.26 is documented as a boundary, so the supersession should be **stated** rather than inferred by whoever next touches the code.

The argument each way

For §46 superseding (my reading): - §12.26's stated *reason* is *"so the system proc never mints host memory"*. Under the single store that premise is obsolete: the store is **one reserved device-local object minted once at realize**. A guest-RAM pin against it mints nothing; it names an offset in something that already exists. - The scrub is real work with an observable end state — memory that must read as cleared. §46 is explicit that this is not a no-op we may fake. - The same reasoning already moved the guest's CE copies onto hardware, and nothing about the system proc makes its bytes different bytes.

Against / what to be careful of: - §12.26 is about a **lifetime regime**, not a byte count: the system proc's isolate exists for the device's own bring-up and is not torn down with any guest process. Handing it data-plane state is a new lifetime question — *what frees these pins, and when?* - `ce_executor_tree.md` (owner, 2026-08-07) separately rules that `ce_copy(0urs)` must keep refusing inside the isolate, calling it *"the security boundary refusing to leak guest memory into the sandbox, working as designed"*. ∅ That is a **different** refusal from `SystemDataPlane` and is **not** what this page proposes changing — but the two are adjacent and must not be conflated in a hurry.

If the ruling is "§46 wins", the work and its gate

1. `plan_pin_guest_ram` stops refusing `Gpu::SYSTEM_PROC` — and the lifetime question above gets an answer in the same change, not later.
2. The scrub's CE doorbell forwards like any other.

3. **The gate already exists and needs no new instrument:** tokens `0x00010001` and `0x00010004` must flip from `forwarded=0` to `forwarded>0` in the `DOORBELL-LEDGER`, and `run_fast_guest.sh`'s stranded-token check must be widened from guest tokens to all tokens. □ Until then that check is deliberately scoped to `tok=0x000000xx` **precisely so it cannot pre-empt this ruling** — a gate that asserted the answer would make the question unaskable.

What must not be read into this page

∅ This is not a claim that the scrub is broken. It is forged, deliberately, by a rule that was correct when written. The question is only whether the rule survives the single store.